



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ



FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

**DESARROLLO Y EVALUACIÓN COMPARATIVA DE UN PROTOTIPO WEB:
ARQUITECTURA MONOLÍTICA VS. MICROSERVICIOS CON DOCKER**

ASESOR

JOSÉ JAVIER CHIRÚ FIGUEROA

INTEGRANTE

ALFONSO BROWN (8-907-1818)

**TRABAJO DE GRADUACIÓN PARA OPTAR AL TÍTULO DE LICENCIATURA
EN DESARROLLO DE SOFTWARE**

AÑO 2026

DEDICATORIA

Quiero dedicar este proyecto, ante todo, a mi familia, quienes han sido el sustento más firme en cada paso de mi formación. Su apoyo constante, sus palabras de ánimo y el cariño que siempre me ofrecieron sin condiciones me dieron la motivación necesaria para continuar incluso cuando el camino se volvía difícil. Gracias por creer en mí más de lo que yo mismo lo hice en algunos momentos.

A mis amigos, cuyo acompañamiento convirtió esta etapa en una experiencia más llevadera y significativa. Compartir con ustedes las largas jornadas de estudio, las preocupaciones, las metas y también las pequeñas celebraciones hizo que este proceso tuviera un valor especial. Su amistad ha sido un refugio y una fuente de energía en los días complicados.

Dedico también este trabajo a los docentes que formaron parte de mi camino académico. Cada consejo, comentario y enseñanza que me brindaron ayudó a construir la base de mi crecimiento profesional. Gracias a su apoyo aprendí a mejorar, a cuestionarme y a seguir avanzando con responsabilidad dentro del campo tecnológico.

Finalmente, extendiendo esta dedicatoria a todas las personas que, de alguna forma, aportaron a este logro. A quienes me ofrecieron una palabra de aliento, un consejo oportuno o simplemente su compañía en los momentos necesarios. Gracias por ser parte de este camino y por ayudarme a llegar hasta aquí.

AGRADECIMIENTO

Agradezco, primeramente, a Dios por darme la oportunidad, la fortaleza y la claridad para culminar este proyecto; así como por acompañarme en cada etapa de mi crecimiento personal y profesional.

Deseo expresar mi más profundo agradecimiento a mi familia: a mi papa John Brown, mi madre Nelda Fernández y mi gran hermano John Eric Brown cuyo apoyo incondicional, paciencia y cariño han sido el motor que me impulsó a continuar incluso en los momentos más retadores. Su presencia y respaldo constante han sido fundamentales para avanzar con determinación.

Extiendo mi sincero agradecimiento al Ing. José Javier Chirú, mi asesor de tesis, por su orientación continua, su paciencia y su compromiso durante todo el desarrollo de este trabajo. Su experiencia, sus observaciones claras y su disposición para compartir conocimientos fueron esenciales para la consolidación de este proyecto. Su guía dejó una huella significativa en mi formación académica y profesional.

Asimismo, agradezco profundamente a todos los docentes de la carrera, quienes con su dedicación, experiencia y vocación de enseñanza han enriquecido mi aprendizaje. Sus clases, consejos y acompañamiento contribuyeron no solo a fortalecer mis capacidades técnicas, sino también a afianzar mis valores éticos y profesionales.

De igual manera, quiero expresar mi gratitud a mis compañeros y colegas, cuyo compañerismo, colaboración e intercambio de ideas hicieron más enriquecedor este proceso. Las discusiones, las aportaciones y el esfuerzo conjunto aportaron valor y significado a esta experiencia.

También expreso mi agradecimiento a la Universidad Tecnológica de Panamá por ofrecerme las herramientas, los espacios y el apoyo institucional que hicieron posible el desarrollo de este trabajo. Su ambiente académico fue clave para fortalecer mis conocimientos y avanzar en mi formación.

INDICE GENERAL

DEDICATORIA.....	ii
AGRADECIMIENTO.....	iii
INDICE GENERAL	iv
INDICE DE FIGURAS	ix
INDICE DE TABLAS	xi
INTRODUCCIÓN	xiii
Resumen (Abstract)	xv
Capítulo I: Contextualización y Enfoque Metodológico	16
1.1. Situación actual.....	17
1.2. Propuesta y mejora	17
1.3. Definición y alcance	18
1.4. Metodología	19
1.5. Técnica de investigación.....	19
1.6. Objetivo general.....	20
1.7. Objetivos específicos	20
Capítulo II: Marco Teórico	21
2.1. Concepto de arquitecturas monolítica y de microservicios: Definición y características principales.....	22
2.2. Historia y evolución de las arquitecturas de software: De las aplicaciones monolíticas a los microservicios.....	25
2.3. Impacto de las arquitecturas en el desarrollo de aplicaciones empresariales: Eficiencia, escalabilidad y mantenimiento.....	27
2.4. Fundamentos de contenedores y Docker: Introducción a la contenerización y su relación con los microservicios.....	30

2.5. Relación entre arquitecturas de software y rendimiento: Cómo influye la elección arquitectónica en el desempeño de las aplicaciones.	35
Capítulo III: Fundamentos Técnicos para la Implementación Arquitectónica	39
3.1. Principios de diseño en arquitecturas monolíticas y de microservicios: Comparación de patrones comunes de diseño y su impacto en el desarrollo, mantenimiento y escalabilidad de las aplicaciones.	40
3.2. Estrategias de comunicación entre componentes: API REST, mensajería y eventos.	43
3.3. Gestión de datos: Análisis de cómo se manejan las bases de datos en ambos enfoques.....	49
3.5. Aplicación práctica en el prototipo: Síntesis de los aspectos técnicos que guiarán el diseño del prototipo.	56
Capítulo IV: Diseño y Desarrollo del Prototipo	58
4.1. Objetivos del prototipo: Comparar rendimiento y escalabilidad entre arquitecturas monolítica y microservicios.....	59
4.2. Arquitectura del sistema: Implementación de módulos: login, gestión de productos y ventas.	59
4.3. Selección de tecnologías: Uso de Node.js, MongoDB, Docker y React.js. .	63
4.4. Lógica de negocio y comunicación: Desarrollo de funcionalidades y su integración.	63
4.5. Microservicios y Docker: Despliegue de microservicios en contenedores Docker.....	64
Capítulo V: Pruebas y validación.....	66
5.1. Definición de criterios de evaluación del prototipo: Evaluación de rendimiento, escalabilidad y consumo de recursos.....	67
5.2. Métodos de prueba con usuarios: Pruebas de carga utilizando Apache JMeter.	71
5.2.1. Alcance y escenarios ejecutados.....	72

5.2.2. Métricas registradas.....	72
5.2.3. Resultados por escenario	73
5.3. Pruebas de estrés: Simulación de múltiples usuarios concurrentes para medir rendimiento bajo alta demanda.	76
5.3.1. Escenarios y configuración de las pruebas.....	76
5.3.2. Resultados de estrés de ambas arquitecturas.	77
5.4. Recolección de datos y observaciones: Métricas clave: tiempo de respuesta, uso de recursos, etc.....	81
5.4.1. Procedimiento de recolección	81
5.4.2. Herramientas de monitoreo y fuentes de información.....	82
5.4.3. Métricas clave recolectadas.....	83
5.4.4. Observaciones técnicas.....	84
5.5. Análisis preliminar de desempeño: Comparación de resultados entre ambas arquitecturas.	86
5.5.1. Análisis de la fase de carga (250, 500 y 1000 usuarios).....	86
5.5.2. Análisis de la fase de estrés (2500, 5000 y 10 000 usuarios).....	89
5.5.3. Síntesis interpretativa	92
Capítulo VI: Evaluación y Resultados	93
6.1. Interpretación de los datos recolectados en las pruebas: Análisis de rendimiento y escalabilidad.....	94
6.1.1. Gráficas globales de rendimiento (nivel “sistema”)	94
6.1.2. Gráficas por operación (nivel “endpoint”).....	99
6.1.3. Gráficas de “escalabilidad efectiva”	102
6.1.4. Lectura integrada de las gráficas	103
6.1.5. Síntesis	104

6.2. Discusión sobre la eficiencia de las arquitecturas: Evaluación de la arquitectura monolítica frente a los microservicios.....	104
6.2.1. Rendimiento operativo	105
6.2.2. Escalabilidad y estabilidad	106
6.2.3. Eficiencia en el uso de recursos	107
6.3. Evaluación de la percepción del rendimiento en usuarios finales: Impacto en la experiencia de usuario.	108
6.3.1. Relación entre métricas técnicas y percepción humana del rendimiento	109
6.3.2. Experiencia subjetiva en operaciones críticas	110
6.3.3. Efecto de la latencia y la variabilidad (p95/p99) en la experiencia	111
6.3.4. Impacto en la percepción de estabilidad y disponibilidad	111
6.3.5. Conclusión del análisis	112
6.4. Conclusiones sobre la viabilidad de cada enfoque: Análisis de las ventajas y desventajas de ambas arquitecturas.....	112
6.4.1. Eficiencia y rendimiento	113
6.4.2. Escalabilidad y resiliencia	113
6.4.3. Consumo y distribución de recursos	114
6.4.4. Evaluación de viabilidad técnica y estratégica	115
6.4.5. Conclusión general de viabilidad	116
6.5. Recomendaciones para futuros desarrollos y mejoras: Optimización y posibles avances en la implementación.....	117
6.5.1. Optimización técnica del rendimiento	117
6.5.2. Mejoras en infraestructura, despliegue y monitoreo	118
6.5.3. Mejoras orientadas a la experiencia del usuario	119
CONCLUSIONES.....	120

BIBLIOGRAFÍA	122
--------------------	-----

INDICE DE FIGURAS

Ilustración 1. Diagrama comparativo de monolítica vs Microservicio.....	25
Ilustración 2. Diagrama línea de tiempo de la evolución de arquitecturas de software.....	27
Ilustración 3. Diagrama impacto de las arquitecturas en aplicaciones empresariales	30
Ilustración 4. Diagrama comparación máquinas virtuales contra contenedores	35
Ilustración 5. Interfaz de login.....	59
Ilustración 6. Interfaz de registro	60
Ilustración 7. Interfaz de creación de reseñas	60
Ilustración 8. Interfaz de carrito de compras.....	61
Ilustración 9. Interfaz de órdenes / historial de compras	61
Ilustración 10. Interfaz del flujo de navegación.....	62
Ilustración 11. Contenedores del prototipo desplegados con Docker Compose....	65
Ilustración 12. Tiempo promedio de respuesta vs. usuarios (Monolítica vs. Dockerizada).....	94
Ilustración 13. Latencia p95 vs. usuarios (Monolítica vs. Dockerizada).....	95
Ilustración 14. Latencia p99 (escala log) vs. usuarios	96
Ilustración 15. Throughput (req/s) vs. usuarios.....	97
Ilustración 16. Tasa de errores (%) vs. usuarios	98
Ilustración 17. Promedio por operación (barras agrupadas) – Escenario de carga (p.ej., 1000 usuarios).....	99
Ilustración 18. p95 por operación (barras agrupadas) – Escenario de estrés (p.ej., 5000 usuarios).....	100
Ilustración 19. Errores por operación (barras) – Escenario 10 000 usuarios	101

Ilustración 20. Degradación relativa del tiempo promedio (Índice = 1 a 250 usuarios).....	102
Ilustración 21. Razón Docker/Monolítica del p95 (línea)	103

INDICE DE TABLAS

Tabla 1. Tabla comparativa de ambas arquitecturas	23
Tabla 2. Comparativa del impacto en aplicaciones empresariales	29
Tabla 3. Comparativas máquinas virtuales contra contenedores	34
Tabla 4. Comparativa: Rendimiento según arquitectura	38
Tabla 5. Métricas de rendimiento utilizadas en las pruebas	67
Tabla 6. Criterios de evaluación del rendimiento del sistema.....	69
Tabla 7. Parámetros de monitoreo de recursos del sistema.....	70
Tabla 8. Configuración de los escenarios de prueba.....	71
Tabla 5.2-A. Resumen de métricas (Escenario 250 usuarios – Monolítica).....	73
Tabla 5.2-B. Resumen de métricas (Escenario 500 usuarios – Monolítica).....	73
Tabla 5.2-C. Resumen de métricas (Escenario 1000 usuarios – Monolítica).....	74
Tabla 5.2-A. Resumen de métricas (Escenario 250 usuarios – Dockerizada) ...	74
Tabla 5.2-B. Resumen de métricas (Escenario 500 usuarios – Dockerizada) ...	75
Tabla 5.2-C. Resumen de métricas (Escenario 1000 usuarios – Dockerizada) .	75
Tabla 9. Escenarios de pruebas de estrés	76
Tabla 5.3-A. Resultados de estrés – Arquitectura Monolítica (2500 usuarios)...	77
Tabla 5.3-B. Resultados de estrés – Arquitectura Monolítica (5000 usuarios)...	78
Tabla 5.3-C. Resultados de estrés – Arquitectura Monolítica (10 000 usuarios)	78
Tabla 5.3-A. Resultados de estrés – Arquitectura Dockerizada (2500 usuarios)	79
Tabla 5.3-B. Resultados de estrés – Arquitectura Dockerizada (5000 usuarios)	79
Tabla 5.3-B. Resultados de estrés – Arquitectura Dockerizada (10 000 usuarios)	80
Tabla 10. Herramientas y métricas utilizadas en la evaluación del sistema	82
Tabla 11. Métricas de rendimiento, escalabilidad y consumo de recursos	83

Tabla 12. Comparativa de rendimiento entre arquitecturas.....	87
Tabla 13. Comparativa de rendimiento en escenarios de estrés.....	89
Tabla 14. Comparación integral entre ambas fases	91

INTRODUCCIÓN

En el panorama actual del desarrollo de software, las organizaciones enfrentan decisiones arquitectónicas cruciales que impactan directamente en la escalabilidad, mantenibilidad y rendimiento de sus aplicaciones. La elección entre una arquitectura monolítica tradicional y una basada en microservicios representa uno de los dilemas fundamentales para equipos técnicos y líderes de desarrollo.

Las arquitecturas monolíticas han sido ampliamente utilizadas durante décadas, destacando por su simplicidad en términos de diseño, implementación y despliegue. Al integrar todos los componentes funcionales de una aplicación en una sola base de código y ejecutarse como un único proceso, ofrecen una comunicación interna eficiente y una gestión más directa de la lógica de negocio. Esto facilita el desarrollo inicial, simplifica las pruebas y permite un despliegue unificado. Sin embargo, a medida que las aplicaciones crecen en tamaño y complejidad, este modelo comienza a presentar limitaciones significativas. La imposibilidad de escalar, individualmente, sus componentes; la dificultad para mantener una base de código extensa y el riesgo asociado al despliegue completo ante cualquier cambio representan obstáculos relevantes para su evolución.

Ante este panorama, ha surgido la arquitectura de microservicios como una alternativa orientada a mejorar la escalabilidad, la resiliencia y la flexibilidad del desarrollo. Este enfoque divide la aplicación en múltiples servicios pequeños e independientes, cada uno encargado de una funcionalidad específica del sistema, y que se comunican entre sí mediante APIS o mecanismos de mensajería. Esta separación permite a los equipos trabajar de forma paralela en diferentes módulos, escalar de manera puntual los componentes más exigidos y desplegar actualizaciones sin afectar al sistema completo. La contenerización mediante herramientas como Docker ha facilitado la adopción de este modelo, al proporcionar entornos consistentes, portables y ligeros para el despliegue de cada microservicio junto con sus dependencias, lo que ha optimizado los flujos de trabajo DevOps y ha reducido los tiempos de entrega de software.

Este estudio comparativo busca evaluar, objetivamente, ambos enfoques arquitectónicos mediante métricas cuantitativas y cualitativas en un escenario de aplicación real. Al analizar el comportamiento de sistemas idénticos implementados bajo estos dos paradigmas, pretendemos ofrecer datos empíricos que ayuden a los equipos de desarrollo a tomar decisiones fundamentadas según sus requisitos específicos, volumen de carga esperado y recursos disponibles.

Resumen (Abstract)

En esta tesis se realiza un estudio comparativo entre una arquitectura de software monolítica tradicional y otra basada en microservicios dockerizados, con el objetivo de evaluar las diferencias de rendimiento, escalabilidad y uso de recursos en el contexto de una aplicación de comercio electrónico. Para ello, se desarrolló un prototipo de aplicación de comercio electrónico utilizando Node.js y Express en el servidor, React en el cliente y MongoDB Atlas como base de datos en la nube. Se implementaron dos versiones de la aplicación (monolítica y basada en microservicios) y ambas se desplegaron mediante contenedores Docker. La metodología experimental incluyó pruebas de carga progresiva y de estrés empleando Apache JMeter, midiendo métricas de rendimiento (tiempos de respuesta y peticiones por segundo) y examinando la escalabilidad y el consumo de recursos (CPU, memoria) bajo distintos niveles de usuarios concurrentes, lo que permitió analizar las diferencias técnicas y operativas de ambas arquitecturas y destacar sus ventajas e inconvenientes según el nivel de carga. Los resultados obtenidos indican que la arquitectura de microservicios ofrece una mayor capacidad de escalado horizontal y mantiene un rendimiento más estable bajo cargas elevadas, a costa de un mayor consumo de recursos y una mayor complejidad operativa, mientras que, la arquitectura monolítica presenta un desempeño adecuado en escenarios de baja a moderada concurrencia, con menor sobrecarga inicial y mayor simplicidad en el despliegue, aunque experimenta cuellos de botella y degradación del rendimiento ante picos de demanda.

En conclusión, se evidencia que cada enfoque arquitectónico resulta más adecuado según el contexto de uso: los microservicios brindan flexibilidad, escalabilidad y resiliencia para aplicaciones de alta demanda, mientras que un monolito puede ser eficiente y sencillo de gestionar en entornos de menor escala.

Capítulo I: Contextualización y Enfoque Metodológico

1.1. Situación actual

En la actualidad, muchas organizaciones del sector público y privado enfrentan una creciente presión para modernizar sus sistemas tecnológicos y responder con agilidad a las demandas de los usuarios. Instituciones bancarias, empresas de comercio electrónico, entidades gubernamentales y startups tecnológicas requieren soluciones que no solo sean funcionales, sino también escalables, seguras y fácilmente mantenibles. En este entorno, aún persisten numerosos sistemas heredados con arquitecturas monolíticas que dificultan la implementación de mejoras, la integración con nuevas tecnologías y la adopción de modelos de despliegue continuo. Al mismo tiempo, existe un interés creciente por adoptar arquitecturas más flexibles como los microservicios, que permitan una evolución progresiva de las plataformas digitales sin comprometer la estabilidad ni aumentar excesivamente los costos operativos.

1.2. Propuesta y mejora

Este proyecto plantea el desarrollo de una aplicación funcional construida bajo dos enfoques arquitectónicos distintos: uno utilizando una estructura monolítica tradicional y otro basado en microservicios implementados en contenedores Docker. A través de esta propuesta, se busca generar un entorno controlado que permita comparar ambos modelos desde una perspectiva técnica, aplicando métricas relevantes relacionadas con el rendimiento, el consumo de recursos y la escalabilidad. Con el fin de medir objetivamente el comportamiento de cada arquitectura bajo diferentes niveles de carga, se utilizará Apache JMeter como herramienta principal para la ejecución de pruebas de rendimiento. Esta permitirá simular múltiples usuarios concurrentes, recopilar estadísticas clave y visualizar el impacto de las solicitudes sobre los recursos del sistema.

Esta propuesta no pretende establecer un modelo arquitectónico superior, sino ofrecer información empírica y contextualizada que permita evaluar de forma crítica las características, ventajas y limitaciones de cada enfoque, facilitando la toma de

decisiones fundamentadas en futuros proyectos de desarrollo y modernización tecnológica.

1.3. Definición y alcance

El presente proyecto consiste en el desarrollo de una aplicación prototipo implementada bajo dos paradigmas arquitectónicos distintos: una versión monolítica tradicional y otra basada en microservicios desplegados en contenedores Docker. El objetivo principal es establecer una base funcional equivalente entre ambas implementaciones que permita realizar una comparación técnica fundamentada a través de pruebas de rendimiento, consumo de recursos y capacidad de escalado.

La evaluación será llevada a cabo utilizando Apache JMeter, una herramienta ampliamente empleada en pruebas de carga, que permitirá simular diferentes niveles de usuarios concurrentes y extraer métricas cuantitativas sobre el comportamiento de cada arquitectura en condiciones similares. El análisis de los resultados busca proporcionar una visión objetiva de las implicaciones técnicas que representa la adopción de uno u otro enfoque, sin proponer una solución definitiva ni recomendar una arquitectura específica.

El alcance de este proyecto incluye:

- Implementación de dicha aplicación bajo dos modelos arquitectónicos: monolítico y microservicios con Docker.
- Ejecución de pruebas de rendimiento utilizando Apache JMeter, incluyendo simulación de múltiples usuarios y distintos niveles de carga.
- Recolección y análisis de métricas como tiempo de respuesta, uso de CPU y memoria, comportamiento ante la escalabilidad, entre otras.

Sin embargo, este proyecto NO incluirá:

- La implementación de funcionalidades avanzadas fuera del alcance de un entorno prototipo, ni la integración con servicios de terceros o bases de datos en producción.

1.4. Metodología

Para el desarrollo de este proyecto, se adoptará una metodología ágil basada en el marco de trabajo SCRUM, que permitirá una planificación flexible y una adaptación continua a los requerimientos del prototipo. SCRUM facilita un proceso iterativo e incremental, donde el desarrollo se organiza en sprints de duración determinada, cada uno con objetivos específicos claramente definidos. Durante estos sprints, se abordará la construcción de los módulos funcionales del sistema, tales como la autenticación de usuarios, la gestión de productos y el procesamiento de ventas, asegurando que dichas funcionalidades se implementen de manera equivalente en ambas arquitecturas —monolítica y de microservicios— para mantener condiciones homogéneas de evaluación.

Al finalizar cada sprint, se llevarán a cabo reuniones de revisión para evaluar el progreso, identificar posibles obstáculos y ajustar la planificación de acuerdo con los avances y necesidades emergentes del proyecto. Este enfoque permitirá mantener un control continuo sobre el desarrollo del prototipo, garantizando entregas parciales funcionales que puedan ser puestas a prueba de forma constante.

1.5. Técnica de investigación

La principal técnica de investigación utilizada en este proyecto será la **prueba de rendimiento automatizada**, específicamente, a través de la simulación de usuarios concurrentes utilizando la herramienta **Apache JMeter**. Este tipo de prueba permite evaluar de forma cuantitativa el comportamiento de las aplicaciones web ante diferentes niveles de carga, sin requerir la intervención directa de usuarios reales.

1.6. Objetivo General

Desarrollar y evaluar, comparativamente, una aplicación web prototipo implementada en arquitectura monolítica y microservicios contenerizados en Docker, analizando su rendimiento, capacidad de escalado y eficiencia.

1.7. Objetivos específicos

- Realizar una revisión bibliográfica sobre las arquitecturas monolítica y de microservicios, enfocándose en su rendimiento, escalabilidad y eficiencia.
- Diseñar la estructura funcional de una aplicación web prototipo con funcionalidades clave, como autenticación, gestión de productos y procesamiento de transacciones.
- Implementar la aplicación con una arquitectura monolítica, siguiendo buenas prácticas de desarrollo.
- Implementar la misma aplicación con arquitectura de microservicios, utilizando contenedores Docker para el despliegue de servicios independientes.
- Aplicar pruebas de rendimiento y escalabilidad utilizando Apache JMeter para obtener métricas clave.
- Analizar y comparar los resultados de ambas implementaciones en términos de eficiencia y capacidad de escalado.

Capítulo II: Marco Teórico

2.1. Concepto de arquitecturas monolítica y de microservicios: Definición y características principales.

Arquitectura Monolítica

Una arquitectura monolítica agrupa todos los componentes de la aplicación en un único artefacto de despliegue (por ejemplo, un solo servidor o proceso). En sus inicios, este modelo ofrece **simplicidad**: la aplicación es más sencilla de desarrollar y probar inicialmente, pues todos los módulos se ejecutan en el mismo entorno y pueden comunicarse mediante llamadas directas en memoria. Esto conlleva menos sobrecarga de comunicación y una cohesión fuerte, facilitando el arranque del proyecto con un esfuerzo cognitivo menor. Sin embargo, a medida que el sistema crece en tamaño y complejidad, surgen importantes **limitaciones**. Al ser una sola unidad, la aplicación monolítica **no permite escalar componentes individualmente** para soportar más carga se suele tener que escalar toda la aplicación a la vez, desperdiciando recursos en partes poco utilizadas. Además, **los despliegues son riesgosos y lentos**, ya que un cambio menor requiere volver a desplegar todo el sistema. Esto implica que una actualización de una pequeña parte puede introducir errores que afecten a toda la aplicación, comprometiendo la estabilidad. Otra desventaja es la **dificultad de mantenimiento**: con el tiempo la base de código monolítica se vuelve muy grande y entrelazada, haciendo más complejo agregar nuevas funcionalidades o corregir errores sin impactar otros módulos. Pese a ello, en proyectos pequeños o etapas iniciales, la simplicidad de un monolito puede ser ventajosa, evitando la sobrecarga inicial de orquestar múltiples servicios.

Arquitectura de Microservicios

La arquitectura de microservicios propone desacoplar la aplicación en **servicios independientes**, cada uno encargado de un **contexto de negocio** específico (por ejemplo, servicio de usuarios, servicio de pagos, servicio de catálogo de productos, etc.). Cada microservicio tiene su propia lógica y usualmente su propia base de datos, y se comunica con otros a través de **APIs o mensajes** de red en lugar de

llamadas directas en memoria. Este aislamiento aporta varias **ventajas claves**: cada servicio puede **desplegarse y escalarse de forma independiente**, permitiendo asignar más recursos solo a los componentes que enfrenten mayor carga sin tener que escalar toda la aplicación. También habilita ciclos de desarrollo y despliegue más ágiles es posible actualizar un microservicio específico y desplegarlo sin interrumpir el funcionamiento del resto del sistema, lo que facilita entregas continuas de software. En términos de resiliencia, si un microservicio falla, la aplicación global puede seguir funcionando (degradada en esa funcionalidad específica) en lugar de caer completamente, aportando **alta fiabilidad** al aislar fallos. Asimismo, los microservicios brindan **flexibilidad tecnológica**, ya que, cada servicio puede construirse con diferentes lenguajes o tecnologías según convenga, en lugar de verse limitado a un único stack para toda la aplicación. No obstante, este paradigma también conlleva desafíos: **no reduce la complejidad total, sino que la distribuye** en muchos componentes. La comunicación ahora es remota (vía red), añadiendo latencia y posibles fallos de red; y la gestión de numerosos servicios introduce sobrecarga en la orquestación, el monitoreo, la configuración y la comunicación entre equipos de desarrollo. En resumen, los microservicios ofrecen escalabilidad y agilidad superiores a costa de una mayor complejidad operativa, por lo que su adopción debe evaluarse según el tamaño del proyecto y las capacidades del equipo.

Tabla 1. Tabla comparativa de ambas arquitecturas

Aspecto	Arquitectura Monolítica	Arquitectura de Microservicios
Despliegue y Actualización	Un solo artefacto deployable; cualquier cambio requiere desplegar toda la aplicación nuevamente.	Múltiples servicios independientes; cada uno se puede desplegar o actualizar sin afectar al resto.

Escalabilidad	Escalado vertical principalmente (aumentar recursos del servidor monolítico); difícil escalar solo partes específicas.	Escalado horizontal por servicio (replicar solo los microservicios necesarios), permitiendo asignar recursos donde haya más carga.
Comunicación interna	Llamadas en memoria dentro del mismo proceso, muy rápidas y directas.	Llamadas remotas vía API/HTTP o mensajería; añaden latencia y requieren manejar fallos de red.
Aislamiento de fallos	Bajo un error crítico puede afectar a toda la aplicación (toda la aplicación cae).	Alto un fallo en un microservicio no derriba toda la plataforma, solo la funcionalidad de ese servicio.
Mantenibilidad	Con el tiempo, base de código extensa y acoplada; difícil de comprender y refactorizar en conjunto.	Cada servicio es más pequeño y acotado, facilitando su comprensión; aunque aumenta la complejidad global por la cantidad de componentes.
Tecnologías	Stack único para toda la aplicación (misma base tecnológica para todo).	Heterogeneidad posible cada microservicio puede elegir la tecnología o lenguaje óptimo para su responsabilidad.
Complejidad Operativa	Menor al inicio: una única aplicación que desplegar y monitorear.	Mayor necesidad de orquestación: se requieren herramientas

		para coordinar múltiples despliegues, registro de servicios, monitoreo distribuido, etc.
Tiempo de desarrollo inicial	Más rápido al comenzar, con menos configuración (un solo proyecto).	Inicialmente más lento de arrancar por configurar comunicaciones, infraestructura de múltiples servicios.

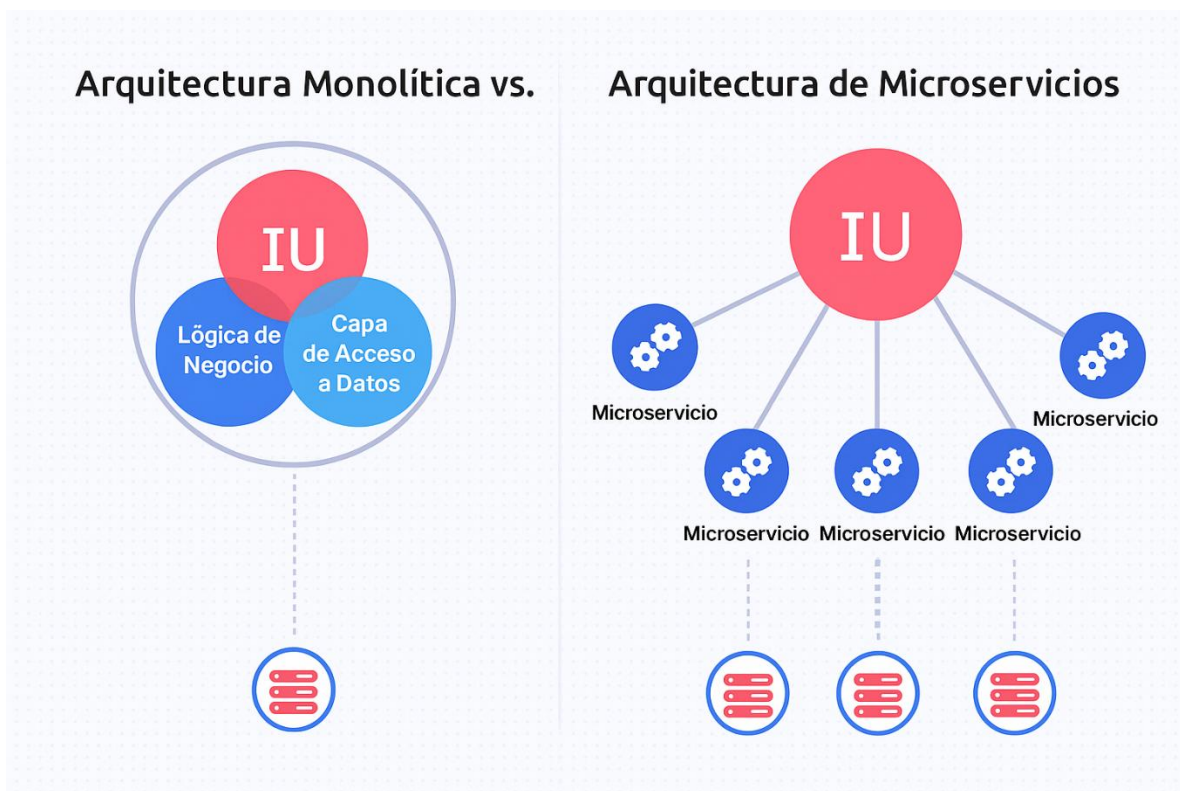


Ilustración 1. Diagrama comparativo de monolítica vs Microservicio

2.2. Historia y evolución de las arquitecturas de software: De las aplicaciones monolíticas a los microservicios.

La evolución de las arquitecturas de software ha estado estrechamente ligada a los avances en hardware, redes y metodologías de desarrollo. Cada etapa respondió a

los retos de su tiempo: desde sistemas centralizados, pasando por arquitecturas cliente-servidor, hasta llegar a los microservicios que hoy soportan las aplicaciones más exigentes.

1. Sistemas monolíticos iniciales (décadas de 1960–1980).

El software se ejecutaba en **mainframes** y estaciones centralizadas. La arquitectura era monolítica por naturaleza: todos los procesos (interfaz, lógica de negocio y almacenamiento de datos) estaban integrados en un solo programa. Esto simplificaba el control, pero limitaba la escalabilidad y requería hardware costoso.

2. Cliente-servidor y arquitecturas en capas (años 1990).

Con la masificación de las PC y las redes locales surgió el modelo **cliente-servidor**, donde las aplicaciones se dividían en dos capas: el cliente (interfaz) y el servidor (procesamiento de datos). Posteriormente, se consolidó la **arquitectura en tres capas** (presentación, lógica de negocio y datos), que permitió mayor modularidad y mantenibilidad.

3. Monolitos modernos y frameworks web (años 2000).

El auge de Internet y los frameworks (Java EE, .NET, PHP, Ruby on Rails) dio lugar a **aplicaciones web monolíticas** más estructuradas. Aunque seguían siendo un solo bloque de despliegue, ya contaban con capas bien definidas y mayor facilidad para construir aplicaciones complejas. Sin embargo, con el crecimiento de los usuarios y las funcionalidades, surgieron problemas de escalabilidad y despliegue.

4. Arquitectura orientada a servicios (SOA, 2000–2010).

Como respuesta, grandes organizaciones adoptaron la **SOA (Service Oriented Architecture)**, donde distintas funcionalidades se exponían como servicios independientes, normalmente sobre SOAP y buses de servicios empresariales (ESB). Aunque representó un paso hacia la popularización, SOA era complejo, pesado y de difícil adopción en entornos ágiles.

5. Microservicios (2010 en adelante).

Con la llegada de la **nube**, **DevOps** y la **contenerización**, nació la **arquitectura de microservicios**, donde cada funcionalidad se implementa como un servicio independiente, desplegado en contenedores (ejemplo: Docker). Estos servicios se comunican mediante APIS REST o mensajería ligera. Esto permite escalar, desplegar y mantener cada servicio de manera autónoma, dando lugar a aplicaciones más resilientes y flexibles.

6. Actualidad: convergencia y adopción híbrida.

Hoy en día, muchas empresas operan en modelos **híbridos**, combinando sistemas monolíticos heredados con microservicios modernos. El reto es decidir cuándo conviene migrar a microservicios y cuándo un monolito sigue siendo suficiente, dependiendo de la complejidad, recursos y objetivos del proyecto.

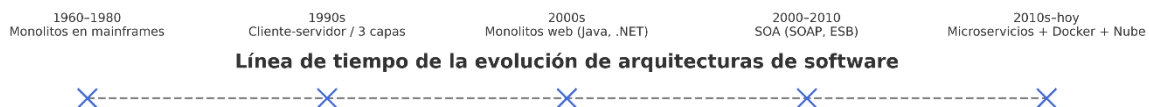


Ilustración 2. Diagrama línea de tiempo de la evolución de arquitecturas de software

2.3. Impacto de las arquitecturas en el desarrollo de aplicaciones empresariales: Eficiencia, escalabilidad y mantenimiento.

La elección arquitectónica en una aplicación empresarial determina en gran medida su capacidad para responder a la demanda del mercado, optimizar recursos y garantizar la continuidad operativa. Entre las más relevantes se destacan tres dimensiones clave: **eficiencia, escalabilidad y mantenimiento**.

Eficiencia

- **Arquitectura monolítica:**

Generalmente más eficiente en entornos iniciales debido a la comunicación interna directa (en memoria), lo que reduce la latencia. No obstante, a medida que la aplicación crece, la complejidad del código provoca ralentizaciones, y los tiempos de despliegue aumentan porque se debe reiniciar todo el sistema ante cada cambio.

- **Arquitectura de microservicios:**

Aunque introduce cierta sobrecarga por la comunicación vía red entre servicios, permite optimizar la eficiencia en la asignación de recursos. Cada microservicio puede ajustarse a las necesidades de su carga, evitando el uso innecesario de recursos en módulos poco utilizados.

Escalabilidad

- **Arquitectura monolítica:**

Su escalabilidad es principalmente **vertical** (aumentar CPU/RAM del servidor). Esto genera altos costos a medida que crece la demanda, y limita la flexibilidad, ya que, no se pueden escalar únicamente los módulos más exigidos.

- **Arquitectura de microservicios:**

Se caracteriza por la **escalabilidad horizontal**. Los servicios críticos (ejemplo: pagos en un e-commerce) pueden replicarse de forma independiente, garantizando que el sistema responda adecuadamente al aumento de usuarios concurrentes sin necesidad de duplicar toda la aplicación.

Mantenimiento

- **Arquitectura Monolítica:**

La base de código suele crecer de forma desordenada con el tiempo, volviéndose difícil de comprender y modificar. Los equipos de desarrollo tienen más riesgo de introducir errores al realizar cambios en funcionalidades específicas.

- **Arquitectura de Microservicios:**

El mantenimiento se distribuye: cada servicio tiene su propio ciclo de vida, repositorio y equipo encargado. Esto facilita la incorporación de nuevas funcionalidades y la corrección de errores. Sin embargo, requiere un mayor esfuerzo de coordinación entre servicios y una infraestructura de monitoreo adecuada.

Tabla 2. Comparativa del impacto en aplicaciones empresariales

Dimensión	Arquitectura monolítica	Arquitectura de microservices'
Eficiencia	Rápida comunicación interna pero baja flexibilidad a largo plazo.	Comunicación por red, pero uso optimizado de recursos por servicio.
Escalabilidad	Vertical, costosa y limitada.	Horizontal, granular y más económica a gran escala.
Mantenimiento	Complejo, base de código extensa y acoplada.	Distribuido, más ágil, pero con mayor complejidad de coordinación.

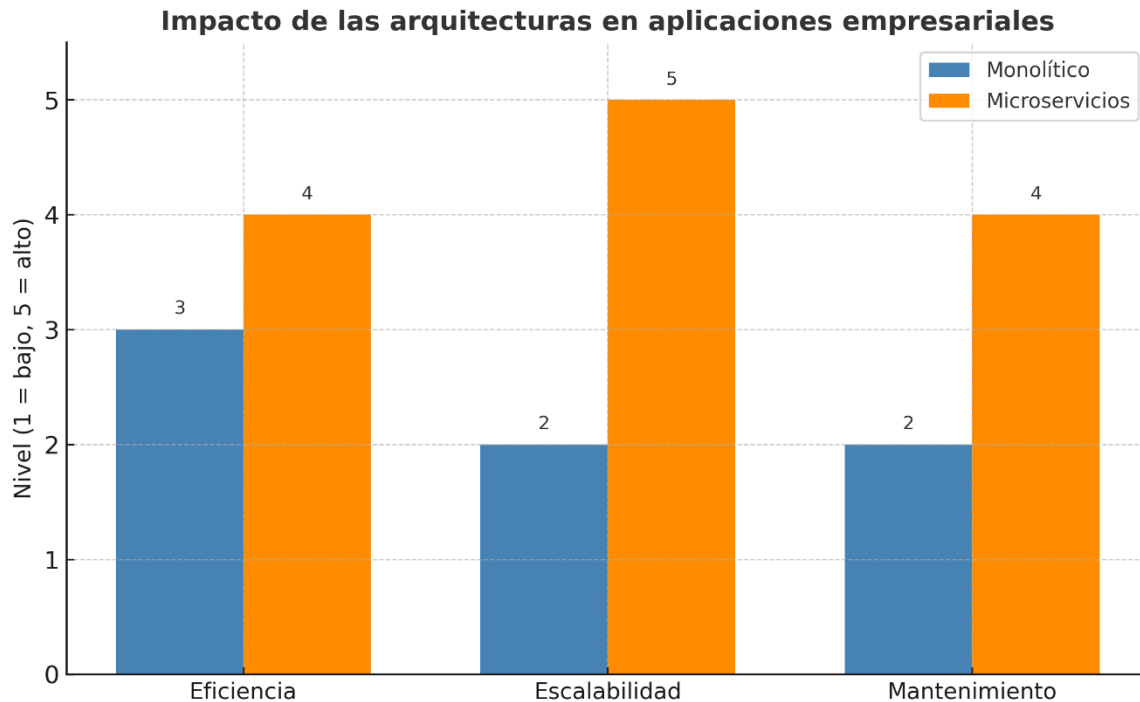


Ilustración 3. Diagrama impacto de las arquitecturas en aplicaciones empresariales

2.4. Fundamentos de contenedores y Docker: Introducción a la contenerización y su relación con los microservicios.

En el contexto de la arquitectura de microservicios, es fundamental contar con mecanismos para desplegar y ejecutar múltiples servicios de forma consistente. Aquí es donde entra **Docker**, la plataforma líder de **contenedores** de software. Un **contenedor Docker** es un paquete ligero que incluye una aplicación junto con todas sus dependencias (librerías, runtime, configuraciones) encapsuladas, de modo que pueda ejecutarse en cualquier entorno que tenga Docker sin conflictos. Docker ha revolucionado la forma de desplegar aplicaciones, ya que, garantiza que "funciona en mi máquina" sea reproducible en producción, al aislar cada servicio en su propio entorno inmutable.

La idea de contenerización aísla la aplicación del sistema operativo host, pero a diferencia de las máquinas virtuales tradicionales, los contenedores **no emulan todo un hardware** completo, sino que comparten el kernel del sistema operativo host, lo que los hace mucho más livianos. La **diferencia clave entre contenedores**

y máquinas virtuales es que una VM virtualiza una máquina entera (incluyendo sistema operativo invitado completo y hardware virtual) mientras que un contenedor solo virtualiza las capas de software sobre el OS, reutilizando el kernel del host. En consecuencia, **los contenedores son mucho más eficientes en recursos**: inician en segundos, ocupan megabytes en lugar de gigabytes, y permiten densidad elevada (muchos contenedores corriendo en un mismo servidor sin apenas sobrecarga). Esta ligereza se ilustra en la Figura 3, donde se ve que múltiples contenedores comparten el mismo OS base, a diferencia de varias VMs que replican cada una un sistema operativo completo.

Docker se compone ,principalmente, de: el **Docker Engine** o motor de contenedores (que corre como servicio en el host y se encarga de construir y ejecutar contenedores), el formato de **imágenes Docker** (plantillas de solo-lectura con todo el software necesario para crear un contenedor) y el **Docker Hub** (registro central de imágenes). Para crear una imagen se escribe un archivo *Dockerfile* con instrucciones (base de sistema, copiar archivos de la aplicación, instalar dependencias, comando de inicio, etc.), y Docker genera la imagen que luego puede instanciarse en uno o varios **contenedores**. Cada contenedor es esencialmente un proceso aislado que ejecuta la aplicación especificada.

El uso de Docker en este proyecto permite **empaquetar el backend Node.js, la base de datos MongoDB, y otros servicios** en contenedores separados, garantizando que cada microservicio tenga su entorno de ejecución consistente con sus dependencias exactas. Por ejemplo, se creará una imagen Docker para el servicio Node (incluyendo el código del servidor y las librerías Node necesarias), otra imagen oficial de MongoDB para la base de datos, etc. Esto aporta varios beneficios:

- **Portabilidad**: los mismos contenedores se pueden ejecutar en cualquier máquina (local, servidor, o nube) con Docker, sea en un entorno CentOS (como el usado en desarrollo) u otra distro, sin problemas de compatibilidad.

- **Aislamiento:** cada servicio corre aislado, evitando que bibliotecas o configuraciones se interfieran entre sí (por ejemplo, puede haber dos servicios con diferentes versiones de Node.js sin conflicto).
- **Despliegue simplificado:** desplegar una nueva versión se reduce a construir una nueva imagen y reiniciar el contenedor con esa imagen, lo que facilita la entrega continua. Además, revertir cambios es tan fácil como volver a lanzar el contenedor con la imagen previa conocida.

Docker ha sido un habilitador clave de la adopción de microservicios, pues proporciona entornos **consistentes, portables y ligeros** para cada servicio, simplificando los flujos DevOps y reduciendo los tiempos de despliegue. En lugar de configurar manualmente servidores para cada componente, los contenedores aseguran que todos tengan la misma configuración prevista. En este proyecto, para la versión microservicios, todos los componentes (servicios Node, base de datos Mongo, etc.) se desplegarán como contenedores Docker, incluso ejecutándose en una máquina virtual CentOS como host. Vale resaltar que Docker es agnóstico al sistema operativo base siempre que soporte su Engine; en nuestro caso CentOS (Linux) provee un entorno robusto para ejecutar Docker Engine y manejar los contenedores con eficiencia.

Docker Compose (Orquestación de Contenedores)

Para gestionar múltiples contenedores Docker que conforman una aplicación, se emplea **Docker Compose**, una herramienta que facilita la definición y ejecución conjunta de un **conjunto de contenedores interconectados**. Compose utiliza un archivo de configuración (usualmente `docker-compose.yml`) donde se declaran los servicios que componen la aplicación, cada uno con su imagen o Dockerfile, puertos expuestos, volúmenes, variables de entorno, redes, etc. Con una sola orden (`docker-compose up`), Compose es capaz de construir (si es necesario) y lanzar todos los contenedores definidos, orquestándolos para que funcionen en conjunto.

En esencia, **Docker Compose** está diseñado para ejecutar varios contenedores como si fueran un único servicio coordinado. Esto resulta sumamente útil en

entornos de desarrollo y pruebas donde una aplicación completa puede componerse de, por ejemplo, un contenedor de base de datos, otro del backend, otro de una aplicación frontend, etc., que necesitan convivir. Compose permite evitar el arranque manual uno por uno de los contenedores y simplifica la configuración de las conexiones entre ellos. Los pasos básicos para usar Docker Compose en nuestro proyecto son:

1. **Definir Dockerfiles** para cada servicio personalizado de la aplicación (por ejemplo, un Dockerfile para construir la imagen del backend Node.js). Las imágenes oficiales (como MongoDB) se pueden usar directamente sin Dockerfile propio.
2. **Crear el archivo docker-compose.yml** listando los servicios. En nuestro caso incluirá, por ejemplo: un servicio backend que utiliza la imagen Node del servidor Express, un servicio db usando la imagen de MongoDB, quizás un servicio frontend para el servidor estático de React (si se separa), etc. En este archivo se configuran puertos (p. ej. mapear el puerto 80 del contenedor backend a uno del host), volúmenes (p. ej. persistencia de datos de MongoDB), y una red común para que los contenedores puedan comunicarse (Docker Compose crea una red virtual compartida para los servicios).
3. **Ejecutar Docker Compose** con el comando apropiado, el cual iniciará todos los contenedores definidos. Compose se encarga de respetar dependencias (por ejemplo, iniciar antes la base de datos si el backend depende de que esté disponible) y agrupa los logs de todos los servicios para facilidad de monitoreo.

Con esta herramienta, podemos levantar **todo el entorno de microservicios con un solo comando**, y asimismo bajarlo limpiamente. Además, Docker Compose permite escalar instancias de un servicio con facilidad (por ejemplo, levantar 2 contenedores del servicio backend para pruebas de carga) con un parámetro. Es importante notar que Docker Compose está orientado principalmente a entornos locales de desarrollo y staging; para orquestación en producción a gran escala, suelen emplearse herramientas más robustas como Kubernetes. No obstante, en

un entorno controlado de pruebas o prototipo como este, Compose resulta ideal por su **sencillez y velocidad de configuración**.

Por ejemplo, en este proyecto, un posible docker-compose.yml definiría los servicios:

- **backend**: construido a partir del Dockerfile local de la aplicación Node/Express, exponiendo el puerto 3000.
- **db**: usando la imagen oficial mongo:latest, montando un volumen para persistencia, exponiendo el puerto 27017 internamente.
- **jmeter** (opcionalmente para pruebas de rendimiento): podría definirse un contenedor con JMeter en caso de orquestar pruebas dentro de Docker.

De esta forma, con docker-compose up -d se iniciará la base de datos MongoDB y luego el backend, conectados en la misma red Docker (por lo que el backend podría acceder a Mongo mediante el hostname de servicio db). Esta consistencia en los nombres y redes evita la necesidad de configurar manualmente IPs o hosts en archivos de configuración. Compose resuelve la **conectividad interna** automáticamente.

Tabla 3. Comparativas máquinas virtuales contra contenedores

Característica	Máquinas Virtuales (VMs)	Contenedores
Aislamiento	Cada VM incluye un sistema operativo completo.	Comparten el kernel del host, solo aíslan librerías/dependencias.
Peso	Gigabytes (GB).	Megabytes (MB).
Tiempo de inicio	Minutos.	Segundos.
Eficiencia	Consumo elevado de recursos.	Uso eficiente de CPU y RAM.

Portabilidad	Limitada, depende del hipervisor.	Alta, se ejecutan en cualquier host con Docker.
---------------------	-----------------------------------	---

Comparación: Máquinas Virtuales vs Contenedores

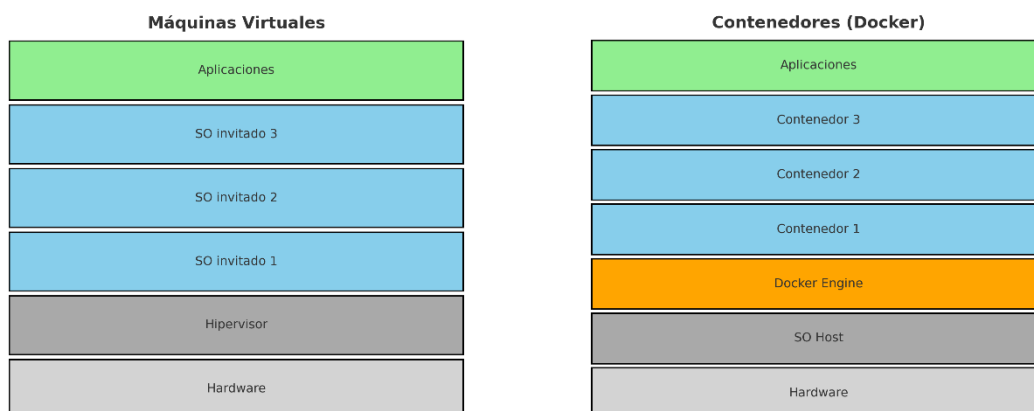


Ilustración 4. Diagrama comparación máquinas virtuales contra contenedores

2.5. Relación entre arquitecturas de software y rendimiento: Cómo influye la elección arquitectónica en el desempeño de las aplicaciones.

La **arquitectura de software** constituye uno de los factores más determinantes en el **rendimiento global de una aplicación empresarial**. La manera en que se estructuran los componentes, cómo interactúan entre sí y cómo son desplegados en la infraestructura condiciona la capacidad del sistema para responder eficientemente a las demandas de los usuarios. Elegir entre una **arquitectura monolítica** o una de **microservicios** no es simplemente una decisión técnica, sino estratégica, pues influye en aspectos tan críticos como la velocidad de respuesta, el consumo de recursos, la escalabilidad y la capacidad de recuperación ante fallos.

Tiempo de respuesta y latencia

En un sistema **monolítico**, todos los módulos residen en un mismo bloque de código y proceso. Esto significa que la comunicación entre ellos ocurre en **memoria**, lo cual es extremadamente rápido. Por ejemplo, una llamada desde el módulo de

“autenticación” hacia el módulo de “órdenes” se resuelve mediante funciones internas sin necesidad de atravesar la red, generando una latencia mínima. En contraste, en una arquitectura de **microservicios**, cada módulo funciona como un servicio independiente, expuesto generalmente a través de una API REST o de mecanismos de mensajería (como RabbitMQ o Kafka). Esto implica que una solicitud debe viajar a través de la red, procesarse en otro servicio y devolver la respuesta. Aunque este enfoque añade una **latencia adicional**, también permite desacoplar las funcionalidades y mejorar la resiliencia. Para mitigar el impacto en el tiempo de respuesta, las empresas suelen implementar técnicas como **cachés distribuidos**, **protocolos de comunicación ligeros** (ej. gRPC), balanceadores de carga y compresión de datos en tránsito.

Escalabilidad

La **escalabilidad** se refiere a la capacidad de un sistema para manejar un aumento en la carga de trabajo sin degradar su rendimiento.

En arquitecturas **monolíticas**, la escalabilidad suele ser **vertical**; es decir, se mejora el hardware donde corre la aplicación: más memoria RAM, CPUs más potentes, almacenamiento más rápido. Aunque esto puede funcionar hasta cierto punto, presenta limitaciones de costo y de capacidad física; además, escalar todo el sistema es ineficiente si solo un módulo específico requiere mayor capacidad. Por otro lado, las arquitecturas de **microservicios** permiten la **escalabilidad horizontal**, donde únicamente los servicios más demandados se replican en múltiples instancias. Por ejemplo, en un sistema de comercio electrónico, el servicio de “catálogo de productos” puede recibir miles de solicitudes por segundo durante un evento de ofertas masivas, mientras que el servicio de “gestión de perfiles” recibe menos tráfico. Con microservicios, es posible escalar únicamente el catálogo, optimizando los recursos y evitando costos innecesarios.

Consumo y optimización de recursos

El **consumo de recursos** (CPU, memoria, almacenamiento y ancho de banda) también se ve afectado por la arquitectura.

En un **monolito**, los recursos suelen aprovecharse de manera eficiente en entornos pequeños, ya que todo corre en un único proceso. Sin embargo, cuando el sistema crece, el monolito puede llegar a requerir grandes cantidades de memoria y CPU, incluso si solo un módulo es intensivo en procesamiento. Además, cada despliegue reinicia toda la aplicación, provocando picos de consumo y tiempos de inactividad. En **microservicios**, cada componente corre en un contenedor independiente. Esto permite asignar recursos de forma más granular: un servicio ligero (como el de autenticación) puede correr con poca memoria, mientras que un servicio de análisis de datos puede tener mayor capacidad. La desventaja es la **sobreutilización global de recursos**, ya que, ejecutar múltiples contenedores introduce overhead en memoria y CPU. No obstante, con técnicas de orquestación como Kubernetes o Docker Compose, es posible balancear la carga y optimizar la utilización de recursos a nivel de clúster.

Tolerancia a fallos y disponibilidad

La **tolerancia a fallos** es crucial en sistemas empresariales donde la caída de una aplicación puede representar pérdidas significativas.

En un **monolito**, un fallo en un componente crítico (por ejemplo, un error en el módulo de base de datos) puede comprometer toda la aplicación, dejando a los usuarios sin servicio hasta que se restaure el sistema completo. En **microservicios**, los fallos tienden a aislarse: si el servicio de “notificaciones por correo” se cae, los usuarios aún pueden seguir utilizando el resto de la plataforma, aunque sin recibir notificaciones. Esto proporciona una mayor **resiliencia**. Sin embargo, la tolerancia a fallos en microservicios depende de un **diseño robusto**, que incluya técnicas como *circuit breakers*, reintentos automáticos, balanceo de carga y monitoreo constante.

Mantenibilidad y evolución

Aunque este punto se relaciona más con el desarrollo, tiene un impacto directo en el rendimiento a largo plazo.

- En un **monolito**, mantener y evolucionar la aplicación es más difícil conforme crece. La deuda técnica y el acoplamiento pueden degradar el rendimiento, pues el sistema tarda más en procesar solicitudes y sufre con despliegues más lentos.
- En **microservicios**, la mantenibilidad mejora porque cada servicio puede evolucionar de manera independiente. Esto evita cuellos de botella en el rendimiento a largo plazo, aunque requiere invertir en infraestructura de observabilidad y en prácticas DevOps avanzadas.

Tabla 4. Comparativa: Rendimiento según arquitectura

Dimensión	Arquitectura monolítica	Arquitectura de microservicios
Tiempo de respuesta	Muy bajo (comunicación en memoria).	Mayor, por llamadas de red; mitigable con optimización.
Escalabilidad	Vertical, limitada y costosa.	Horizontal, granular, flexible y más económica.
Consumo de recursos	Eficiente en entornos pequeños, ineficiente a gran escala.	Overhead inicial mayor, pero recursos mejor distribuidos.
Tolerancia a fallos	Baja: un fallo afecta a todo el sistema.	Alta: fallos aislados por servicio, resiliencia mejorada.
Mantenibilidad	Difícil, aumenta la deuda técnica con el tiempo.	Alta, cada servicio evoluciona de forma independiente.

Capítulo III: Fundamentos Técnicos para la Implementación Arquitectónica

3.1. Principios de diseño en arquitecturas monolíticas y de microservicios: Comparación de patrones comunes de diseño y su impacto en el desarrollo, mantenimiento y escalabilidad de las aplicaciones.

En una arquitectura **monolítica**, todos los componentes de la aplicación (interfaz de usuario, lógica de negocio, acceso a datos, etc.) se construyen y despliegan como una sola unidad indivisible. Típicamente, sigue un **patrón en capas** tradicional: la interfaz de usuario invoca a la capa de lógica de negocio, que a su vez accede a la base de datos, todo ello dentro del mismo proceso o instancia de servidor. Esta estructura integrada facilita al inicio el desarrollo y el despliegue, ya que, existe *un único paquete de código* que concentra todas las funcionalidades y se implementa como una sola entidad ejecutable. Por ejemplo, una aplicación web monolítica puede consistir en un único proyecto que incluye módulos de pagos, carrito de compras e inventario, todos corriendo en un solo servidor (o instancia) y usando una única base de datos, como ilustra la figura. Esta simplicidad inicial se traduce en una menor **sobrecarga de planificación** al comienzo: es fácil empezar agregando módulos al mismo código base conforme se requieren nuevas funciones. Asimismo, los **entornos de desarrollo y pruebas** suelen ser más sencillos, pues basta con ejecutar y depurar una sola aplicación unificada, lo que permite seguir el flujo completo de datos dentro de un mismo entorno. En las primeras etapas de un proyecto, este enfoque monolítico puede reducir la complejidad cognitiva para los desarrolladores y acelerar la entrega inicial de funcionalidades.

No obstante, el **acoplamiento interno** inherente a los monolitos implica que sus componentes están fuertemente dependientes entre sí. La comunicación entre módulos ocurre mediante llamadas directas a funciones o acceso a objetos compartidos en memoria, lo cual, si bien es eficiente, **genera dependencias rígidas** dentro del sistema. Con el crecimiento del código base, esta falta de encapsulamiento puede conducir a lo que se conoce informalmente como un *“big ball of mud”*, dificultando aislar cambios sin afectar otras partes. Cada modificación pequeña en una parte del sistema puede tener efectos colaterales amplios, requiriendo volver a probar e incluso redespregar toda la aplicación completa ante cada cambio significativo. Esto vuelve el mantenimiento más **restrictivo y lento**, ya

que, las actualizaciones deben realizarse con extremo cuidado para no romper funcionalidades existentes. De hecho, en un monolito, **un simple cambio** puede obligar a recompilar y desplegar de nuevo el sistema entero, y las pruebas deben abarcar todo el conjunto para garantizar la integridad después de cada modificación. Esta situación impacta negativamente la **agilidad de desarrollo** a largo plazo. Empresas como Netflix o Amazon experimentaron estas limitaciones al escalar sus antiguos sistemas monolíticos: un incremento de carga en un módulo (por ejemplo, el carrito de compras) exigía escalar toda la aplicación completa, desaprovechando recursos, y la entrega continua de nuevas funciones se volvía más lenta por la necesidad de coordinar un código base gigantesco. En cuanto a **escalabilidad**, la arquitectura monolítica favorece inicialmente la **escalabilidad vertical** (mejorar el hardware de la única instancia), pero presenta dificultades para **escalar horizontalmente** (replicar la aplicación en múltiples instancias) sin introducir complejidades de sesión, consistencia y balanceo de carga. Si la demanda crece de forma desigual entre componentes, el monolito obliga a escalar todo el conjunto, generando un uso ineficiente de recursos al tener que asignar más capacidad de la necesaria a partes poco exigentes del sistema. En resumen, los monolitos son apropiados para aplicaciones más **pequeñas o de alcance limitado**, donde brindan simplicidad y alto rendimiento interno al no incurrir en sobrecarga de comunicación interprocesos. Pero en aplicaciones grandes y de rápido crecimiento presentan retos de mantenibilidad, despliegue y escalamiento, debido al fuerte acoplamiento de sus componentes y a la necesidad de escalar todo como un bloque único.

Por el contrario, la **arquitectura de microservicios** propone un diseño modular extremo: la aplicación se descompone en **múltiples servicios pequeños e independientes**, cada uno encargado de una *única función de negocio* y desplegado por separado. En vez de un solo código unificado, se tienen muchos **microcomponentes cooperantes**, típicamente alineados con **contextos de negocio bien delimitados**. Este enfoque sigue principios cercanos a la **alta cohesión y bajo acoplamiento**: cada microservicio encapsula completamente su lógica y datos, exponiendo solo interfaces bien definidas para interactuar con otros. En términos de patrones de diseño, los microservicios suelen apoyarse en

conceptos de la **arquitectura orientada al dominio (DDD)**, donde se identifican *contextos delimitados (bounded contexts)* que corresponden a servicios autónomos. Cada microservicio se convierte así en la unidad de despliegue y escala, responsable de **una sola responsabilidad de negocio** (por ejemplo, un servicio de *Pagos*, otro de *Carrito* y otro de *Inventario*, como se ilustra en la figura). Esta separación exige un mayor **esfuerzo de diseño inicial**, ya que, es necesario definir fronteras de servicio claras y contratos de comunicación entre ellos antes de empezar a codificar. Sin embargo, dicha planificación proporciona a largo plazo importantes beneficios en desarrollo y mantenimiento: al aislar funcionalidades, los equipos pueden trabajar **en paralelo** en servicios distintos sin estorbarse, agregar nuevas características sin tener que comprender o desplegar el sistema completo, y reutilizar servicios ya existentes en diferentes contextos. La **implementación continua** se vuelve más sencilla, pues cada microservicio puede actualizarse e implementarse independientemente, facilitando ciclos de despliegue frecuentes (incluso múltiples por día) sin interrumpir toda la aplicación. Esto eleva significativamente la agilidad para introducir mejoras y correcciones, mejorando el *time-to-market* de nuevas funcionalidades. Además, el **aislamiento de fallos** es mayor: si un microservicio específico falla o necesita reiniciarse, los demás pueden continuar operando, aumentando la **resiliencia global** del sistema. Este diseño naturalmente soporta la **escalabilidad horizontal**: cada servicio puede escalarse de forma independiente según la carga que experimente, en lugar de escalar todo el sistema monolítico. Por ejemplo, si el servicio de *Inventario* es el cuello de botella, se pueden levantar instancias adicionales únicamente de ese microservicio, sin tener que duplicar componentes de pagos o carrito que no lo necesitan. Esto conlleva un uso más eficiente de los recursos y una capacidad de adaptación granular a la demanda.

Pese a sus ventajas, las arquitecturas de microservicios **no eliminan la complejidad**, sino que la **redistribuyen**. En lugar de lidiar con un único gran sistema, ahora se gestionan muchos servicios pequeños, lo que introduce desafíos de coordinación, orquestación y consistencia que antes estaban resueltos implícitamente dentro del monolito. Se requiere una gestión rigurosa de la

configuración y versiones de cada servicio, así como mecanismos de descubrimiento, balanceo de carga y tolerancia a fallos en red. De hecho, el diseño con microservicios trae consigo una **curva de aprendizaje**: desarrollarlos suele demandar más tiempo y esfuerzo inicial, pues implica establecer infraestructura para su comunicación, despliegue independiente, monitoreo distribuido, etc., tareas innecesarias en un monolito simple. Esto significa que, para proyectos pequeños o equipos sin experiencia en sistemas distribuidos, un microservicio puede ser excesivo y generar **sobrecarga innecesaria**. En cambio, en escenarios de aplicaciones de gran escala, con módulos de negocio bien diferenciados y requerimientos cambiantes, los microservicios ofrecen **flexibilidad y escalabilidad superiores** a costa de afrontar esa complejidad añadida. En la práctica, muchas organizaciones adoptan una transición progresiva: comienzan con un monolito modular (*modular monolith*) durante el inicio del proyecto (por rapidez y sencillez) y, conforme el sistema crece, migran gradualmente componentes hacia microservicios independientes cuando la necesidad de escalar o desplegar selectivamente ciertas partes se vuelve crítica. Esta estrategia híbrida busca equilibrar la **simplicidad inicial** con la **adaptabilidad futura**, evitando incurrir prematuramente en la complejidad de los microservicios antes de que el tamaño del sistema lo justifique.

3.2. Estrategias de comunicación entre componentes: API REST, mensajería y eventos.

En las arquitecturas distribuidas basadas en microservicios, **la comunicación entre componentes** es un aspecto técnico crucial. A diferencia de un monolito, donde los diferentes módulos se invocan mediante llamadas directas en memoria (por ejemplo, invocando funciones o métodos dentro del mismo proceso), en un sistema de microservicios cada servicio es un proceso independiente, a menudo ejecutándose en máquinas o contenedores separados. Por tanto, *los microservicios deben interactuar a través de la red*, utilizando interfaces externas y protocolos de comunicación bien definidos. Existen principalmente dos enfoques complementarios para esta comunicación: **mecanismos síncronos** (basados en solicitud-respuesta, típicamente sobre HTTP) y **mecanismos asíncronos** (basados

en intercambio de mensajes o eventos, típicamente mediante colas o buses de mensajería). La elección de una estrategia u otra, o la combinación de ambas, tiene implicaciones en la latencia percibida, el acoplamiento entre servicios, la consistencia de los datos y la resiliencia del sistema.

Comunicación vía API REST (sincrónica): La forma más común y directa de interacción entre microservicios es a través de APIs RESTful utilizando HTTP como protocolo de transporte. En este modelo, un servicio actúa como **cliente** realizando una petición (por ejemplo, una solicitud HTTP GET o POST) a la API pública de otro servicio, el cual actúa como **servidor** y devuelve una respuesta inmediata. El servicio emisor bloquea su ejecución esperando la respuesta antes de continuar su flujo. Este estilo de comunicación se denomina *síncrono* porque implica una sincronización temporal: el llamador y el receptor deben estar activos al mismo tiempo y la respuesta debe recibirse para proceder. Las APIs REST (Representational State Transfer) se han popularizado por su simplicidad y estandarización: usan verbos HTTP (GET, POST, PUT, DELETE) y estructuras de datos como JSON o XML, siendo independientes del lenguaje y fáciles de consumir por cualquier plataforma. Por ejemplo, un microservicio de *Catálogo* podría exponer un endpoint REST `/productos/{id}` que otro microservicio de *Pedidos* invoque para obtener información del producto en tiempo real. Entre las **ventajas** de REST está su **sencillez y familiaridad** para los desarrolladores, un flujo claro de solicitud-respuesta que facilita el rastreo de transacciones, y la posibilidad de usar herramientas y servidores web convencionales. Sin embargo, la comunicación síncrona introduce también **desventajas**: acopla temporalmente a los servicios (uno depende de que el otro responda de inmediato), añade **latencia** por el intercambio a través de la red y hace al sistema más sensible a fallos o demoras si el servicio destino está caído o lento, la petición se bloquea y posiblemente falle. En sistemas con muchas dependencias, las llamadas REST encadenadas pueden derivar en altas latencias globales y propagar errores (el llamado efecto dominó), por lo que es necesario diseñar con cuidado para evitar *cascadas* de invocaciones entre decenas de microservicios. Aun con estos retos, REST es adecuado para operaciones que requieren **consulta inmediata** y consistencia sincrónica, como validaciones en

línea u obtención de datos puntuales bajo demanda. Es común complementarlo con patrones como **API Gateway**, que centraliza las llamadas de clientes externos en un único punto de entrada y distribuye las solicitudes a los microservicios internos correspondientes. Un API Gateway puede mejorar la eficiencia reduciendo viajes de ida y vuelta y manejando preocupaciones transversales (autenticación, *logging*, control de versiones de API), aunque agrega también una capa que puede incurrir en cierta latencia adicional.

Comunicación basada en mensajería y eventos (asíncrona): Para lograr un menor acoplamiento temporal y una mayor resiliencia, los microservicios suelen recurrir a **mecanismos asíncronos de comunicación**, donde un servicio emite un mensaje o evento y continúa con su procesamiento sin esperar una respuesta inmediata del receptor. En este modelo, los mensajes viajan típicamente a través de un **sistema de mensajería** (como un *message broker* o una cola de mensajes) que actúa como intermediario, almacenando y entregando los mensajes a los servicios destinatarios según corresponda. Hay dos sub-paradigmas principales aquí:

- **Mensajería con colas (point-to-point):** Un servicio envía un mensaje dirigido a uno específico (o a un grupo) mediante un canal de mensajes. Por ejemplo, un servicio de *Pedidos* podría colocar en una cola un mensaje **“Pedido Creado”** con los datos del nuevo pedido; el servicio de *Facturación* estaría suscrito a esa cola y la leería para procesar el cobro. Este esquema suele garantizar que cada mensaje es procesado por un solo receptor, típico en patrones de **cola de trabajo** o tareas asíncronas. Tecnologías como **AMQP** (p.ej., RabbitMQ) implementan este patrón. La ventaja es que el emisor no necesita saber nada del receptor (desacoplamiento), y el receptor puede procesar a su ritmo; si está saturado o momentáneamente caído, el mensaje queda pendiente en la cola y no se pierde. Esto mejora la **tolerancia a fallos**: la comunicación no requiere que ambos servicios coincidan en tiempo real, otorgando robustez frente a picos de carga o caídas temporales de algún componente. Además, habilita la **escalabilidad horizontal**, pues múltiples instancias de un mismo microservicio consumidor pueden leer de

la cola repartiendo la carga de procesamiento de mensajes concurrentes. La desventaja principal es la **complejidad adicional**: manejar colas implica asegurarse de la correcta entrega (muchos sistemas proveen entrega *al menos una vez*, lo que puede resultar en mensajes duplicados que el consumidor debe poder filtrar), gestionar la **ordenación** de mensajes cuando es relevante, y supervisar el broker como pieza crítica del sistema. También, la programación asíncrona puede ser más difícil de razonar y depurar, pues el flujo de eventos no sigue una secuencia lineal clara como en REST.

- **Arquitectura orientada a eventos (pub-sub)**: En este estilo, los microservicios se comunican difundiendo **eventos** en un modelo de publicación/suscripción. Un evento es una notificación de que “algo ocurrió” (por ejemplo, “*Pedido Realizado*”, “*Producto Agotado*”). Los servicios que están interesados en ese tipo de suceso se **suscriben** para recibirlos, pero el emisor no conoce ni dirige a destinatarios específicos. Esta arquitectura de **eventos desacoplados** permite una integración extremadamente flexible: múltiples servicios pueden reaccionar a un mismo evento de diferentes formas. Por ejemplo, cuando se registra el evento “*Pedido Realizado*”, el servicio de *Inventario* podría disminuir las existencias, el servicio de *Notificaciones* podría enviar un email de confirmación al cliente, y el servicio de *Analítica* podría registrar la venta, todo ello ocurriendo en paralelo y sin que el servicio de *Pedidos* tenga que invocar a cada uno directamente. Los eventos suelen distribuirse mediante un **bus de eventos** o una plataforma de *streaming* (como Apache Kafka, AWS Kinesis, etc.), que permiten publicar eventos en *topics* a los cuales otros servicios se suscriben. Una de las ventajas clave de este modelo es que logra un **acoplamiento muy bajo**: los emisores ignoran completamente quién procesa el evento, y los consumidores pueden unirse o dejar de escuchar sin afectar al emisor, fomentando así la evolución independiente de los servicios. Además, los eventos facilitan la implementación de **consistencia eventual** (se analiza más en la sección de gestión de datos): cada servicio mantiene su propio estado y lo ajusta al recibir eventos de cambios ocurridos en otros servicios,

en lugar de hacer bloqueos distribuidos. Esto incrementa la **resiliencia**: si un servicio está caído, los eventos pueden acumularse y se procesarán cuando esté de nuevo disponible, mejorando la robustez general. Cabe destacar que este patrón también habilita la **escalabilidad elástica**, ya que los consumidores pueden escalar en grupo para atender mayores flujos de eventos. Como contrapartida, la arquitectura orientada a eventos introduce retos como asegurar el **orden** de procesamiento (por ejemplo, procesar eventos antiguos antes que los nuevos para la misma entidad, lo cual sistemas como Kafka abordan mediante *particiones ordenadas*) y la dificultad de **trazabilidad**: reconstruir una secuencia completa de acciones de negocio puede requerir correlacionar eventos dispersos en lugar de seguir una llamada REST. También, la falta de respuesta inmediata complica ciertos casos en que se necesita confirmar sincrónicamente una acción (aunque esto puede resolverse combinando eventos con solicitudes puntuales o usando patrones de respuesta asíncrona).

En la práctica, los sistemas de microservicios maduros suelen emplear **una combinación** de estos enfoques, escogiendo la estrategia de comunicación según el caso de uso. Las llamadas **API REST síncronas** se utilizan típicamente para **consultas directas** u operaciones que requieren respuesta en tiempo real, por ejemplo, verificar el saldo de un usuario antes de autorizar un pago, donde es aceptable esperar unos milisegundos por la confirmación. Por otro lado, la **mensajería asíncrona** y los **eventos** se utilizan para **procesos desacoplados o de larga duración**, donde la prioridad es la robustez y la escalabilidad, por ejemplo, el procesamiento de una orden de compra tras checkout puede disparar múltiples pasos (facturación, reserva de envío, actualización de inventario) que se manejan asíncronamente, garantizando que cada servicio trabaje de forma independiente y se logre consistencia eventualmente. Esta combinación permite lograr un **equilibrio**: se mantienen tiempos de respuesta rápidos hacia el usuario final en las interacciones críticas, mientras que las tareas internas se distribuyen de forma no bloqueante, absorbiendo picos de carga y aislando fallos.

Es importante mencionar algunos **patrones de diseño** que complementan estas estrategias de comunicación para mejorar la arquitectura: por ejemplo, el patrón **Circuit Breaker** (cortacircuitos) es fundamental cuando se usan llamadas síncronas, pues detecta cuándo un servicio destino está fallando o lento y “abre el circuito” para evitar seguir esperando o enviando más solicitudes, lo que previene la saturación cascada del sistema. Otro patrón es la **Retransmisión con retrooff** (retry with backoff) para reintentar mensajes o peticiones fallidas de manera controlada. En el mundo de eventos, patrones como **Event Sourcing** (llevar un registro inmutable de eventos como fuente de la verdad) o **CQRS** (separar comandos de lectura/escritura) pueden emplearse para manejar la complejidad de datos distribuidos y consultas materializadas a partir de eventos. Sin entrar en detalles de implementación, estos patrones subrayan la necesidad de **diseñar cuidadosamente** la comunicación: los microservicios deben intercambiar información de manera confiable, consistente y segura. Esto incluye considerar aspectos de **seguridad** (cada API expuesta es un punto de entrada potencial, requiriendo autenticación/autorización robusta, cifrado de canales, etc.), **formato de datos** (establecer contratos claros, posiblemente usando esquemas versionados para JSON/XML, o tecnologías como gRPC con IDL Interfaz Definition Language para garantizar compatibilidad), y **observabilidad** (asegurar *logging* y *tracing* distribuido para poder seguir un flujo que atraviesa varios servicios). En resumen, **API REST, mensajería y eventos** conforman el repertorio básico de mecanismos de interacción en microservicios. Su uso apropiado, alineado con la naturaleza de cada operación de negocio, permite construir sistemas desacoplados y escalables, manteniendo la cohesión de cada servicio, pero logrando la coherencia del conjunto a través de contratos explícitos de comunicación.

3.3. Gestión de datos: Análisis de cómo se manejan las bases de datos en ambos enfoques.

Si bien la arquitectura de microservicios aporta numerosas ventajas de modularidad, escalabilidad y resiliencia, también introduce *dificultades técnicas específicas* que deben ser abordadas cuidadosamente en su implementación. Entre los desafíos más citados en la literatura y la práctica se encuentran los problemas de **latencia en la comunicación remota** y la **consistencia de datos en un entorno distribuido**, aunque no son los únicos. A continuación, analizamos estos dos aspectos clave, junto con otros retos técnicos inherentes a los microservicios.

Latencia y rendimiento distribuido: En un sistema monolítico, las interacciones entre componentes ocurren típicamente mediante llamadas directas en memoria, las cuales son extremadamente rápidas (orden de microsegundos o nanosegundos). Al pasar a microservicios, muchas de esas interacciones se transforman en **llamadas sobre la red** (peticiones HTTP, mensajes a un broker, etc.), las cuales incurren en una latencia significativamente mayor en el orden de milisegundos, dependiendo de la red y la carga. Esta diferencia puede parecer pequeña en términos absolutos, pero en un escenario donde para atender una solicitud de usuario un servicio deba a su vez consultar a varios otros servicios secuencialmente, la latencia total se acumula. Por ejemplo, si cada llamada interservicio toma 20 ms, y un flujo sincrónico involucra 5 servicios encadenados, solo en comunicación ya podrían irse ~100 ms adicionales, sin contar procesamiento interno. La **latencia adicional** introducida por la comunicación entre microservicios es por tanto un reto para considerar desde el diseño. No se limita solo al tiempo de transmisión de datos, sino también al *handshake* o negociación que cada protocolo requiere (establecimiento de conexiones TCP, serialización/deserialización de mensajes, etc.). Además, la variabilidad de la latencia (jitter) puede aumentar en redes saturadas o servicios bajo alta carga, provocando tiempos de respuesta menos predecibles que en un monolito. Esto puede impactar la experiencia del usuario final si no se mitiga adecuadamente. Para enfrentar este desafío, los arquitectos de microservicios aplican varias estrategias:

una es minimizar las dependencias síncronas en serie, favoreciendo diseños donde las llamadas puedan hacerse en paralelo o, mejor aún, evitar llamadas remotas en secciones críticas (por ejemplo, usando **caché local** o replicando ciertos datos para lectura). Otra es introducir patrones de **colapso de peticiones** (*request collapsing*) o agregación, donde un servicio intermedio (como un API Gateway) recibe múltiples pequeñas peticiones y las agrupa en una sola respuesta compuesta, reduciendo el número de viajes de red. Asimismo, se recurre a optimizaciones a nivel de comunicación, como utilizar protocolos más eficientes que HTTP/JSON cuando la latencia es crítica, por ejemplo, **gRPC** o Thrift, que usan llamadas RPC binarias sobre HTTP/2, reduciendo la sobrecarga en comparación con REST tradicional. También es esencial un buen diseño de **granularidad de servicios**: si los microservicios son demasiado finos (*nano-servicios*), una operación de negocio trivial podría implicar decenas de saltos de red entre servicios minúsculos, degradando mucho el rendimiento. Encontrar un balance en el tamaño de cada servicio ayuda a contener la latencia dentro de límites razonables. En sistemas de microservicios bien diseñados, se pueden lograr tiempos de respuesta competitivos con los monolíticos mediante estas técnicas y mediante infraestructura adecuada (por ejemplo, redes internas rápidas, despliegue localizado de servicios que suelen comunicarse entre sí, etc.). No obstante, el hecho fundamental permanece: **la latencia de red nunca es cero**, y asumir lo contrario es una de las falacias clásicas de la computación distribuida. Por eso, los microservicios deben concebirse asumiendo retardo en las comunicaciones y diseñando la experiencia de usuario y la arquitectura para tolerar cierta espera, o bien delegar tareas a procesos asíncronos donde la respuesta inmediata no sea imprescindible.

Además de la latencia, la comunicación distribuida aumenta la posibilidad de **fallos parciales**. En un monolito, o todo el sistema está activo o no lo está; en microservicios, es posible que algunos componentes funcionen mientras otros fallen. Desde la perspectiva de un servicio que hace llamadas síncronas, un fallo en la red o en el servicio objetivo se traduce en una espera prolongada o en un error. Esto puede degradar severamente el rendimiento percibido si no se mitiga. Por ello, **la tolerancia a fallos está estrechamente ligada a la latencia**: patrones como

timeouts (tiempos de espera máximos) y *circuit breakers* son esenciales. Un *timeout* evita que una llamada quede bloqueada indefinidamente por un servicio que no responde, liberando recursos y devolviendo control para tomar acciones alternativas. Un *circuit breaker* va un paso más allá: si detecta repetidos tiempos de espera o errores al llamar a cierto servicio, abre el circuito y previene nuevas llamadas por un tiempo, retornando errores rápidamente en lugar de seguir intentando la operación fallida. Esto no solo mejora la resiliencia sino también la latencia percibida por los componentes que hacen la llamada, ya que, evita esperas innecesarias. Adicionalmente, para reducir el impacto de la latencia se implementan técnicas de **caché** en distintos niveles: caché de respuestas HTTP (para que, si múltiples llamadas piden lo mismo en poco tiempo, solo un viaje por la red), caché de datos en servicios consumidores (guardando temporalmente datos obtenidos de otro servicio para reutilizarlos sin Re consultar cada vez), e incluso cachés distribuidas o *in-memory data grids* compartidas por varios servicios para lecturas de alta frecuencia. Todas estas medidas apuntan a **acercar los datos al lugar donde se necesitan**, mitigando el número y el costo de llamadas remotas.

Consistencia de datos y gestión de transacciones distribuidas: Como se discutió en la sección 3.3, la consistencia de datos es uno de los desafíos centrales al migrar de un modelo monolítico (transaccional) a un modelo distribuido por microservicios. En un monolito, la garantía ACID de la base de datos relacional evita una gran variedad de problemas de integridad: nunca se verá un pedido confirmado sin su registro de pago correspondiente, porque ambas operaciones ocurren bajo la misma transacción; nunca habrá un producto duplicado con el mismo ID, porque la base impone unicidad, etc. En microservicios, al no haber transacciones globales, surgen escenarios de **inconsistencia temporal** que deben manejarse cuidadosamente. Por ejemplo, es posible que un servicio responda antes de que otro relacionado haya completado su parte, exponiendo datos “viejos” por unos instantes. Un caso típico: un servicio de *Usuarios* actualiza la dirección de un cliente; si ese cliente hace inmediatamente una compra, el servicio de *Pedidos* podría, en ese preciso momento, tener aún la dirección antigua si no ha recibido el evento de actualización. Estos *glitches* de consistencia eventual son esperables y en la

mayoría de los casos tolerables (por ejemplo, probablemente unos segundos de diferencia en la dirección no causarán problema, eventualmente el pedido se puede reconciliar). Pero en otros casos más críticos, las inconsistencias pueden ser problemáticas: imaginemos un microservicio de *Inventario* que no descuenta stock porque falló el evento de pedido, permitiendo que otro cliente compre un producto que ya no estaba disponible. Para mitigar estos riesgos, los diseñadores implementan una serie de patrones:

- **Confirmaciones idempotentes y compensaciones:** Dado que en un sistema distribuido un paso puede fallar a la mitad, es importante que las operaciones sean *idempotentes* (repetirlas no cause efectos adversos). Así, si un servicio no recibe confirmación de otro, puede reintentar la operación sin temor a duplicar permanentes. Por ejemplo, mandar dos veces un evento “*reducir Inventario*” con el mismo ID de pedido debería producir el mismo resultado que enviarlo una vez el servicio de inventario debe ignorar el duplicado o reconocer que ya procesó ese pedido. Por otra parte, si un paso definitivamente falló después de que otros ya habían ocurrido, se aplican **transacciones compensatorias**: básicamente, emitir eventos u operaciones que deshacen los efectos de los pasos previos exitosos, restaurando cierto equilibrio. Este es el fundamento del **patrón Saga**, donde un flujo de negocio distribuido se modela como una secuencia de transacciones locales en distintos servicios, cada una con su posible acción de compensación en caso de abortar el flujo. Por ejemplo, si en un proceso de reserva de viaje un microservicio confirma un vuelo, pero luego falla la confirmación del hotel, la saga desencadena una compensación que cancela el vuelo reservado para no dejar una inconsistencia.
- **Consistencia eventual con eventos de confirmación:** En lugar de bloquear un proceso hasta que todos los servicios confirmen (lo cual reintroduciría acoplamiento fuerte y latencia elevada), se suele optar por notificar asíncronamente la conclusión exitosa de flujos. Así, un servicio no asume que la operación global se completó, hasta que reciba *eventos* de confirmación. Por ejemplo, el servicio de *Pedidos* podría marcar un pedido

como “Pendiente” tras su creación, y solo pasar a “Completado” cuando reciba eventos indicando que *Pago aceptado e Inventario descontado*. Esto mantiene cada microservicio trabajando de manera autónoma y comunicándose por eventos para lograr una **coordinación eventual**.

- **Gestión de duplicidades y consenso débil:** En sistemas altamente distribuidos, puede ocurrir que dos servicios procesen eventos relacionados y generen resultados duplicados o en orden distinto. Un reto es garantizar que, a pesar de la asincronía, se respeten ciertas invariantes: por ejemplo, no crear dos cuentas de usuario con el mismo email en dos servicios distintos. Sin una base central que haga cumplir la unicidad, hay que diseñar protocolos de consenso o delegar la responsabilidad de cierta *verdad única* a un servicio maestro. Algunos sistemas usan algoritmos de consenso (como Raft, Paxos) entre microservicios cuando se requiere acuerdo común, aunque suelen aplicarse más a nivel de bases de datos distribuidas que a la lógica de microservicios en sí. Otra técnica es **centralizar la secuencia** de ciertos eventos críticos: por ejemplo, usar un *message broker* que garantice orden para todos los mensajes con un mismo clave (Kafka permite ordenar por clave de partición). Esto ayuda a procesar en secuencia lógica, digamos, todas las actualizaciones de un mismo objeto en el orden correcto.

En síntesis, lograr **consistencia de datos en microservicios** requiere cambiar la mentalidad: de confiar en la sincronización implícita de una base de datos, a explícitamente orquestar la sincronización vía comunicaciones. La *consistencia inmediata* se reemplaza por *consistencia eventual* como objetivo, y ello obliga a identificar qué invariantes críticas deben eventualmente cumplirse y cómo verificar o reconciliar datos. Por ejemplo, podría implementarse un proceso periódico de *reconciliación* que revise que los totales de ventas de un servicio coincidan con los registros contables de otro, corrigiendo discrepancias si las hubiera, para añadir una red de seguridad adicional en pro de la consistencia. En escenarios donde la consistencia fuerte es absolutamente necesaria (p.ej., contabilidad financiera), a veces la solución es no particionar esa área y mantenerla dentro de un único servicio

transaccional; pero en muchos casos, los microservicios optan por consistencia eventual porque priorizan disponibilidad y rendimiento.

Otros desafíos técnicos adicionales: Además de latencia y consistencia, los microservicios presentan otras dificultades técnicas notables:

- **Monitoreo y depuración distribuidos:** En un monolito, basta con revisar los logs de una única aplicación o usar un depurador adjunto al proceso para seguir el flujo completo de una petición. En microservicios, una sola acción de negocio puede tocar múltiples servicios, cada uno generando sus propios logs en distintos lugares. Seguir el rastro de un evento a través de todo el sistema requiere implementar **trazabilidad distribuida** (Distributed Tracing) normalmente incorporando un identificador de correlación único en las peticiones que todos los servicios propaguen y registren. Herramientas como Zipkin, Jaeger o OpenTelemetry permiten reconstruir el camino de una transacción distribuida, pero su adopción y correcta configuración es un esfuerzo adicional que antes no era necesario. Asimismo, la agregación de logs de múltiples microservicios (mediante sistemas centralizados tipo ELK, Graylog, etc.) se vuelve crucial para diagnosticar problemas globales. La **depuración** paso a paso también se complica, pues no es trivial ejecutar todo el sistema en un solo debugger; a menudo se recurre a pruebas integrales y simulaciones para aislar fallas. Por tanto, **la observabilidad** (que engloba logging, tracing y métricas) es un pilar técnico fundamental en microservicios, sin el cual es muy difícil mantener la salud del sistema y encontrar cuellos de botella o errores sutiles.
- **Gestión de configuración y despliegue:** Pasar de desplegar una única aplicación a desplegar decenas de servicios conlleva retos en la consistencia de configuraciones (cada servicio tiene sus parámetros, credenciales, *endpoints* de otros servicios, etc.), en el versionado (asegurar que los cambios en un servicio no rompan a otros clientes que quizás no se han actualizado aún) y en la automatización de despliegues. Tecnologías de contenedores y *orquestradores* (Docker, Kubernetes, etc.) se volvieron casi

necesarias para manejar la complejidad de tener muchos microservicios corriendo, permitiendo programar escalados automáticos, actualizaciones graduales (blue-green deployments, canary releases) y manejo de fallos. Pero esto implica una **curva de aprendizaje técnica**: los equipos deben dominar herramientas de DevOps, lo que puede ser un obstáculo si antes solo gestionaban un servidor monolítico tradicional. La infraestructura como código, los pipelines CI/CD, y la gestión de secretos distribuida (ej. vaults centralizados) pasan a primer plano para poder desplegar y configurar consistentemente docenas de componentes. La ausencia de estos buenos *prácticas* puede traducirse en errores de configuración difíciles de detectar o entornos inconsistentes entre servicios.

- **Seguridad y control de acceso:** En un monolito, típicamente la seguridad (autenticación/autorización) se implementa de manera unitaria en la capa de entrada y luego el resto de la aplicación confía en esa identidad ya validada. En microservicios, cada servicio expone interfaces potencialmente accesibles, por lo que se suele necesitar un enfoque de **cero confianzas** entre ellos: autenticación de cada comunicación (tokens JWT o similares), encriptación de tráfico interno (TLS incluso en llamadas inter-servicio), y autorización granular de qué servicios pueden invocar a cuáles (por ejemplo, mediante mallas de servicio / *service mesh* que definan políticas). Esto agrega sobrecarga en cada llamada y complejidad en la gestión de credenciales y certificados. El área de **seguridad distribuida** es pues otro reto: evitar que la ampliada *superficie de ataque* (muchos endpoints, muchos puntos de fallo) exponga vulnerabilidades, y al mismo tiempo, administrar eficientemente identidades de servicio y de usuario a través de todo el sistema.

3.5. Aplicación práctica en el prototipo: Síntesis de los aspectos técnicos que guiarán el diseño del prototipo.

La aplicación práctica de los fundamentos arquitectónicos expuestos en las secciones anteriores se materializa en el diseño del prototipo de un sistema de comercio electrónico desarrollado en dos versiones paralelas: una basada en una arquitectura monolítica tradicional y otra estructurada bajo el paradigma de microservicios. Esta sección no pretende detallar la implementación técnica completa, sino establecer los criterios y principios que orientaron dicha construcción, sirviendo como puente entre el marco conceptual y las decisiones de diseño presentadas en los siguientes capítulos.

En la versión monolítica, se opta por una aplicación unificada en la que todos los componentes del sistema residen en un mismo código base y comparten una sola instancia de base de datos. Esta aproximación responde a un enfoque tradicional centrado en la simplicidad de despliegue, la eficiencia en la comunicación interna y la facilidad de desarrollo inicial. La organización modular interna permite distinguir lógicas por dominio (usuarios, productos, pedidos, etc.), pero no impone barreras estrictas de separación en tiempo de ejecución ni de acceso a datos, lo que limita la escalabilidad y dificulta la mantenibilidad a largo plazo.

En contraste, la versión basada en microservicios adopta una división por capacidades de negocio, implementando un servicio independiente para cada subdominio funcional. Cada microservicio es desarrollado y desplegado de forma autónoma, siguiendo los principios de alta cohesión y bajo acoplamiento. En términos de persistencia, si bien el ideal arquitectónico dicta que cada servicio debe tener su propia base de datos, en esta implementación se utiliza una única instancia de base de datos MongoDB Atlas compartida por todos los servicios, donde la separación se mantiene a nivel de colecciones lógicamente aisladas. Es decir, cada microservicio accede únicamente a su conjunto de colecciones propias (por ejemplo, el servicio de usuarios opera sobre la colección de usuarios, el servicio de pedidos sobre la de órdenes, etc.), sin invadir el ámbito de persistencia de los demás servicios. Esta estrategia permite preservar la autonomía de datos en un entorno

compartido, facilitando el despliegue y reduciendo la complejidad operativa, sin comprometer los principios de diseño distribuido.

La comunicación entre microservicios se fundamenta en interfaces REST bien definidas, operando bajo HTTP y utilizando formatos de intercambio como JSON. En escenarios donde la sincronización fuerte no es requerida, se contempla el uso de patrones asíncronos basados en eventos y colas de mensajes, permitiendo desacoplar servicios y tolerar fallos transitorios. Asimismo, se reconoce la importancia de incluir un API Gateway como punto de entrada unificado, que dirija las solicitudes del cliente hacia los microservicios correspondientes, gestionando aspectos como enrutamiento, autenticación y posible balanceo de carga.

Capítulo IV: Diseño y Desarrollo del Prototipo

4.1. Objetivos del prototipo: Comparar rendimiento y escalabilidad entre arquitecturas monolítica y microservicios.

El prototipo desarrollado tiene como objetivo **comparar el rendimiento y la escalabilidad** entre dos enfoques arquitectónicos: la arquitectura **monolítica** y la arquitectura de **microservicios desplegados con contenedores Docker**. Para tal fin, se construyó inicialmente una versión monolítica de la aplicación que integra todos los módulos en un solo backend. Posteriormente, esta misma aplicación será adaptada y desplegada bajo un esquema de microservicios para evaluar las diferencias de comportamiento y eficiencia entre ambas alternativas.

4.2. Arquitectura del sistema: Implementación de módulos: login, gestión de productos y ventas.

La aplicación fue diseñada con tres módulos principales que representan un flujo funcional completo dentro del sistema:

Módulo de login y registro de usuarios: encargado de la autenticación, validación de credenciales y gestión de cuentas.

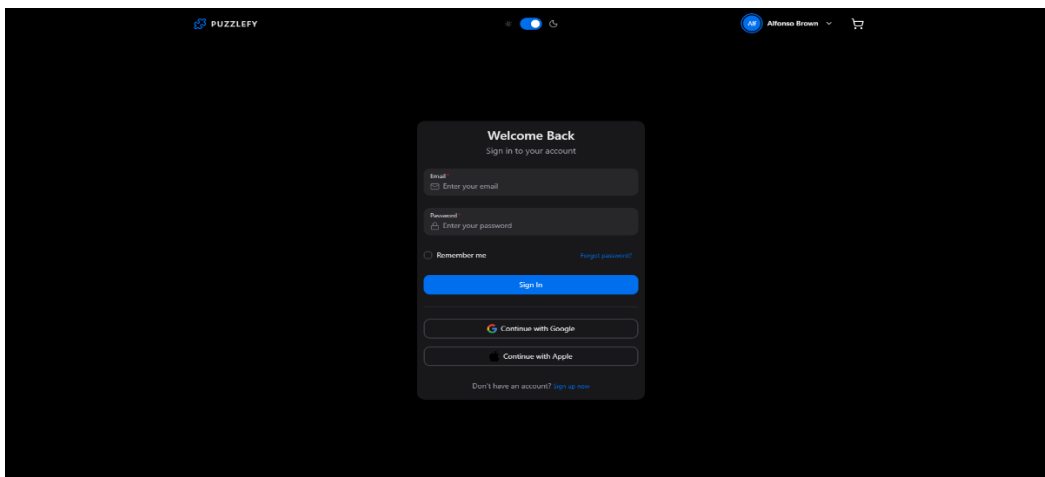


Ilustración 5. Interfaz de Login

La interfaz de login permite que los usuarios existentes accedan al sistema mediante su correo electrónico y contraseña. El formulario valida los campos ingresados y envía las credenciales al backend para ser verificadas. Su diseño está enfocado en

ofrecer un acceso rápido y directo a las funcionalidades principales de la aplicación.

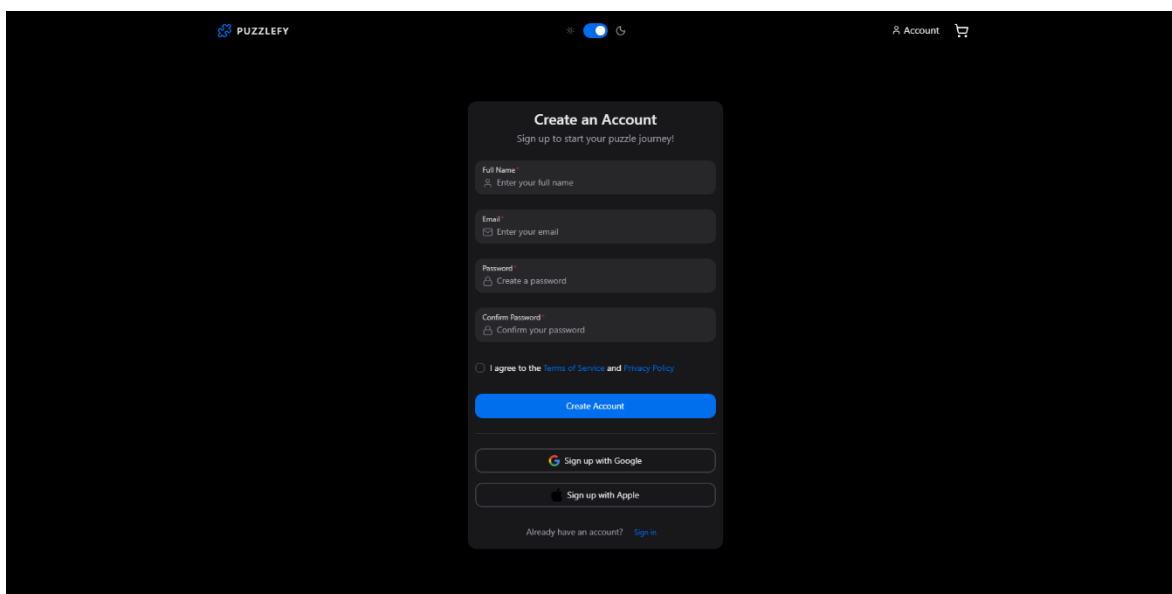
The image shows a mobile app interface for 'PUZZLEFY'. At the top, there's a navigation bar with the app logo, a toggle switch, and links for 'Account' and a shopping cart. The main content is a 'Create an Account' modal. It has a title 'Create an Account' and a subtitle 'Sign up to start your puzzle journey!'. Below this are four input fields: 'Full Name' (placeholder: 'Enter your full name'), 'Email' (placeholder: 'Enter your email'), 'Password' (placeholder: 'Create a password'), and 'Confirm Password' (placeholder: 'Confirm your password'). There's a checkbox for 'I agree to the Terms of Service and Privacy Policy'. A blue 'Create Account' button is below the checkbox. At the bottom of the modal are two buttons: 'Sign up with Google' and 'Sign up with Apple'. A link 'Already have an account? Sign in' is at the very bottom.

Ilustración 6. Interfaz de Registro

La interfaz de registro permite crear nuevas cuentas dentro del sistema. Incluye un formulario donde el usuario proporciona datos básicos como correo electrónico y contraseña. Los datos ingresados son validados y enviados al backend para generar un nuevo perfil de usuario.

Módulo de reseñas (reviews): permite que los usuarios registren, visualicen y gestionen reseñas sobre los productos o servicios disponibles, almacenando la información en la base de datos.

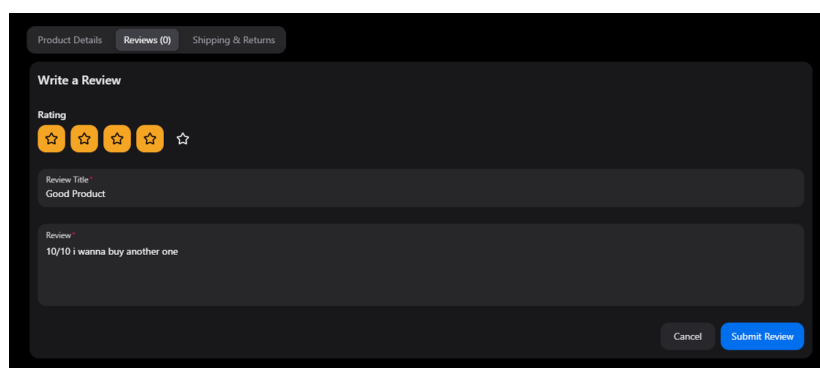
The image shows a mobile app interface for 'PUZZLEFY' with a 'Reviews' tab selected. The main content is a 'Write a Review' form. It has a 'Rating' section with five stars, where the first four are filled and the fifth is empty. Below this is a 'Review Title' field with the placeholder text 'Good Product'. Below that is a 'Review' text area with the placeholder text '10/10 I wanna buy another one'. At the bottom right of the form are two buttons: 'Cancel' and 'Submit Review'.

Ilustración 7. Interfaz de Creación de Reseñas

Módulo de ventas o carrito de compras: simula el proceso de compra, añadiendo productos al carrito y registrando órdenes en la base de datos.

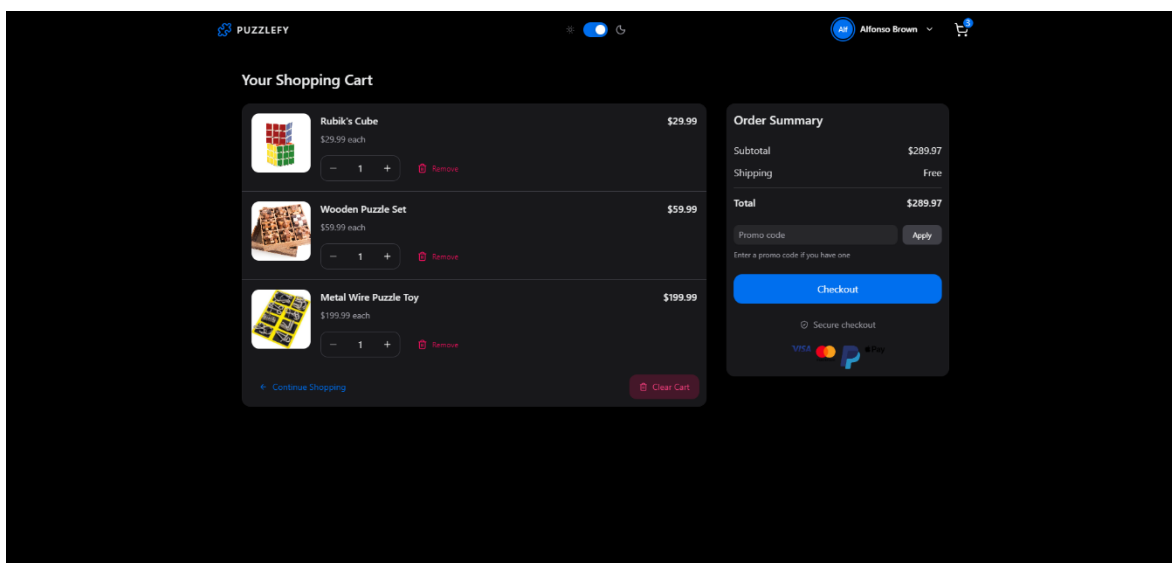


Ilustración 8. Interfaz de Carrito de Compras

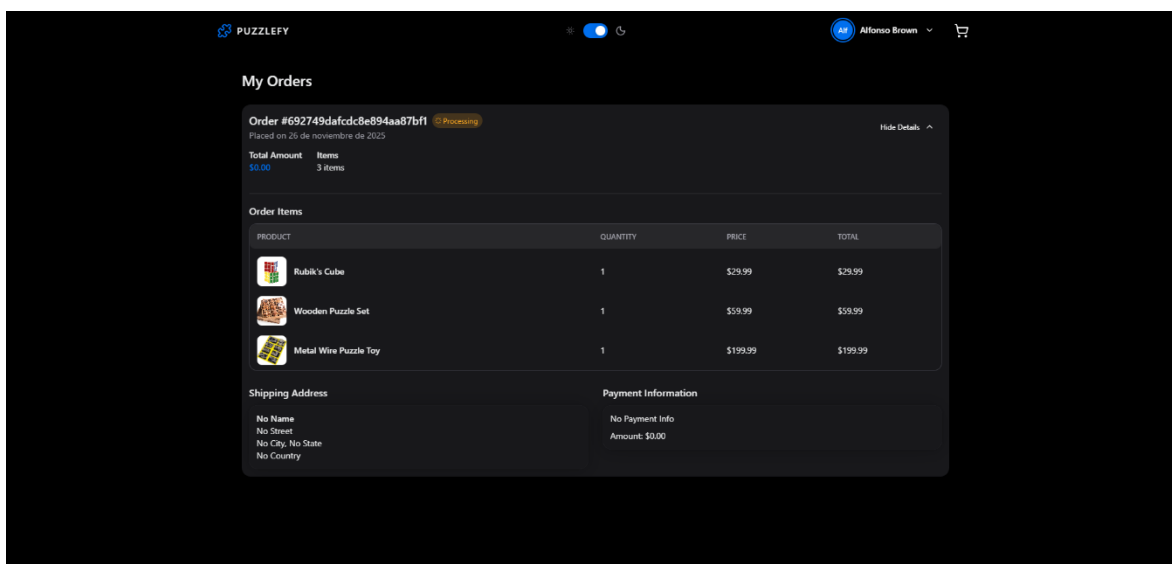


Ilustración 9. Interfaz de Órdenes / Historial de Compras

Flujo general de navegación: Este flujo representa el recorrido típico del usuario y sirve como soporte visual para explicar la lógica de negocio.

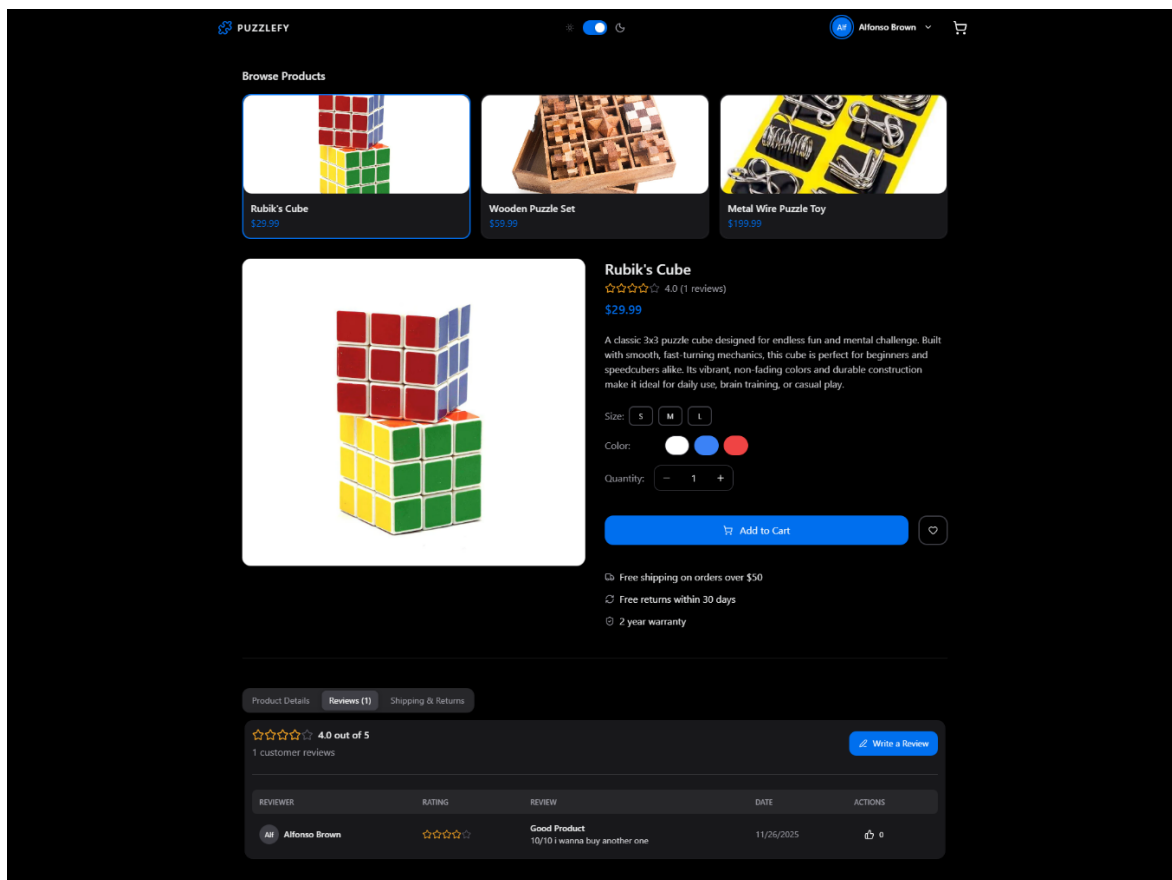


Ilustración 10. Interfaz del Flujo de Navegación

En la **versión monolítica**, estos módulos forman parte de un único proyecto backend desarrollado en **Node.js con Express**, conectado a una base de datos centralizada en **MongoDB Atlas**. El **frontend**, construido en **React.js con Vite**, consume los endpoints REST del backend para ejecutar todas las operaciones del sistema.

En la **versión dockerizada**, esta arquitectura se adapta para que cada componente frontend y backend se ejecute dentro de **contenedores independientes** administrados mediante **Docker Compose**, los cuales se comunican a través de una **red interna** definida por Docker.

Esto permite una mayor independencia entre los servicios, facilita la replicación del entorno en diferentes máquinas y mejora la portabilidad del sistema.

4.3. Selección de tecnologías: Uso de Node.js, MongoDB, Docker y React.js.

Para el desarrollo del prototipo se eligieron tecnologías modernas y ampliamente utilizadas en la industria:

- **Node.js y Express:** facilitan la construcción de APIs REST rápidas y escalables, con un modelo asíncrono no bloqueante.
- **MongoDB Atlas:** ofrece una base de datos NoSQL en la nube, flexible y con alta disponibilidad, eliminando la necesidad de configuración local.
- **React.js con Vite:** permite construir una interfaz de usuario modular y eficiente, con tiempos de compilación reducidos y una experiencia de desarrollo ágil.
- **Docker (para la segunda versión):** se utilizará en la fase de microservicios para contenerizar los componentes y orquestarlos mediante Docker Compose.

Estas herramientas garantizan rapidez en el desarrollo, facilidad de integración y escalabilidad futura del prototipo.

4.4. Lógica de negocio y comunicación: Desarrollo de funcionalidades y su integración.

En la arquitectura monolítica, la **lógica de negocio** se concentra en el backend, que centraliza la gestión de usuarios, productos y ventas dentro de un mismo proyecto. El flujo de comunicación se desarrolla de la siguiente manera:

1. El **frontend** en React envía solicitudes HTTP hacia el **backend**.
2. El **backend** en Node.js/Express procesa las solicitudes, ejecuta la lógica correspondiente y accede a la base de datos.

3. La **base de datos MongoDB Atlas** responde con la información solicitada o almacena los datos procesados (usuarios, productos u órdenes).

Este enfoque proporciona **simplicidad y rapidez** en las fases iniciales del desarrollo, ya que todos los componentes comparten un mismo entorno y se comunican de forma directa. No obstante, presenta limitaciones en **escalabilidad, resiliencia y facilidad de mantenimiento**, aspectos que serán evaluados al compararse con la versión implementada bajo una arquitectura de microservicios.

En la **versión dockerizada** del prototipo, esta misma lógica de negocio se mantiene, pero los componentes se ejecutan en **contenedores independientes** que se comunican a través de una red interna definida por Docker. El backend, el frontend y la base de datos se ejecutan en entornos separados, definidos mediante **Dockerfiles** y gestionados colectivamente por **Docker Compose**, lo que asegura una mayor consistencia y portabilidad del entorno de desarrollo y despliegue.

4.5. Microservicios y Docker: Despliegue de microservicios en contenedores Docker.

Proceso de despliegue de microservicios con Docker

El proceso de despliegue inicia con la contenedorización de cada módulo del sistema. En el caso del presente prototipo, los componentes del backend y del frontend se definieron mediante archivos *Dockerfile*, que especifican las dependencias, el entorno de ejecución y los comandos necesarios para construir la imagen del servicio.

Cada microservicio se convierte en una imagen Docker que puede ejecutarse como un contenedor independiente. Este enfoque permite realizar pruebas individuales de rendimiento y asegurar que las dependencias de cada módulo se mantengan aisladas.

Posteriormente, se utiliza **Docker Compose** para definir y gestionar múltiples contenedores de manera coordinada. Mediante un archivo **docker-compose.yml**,

se establece la configuración de red, los puertos expuestos, las variables de entorno (como las credenciales de conexión a la base de datos MongoDB) y las dependencias entre servicios.

De esta forma, el sistema puede desplegarse completamente con un solo comando (`docker compose up`), levantando de manera automática el entorno del backend y el frontend, junto con las conexiones necesarias para su funcionamiento.

```
root@vml2827764:~/react-puzzlefy# docker compose up -d
[+] Running 5/5
  Network react-puzzlefy/default      Created           0.7s
  Container react-puzzlefy/review-service-1 Started          1.0s
  Container react-puzzlefy/user-service-1 Started          1.0s
  Container react-puzzlefy/order-service-1 Started          1.0s
  Container react-puzzlefy/api-gateway-1 Started          1.2s
root@vml2827764:~/react-puzzlefy# docker ps
CONTAINER ID   IMAGE                                COMMAND          CREATED        STATUS        PORTS                               NAMES
dd87f0668298   react-puzzlefy-api-gateway          "docker-entrypoint.s..." 7 seconds ago   Up 6 seconds   0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp   react-puzzlefy-api-gateway-1
6ca3a18f65fb   react-puzzlefy-review-service       "docker-entrypoint.s..." 8 seconds ago   Up 7 seconds   4003/tcp                                react-puzzlefy-review-service-1
2c157f3ed6ea   react-puzzlefy-order-service        "docker-entrypoint.s..." 8 seconds ago   Up 7 seconds   4002/tcp                                react-puzzlefy-order-service-1
a4fa939b035b   react-puzzlefy-user-service         "docker-entrypoint.s..." 8 seconds ago   Up 7 seconds   4001/tcp                                react-puzzlefy-user-service-1
root@vml2827764:~/react-puzzlefy#
```

Ilustración 11. Contenedores del prototipo desplegados con Docker Compose.

Consideraciones de red y comunicación entre microservicios

En una arquitectura basada en Docker, los microservicios pueden comunicarse a través de una red interna definida por *Docker Compose*. Esta red permite que los contenedores interactúen usando nombres lógicos (por ejemplo, *backend* o *frontend*), en lugar de depender de direcciones IP estáticas. Este mecanismo simplifica la configuración y evita problemas de conectividad entre servicios que se reinician o actualizan.

Además, Docker facilita la exposición de puertos hacia el exterior del contenedor, lo que permite que el sistema pueda ser accedido desde clientes externos, como navegadores o herramientas de prueba como Apache JMeter.

Gestión de entornos y variables de configuración

El uso de archivos (`env`) dentro del entorno Docker permite administrar de forma centralizada las variables sensibles y de configuración, tales como las credenciales de conexión a la base de datos, los puertos de ejecución o las direcciones de API. Esto incrementa la seguridad y facilita la replicación del entorno de despliegue sin necesidad de modificar directamente el código fuente.

Capítulo V: Pruebas y Validación

5.1. Definición de criterios de evaluación del prototipo: Evaluación de rendimiento, escalabilidad y consumo de recursos.

Para determinar el comportamiento y desempeño de los prototipos desarrollados tanto en arquitectura monolítica como en microservicios dockerizados se establecieron criterios de evaluación basados en tres dimensiones principales: **rendimiento, escalabilidad y consumo de recursos**. El objetivo es analizar cómo la estructura arquitectónica influye en la capacidad de respuesta, estabilidad y eficiencia del sistema bajo diferentes niveles de carga concurrente.

1. Rendimiento

El rendimiento se medirá mediante pruebas de carga realizadas con **Apache JMeter**, utilizando escenarios configurados para **250, 500 y 1000 usuarios simultáneos**.

Estas pruebas permiten cuantificar la capacidad del sistema para procesar múltiples solicitudes concurrentes, midiendo los siguientes indicadores clave:

Tabla 5. Métricas de rendimiento utilizadas en las pruebas

Métrica	Descripción	Unidad	Fuente / Herramienta
Tiempo promedio de respuesta (Average Response Time)	Tiempo medio que tarda el sistema en responder a una petición.	Milisegundos (ms)	Apache JMeter
Latencia (Latency)	Tiempo transcurrido entre el envío de la solicitud y la recepción del primer byte de respuesta.	ms	Apache JMeter

Percentiles (p50, p90, p95, p99)	Distribución de tiempos de respuesta que permite identificar la experiencia de la mayoría de los usuarios y los casos extremos.	ms	Apache JMeter
Throughput	Número de solicitudes procesadas por segundo, reflejando la capacidad de procesamiento del sistema.	req/s	Apache JMeter
Tasa de errores (Error Rate)	Porcentaje de solicitudes que resultan en errores de conexión o respuestas inválidas.	%	Apache JMeter

El análisis de estas métricas permitirá evaluar la **eficiencia del backend**, la **capacidad de respuesta del frontend** y la **estabilidad general** del sistema durante la ejecución de cargas crecientes.

2. Escalabilidad

La escalabilidad se evaluará analizando cómo varía el rendimiento del sistema a medida que aumenta la cantidad de usuarios concurrentes (100 → 250 → 500). El comportamiento esperado es que un sistema bien escalado mantenga un **tiempo de respuesta y throughput estables**, aun cuando la carga crezca significativamente.

Tabla 6. Criterios de evaluación del rendimiento del sistema

Criterio	Descripción	Indicador de comparación
Degradación controlada	Evaluar si el tiempo de respuesta se incrementa de forma proporcional al aumento de usuarios.	Incremento ≤ 30 % entre escenarios.
Mantenimiento de throughput	Comprobar si la tasa de procesamiento de peticiones se mantiene estable o mejora.	Throughput decrece ≤ 20 %.
Estabilidad funcional	El sistema debe mantener la integridad de las operaciones (login, compras, reviews) sin errores críticos.	Error Rate ≤ 5 %.

El análisis de escalabilidad permitirá comparar la **resiliencia de la arquitectura monolítica** frente a la **modularidad y aislamiento de los microservicios en Docker**, destacando cómo cada enfoque responde al crecimiento de la demanda.

3. Consumo de recursos

El consumo de recursos será monitoreado paralelamente durante las pruebas de carga, tanto en entorno local (monolítico) como en entorno dockerizado. El objetivo es determinar la eficiencia en el uso de CPU, memoria y red, así como detectar posibles cuellos de botella de infraestructura.

Tabla 7. Parámetros de Monitoreo de Recursos del Sistema

Recurso	Métrica	Descripción	Herramienta
CPU	Porcentaje de uso promedio y pico	Indica la carga de procesamiento que soporta cada contenedor o proceso.	docker stats, htop, Prometheus
Memoria RAM	Uso promedio y máximo.	Evalúa la eficiencia del consumo de memoria por instancia.	docker stats, free -m
Ancho de banda de red	Bytes enviados y recibidos por segundo.	Determina el tráfico de red generado por las solicitudes simultáneas.	docker stats, iftop
Tiempo de despliegue	Segundos hasta disponibilidad total del sistema.	Mide la rapidez de arranque en entornos Docker.	Registro manual / logs

Estas métricas permiten identificar si la contenedorización introduce sobrecarga adicional o mejora la distribución de recursos al separar los servicios en instancias independientes.

4. Configuración de las pruebas

Cada escenario se ejecutará con la siguiente configuración general:

Tabla 8. Configuración de los Escenarios de Prueba

Escenario	Usuarios simultáneos	Ramp-up (s)	Duración (min)	Repeticiones
1	250	30	10	3
2	500	60	10	3
3	1000	120	10	3

- Cada prueba incluirá operaciones de login, consulta de productos, envío de reviews y finalización de venta.
- Entre cada ejecución se dejará un margen de **cinco minutos** para liberar recursos.
- Los resultados se exportarán en formato.csv y se procesarán mediante los reportes HTML de Apache JMeter para análisis estadístico.

5.2. Métodos de prueba con usuarios: Pruebas de carga utilizando Apache JMeter.

Este apartado presenta **resultados preliminares** de las pruebas de carga ejecutadas con **Apache JMeter** sobre el prototipo en sus dos modalidades: **arquitectura monolítica y arquitectura dockerizada**. Los escenarios, métricas y configuración se corresponden con lo definido en el punto **5.1**. Aquí se **describen** los hallazgos cuantitativos principales.

5.2.1. Alcance y escenarios ejecutados

Se ejecutaron tres escenarios de carga que replican flujos de usuario: **signup** → **login** → **catálogo** → **añadir al carrito** → **checkout** → **ver órdenes** → **listar reseñas**.

La configuración por escenario fue:

- **Escenario 1:** 250 usuarios concurrentes, ramp-up 30 s, duración 10 min o 3 repeticiones lo que ocurra primero.
- **Escenario 2:** 500 usuarios concurrentes, ramp-up 60 s, duración 10 min o 3 repeticiones lo que ocurra primero.
- **Escenario 3:** 1000 usuarios concurrentes, ramp-up 120 s, duración 10 min o 3 repeticiones lo que ocurra primero.
- Los resultados se guardaron en **.csv** y se generó el **reporte HTML** de JMeter para percentiles, throughput y errores.

5.2.2. Métricas registradas

Para cada escenario y arquitectura se recopilaron los siguientes indicadores:

- **Average Response Time (ms):** tiempo promedio de respuesta por solicitud.
- **Latency (ms):** tiempo entre el envío y la recepción del primer byte.
- **Percentiles (p50, p90, p95, p99):** distribución de tiempos que refleja la experiencia del usuario típico y los casos extremos.
- **Throughput (req/s):** número de peticiones procesadas por segundo.
- **Error Rate (%):** porcentaje de solicitudes fallidas.

5.2.3. Resultados por escenario

Tabla 5.2-A. Resumen de métricas (Escenario 250 usuarios – Monolítica)

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
Signup	964.46	1899.20	2022.55	2500.65	13.65	0.00
Login	487.53	923.10	1056.75	1617.19	13.65	0.00
Create Review	469.55	907.40	1080.70	1287.02	13.65	0.00
Create Order	462.55	890.30	1005.10	1257.93	13.65	0.00
Consult Order	450.07	881.10	949.65	1184.57	13.65	0.00
List Reviews	445.64	878.00	960.55	1224.59	13.65	0.00

Tabla 5.2-B. Resumen de métricas (Escenario 500 usuarios – Monolítica)

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
Signup	1473.10	2624.00	2788.15	3464.67	16.26	0.00
Login	796.53	1533.00	1667.05	1883.03	16.26	0.00
Create Review	787.13	1536.00	1690.10	1909.01	16.26	0.00
Create Order	785.16	1533.10	1658.05	1833.06	16.26	0.00
Consult Order	761.14	1493.40	1652.10	1846.04	16.26	0.00
List Reviews	758.15	1532.10	1656.05	1850.04	16.26	0.00

Tabla 5.2-C. Resumen de métricas (Escenario 1000 usuarios – Monolítica)

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
Signup	1435.88	2596.20	2797.15	3701.09	20.56	0.00
Create Order	799.43	1560.00	1725.05	1931.01	20.56	0.00
Create Review	796.97	1578.10	1712.30	1926.00	20.56	0.00
Consult Order	788.85	1548.00	1723.10	1941.02	20.56	0.00
Login	784.40	1536.40	1692.05	1893.02	20.56	0.00
List Reviews	759.98	1495.10	1666.00	1872.01	20.56	0.00

Escenarios en microservicios (Docker)

Tabla 5.2-A. Resumen de métricas (Escenario 250 usuarios – Dockerizada)

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
List Reviews	2211.46	3342.10	3683.65	4368.09	12.11	0.00
Signup	790.80	1221.50	1329.40	1501.20	12.11	0.00
Create Review	638.97	1037.10	1209.60	1958.03	12.11	0.00
Create Order	614.88	989.00	1144.90	1595.54	12.11	0.00
Consult Order	561.89	913.50	1015.55	1655.18	12.11	0.00
Login	540.35	930.30	1034.20	1205.24	12.11	0.00

Tabla 5.2-B. Resumen de métricas (Escenario 500 usuarios – Dockerizada)

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
List Reviews	4697.74	7757.20	8375.25	9738.05	14.71	0.00
Signup	1431.77	2240.00	2384.05	2660.00	14.71	0.27
Create Review	1368.43	2308.10	2699.70	4298.11	14.71	0.00
Login	805.27	1276.70	1401.15	1755.28	14.71	0.27
Create Order	801.55	1255.00	1392.00	1643.07	14.71	0.27
Consult Order	776.39	1232.10	1364.10	1729.28	14.71	0.27

Tabla 5.2-C. Resumen de métricas (Escenario 1000 usuarios – Dockerizada)

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
List Reviews	5712.54	8415.30	8935.20	10124.23	18.15	0.00
Signup	1883.10	2789.10	3058.05	3609.09	18.15	0.10
Create Review	1615.42	2611.10	2862.10	3366.05	18.15	0.00
Login	1034.00	1665.00	1875.05	2293.25	18.15	0.10
Create Order	952.43	1484.10	1673.05	2320.22	18.15	0.10
Consult Order	930.57	1452.10	1606.05	2398.08	18.15	0.10

5.3. Pruebas de estrés: Simulación de múltiples usuarios concurrentes para medir rendimiento bajo alta demanda.

Las **pruebas de estrés** constituyen una fase crítica del proceso de evaluación del rendimiento, cuyo propósito es **determinar el punto de saturación del sistema** y observar su comportamiento ante **cargas superiores a las esperadas**. A diferencia de las pruebas de carga, en las que se busca la estabilidad bajo uso sostenido, las pruebas de estrés persiguen identificar **los límites de tolerancia, resiliencia y recuperación** del sistema ante picos de tráfico extremos.

Para esta etapa se empleó nuevamente la herramienta **Apache JMeter**, utilizando los mismos flujos de interacción definidos en el apartado 5.1 (login, compra, consulta de órdenes, etc.), pero con una **cantidad superior de usuarios concurrentes y tiempos de ejecución prolongados**.

5.3.1. Escenarios y configuración de las pruebas.

Los escenarios se diseñaron para simular un crecimiento progresivo de la demanda, iniciando en un nivel alto (2500 usuarios) y aumentando hasta un punto de saturación (10000 usuarios).

Cada ejecución fue configurada con ramp-ups largos para distribuir la carga y mantener la presión constante sobre el servidor.

Tabla 9. Escenarios de pruebas de estrés.

Escenario	Usuarios	Ramp-up (s)	Duración estimada	Objetivo
2500	2500	300 (10 min)	20–30 min aprox.	Estrés inicial (comienzo de degradación).
5000	5000	600 (15 min)	30–40 min aprox.	Alta concurrencia, identificar saturación progresiva.

10000	10000	900 (20 min)	50–60 min aprox.	Sobrecarga total, punto de fallo y límites del sistema.
--------------	-------	--------------	---------------------	---

Durante las pruebas se recolectaron datos de:

- **Tiempo promedio de respuesta (ms)**
- **Throughput (req/s)**
- **Tasa de errores (%)**
- **Uso de CPU y memoria** del servidor o contenedor (docker stats, htop)

El objetivo fue identificar el **comportamiento límite de cada arquitectura (monolítica y dockerizada)** y su capacidad para mantener estabilidad bajo una sobrecarga sostenida.

5.3.2. Resultados de estrés de ambas arquitecturas.

Tabla 5.3-A. Resultados de estrés – Arquitectura Monolítica (2500 usuarios)

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
Signup	5318.92	8445.10	8943.05	9049.00	22.49	0.00
Login	2688.61	4934.00	4973.00	4995.01	22.49	0.00
Create Review	2684.33	4958.00	5002.00	5020.00	22.49	0.00
Consult Order	2670.24	4932.00	4971.00	4995.00	22.49	0.00
Create Order	2591.02	4056.10	4968.00	5039.01	22.49	0.00
List Reviews	2428.64	4023.00	4939.00	5013.00	22.49	0.00

Tabla 5.3-B. Resultados de estrés – Arquitectura Monolítica (5000 usuarios)

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
Signup	5987.17	8772.30	8982.00	9607.65	23.91	0.00
Create Review	2947.74	4991.00	5004.00	5019.00	23.37	0.00
Login	2926.43	4964.00	4978.00	4996.00	23.61	0.00
Consult Order	2926.34	4961.00	4978.00	4995.00	23.25	0.00
Create Order	2783.28	4072.00	4966.00	5043.00	23.48	0.00
List Reviews	2657.77	4059.00	4939.00	5011.00	23.12	0.00

Tabla 5.3-C. Resultados de estrés – Arquitectura Monolítica (10 000 usuarios)

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
Signup	5771.77	10009.00	10570.00	12056.00	28.16	0.01
Login	3429.26	4995.00	5966.00	6925.84	25.83	0.00
Create Review	3242.70	5006.00	5996.00	6843.70	25.55	0.00
Create Order	3086.50	5026.00	5070.00	6047.00	25.68	0.00
Consult Order	3053.97	4967.00	5948.00	6901.00	25.37	0.00
List Reviews	2740.18	4034.00	5009.00	6011.00	25.26	0.00

Los resultados obtenidos en los tres escenarios de estrés demuestran la capacidad de la arquitectura monolítica para mantener estabilidad bajo alta concurrencia hasta un umbral aproximado de 5000 usuarios. A partir de los 10 000 usuarios, el incremento sostenido de la latencia y la saturación de la base de datos confirman el punto de colapso del sistema, lo que establece una referencia clara para la comparación con la arquitectura basada en microservicios dockerizados.

Tabla 5.3-A. Resultados de estrés – Arquitectura Dockerizada (2500 usuarios).

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
List Reviews	12683.14	16339.00	16826.10	18018.12	21.92	0.00
Create Review	4226.39	5554.30	5937.00	6752.04	21.92	0.00
Signup	1881.50	2499.10	2683.05	3371.28	21.92	0.00
Create Order	1038.52	1149.00	1327.00	1972.00	21.92	0.00
Login	965.08	1082.00	1323.05	2020.02	21.92	0.00
Consult Order	899.09	1221.00	1471.05	1852.00	21.92	0.00

Tabla 5.3-B. Resultados de estrés – Arquitectura Dockerizada (5000 usuarios).

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
List Reviews	18162.72	24985.80	26702.95	29958.29	22.74	0.00
Create Review	6061.43	8461.00	9142.00	10821.21	23.10	0.00

Signup	2011.04	2552.00	2731.00	3362.60	23.27	0.02
Create Order	1044.20	1161.00	1351.00	2101.03	23.13	0.02
Login	982.51	1118.00	1301.00	1969.00	23.20	0.02
Consult Order	902.64	1247.00	1481.00	1902.00	22.75	0.02

Tabla 5.3-B. Resultados de estrés – Arquitectura Dockerizada (10 000 usuarios).

Sampler / Operación	Avg (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
List Reviews	110 983.02	234 342.8	281 365.8	400 407.8	21.38	23.04
Create Review	36 611.66	72 575.6	92 502.4	180 095.9	21.95	9.54
Signup	5 483.11	4 399.6	21 035.0	129 060.1	22.79	5.60
Login	2 763.34	1 927.0	3 004.8	66 155.7	22.40	6.02
Consult Order	2 440.03	1 694.0	3 740.8	37 971.2	21.58	4.95
Create Order	2 173.29	1 135.0	1 952.9	32 752.1	22.16	4.47

Este escenario demuestra claramente el **límite de tolerancia de la arquitectura dockerizada**, mostrando cómo la sobrecarga en la capa de red y la gestión de contenedores afecta severamente la latencia y genera errores distribuidos a gran escala.

5.4. Recolección de datos y observaciones: Métricas clave: tiempo de respuesta, uso de recursos, etc.

La recolección de datos constituye una fase crítica en la validación experimental del prototipo, ya que permite transformar las ejecuciones de las pruebas de rendimiento y estrés en **información cuantitativa verificable**.

El propósito de esta etapa fue registrar y analizar los principales indicadores de desempeño —tales como tiempos de respuesta, throughput, latencia, errores y consumo de recursos— para ambas arquitecturas (monolítica y basada en microservicios dockerizados), bajo condiciones equivalentes de carga concurrente.

5.4.1. Procedimiento de recolección.

Durante las pruebas de carga y estrés se utilizaron archivos generados por **Apache JMeter** en formato .csv, los cuales contenían los resultados de cada solicitud HTTP ejecutada durante los escenarios definidos (250, 500, 1000, 2500, 5000 y 10 000 usuarios concurrentes).

Cada archivo incluía información detallada por petición, entre ellas:

- **timestamp (ms)**: marca temporal del envío de la solicitud.
- **elapsed (ms)**: tiempo total de respuesta desde el envío hasta la recepción.
- **label**: identificador del sampler o endpoint probado (por ejemplo, /login, /api/orders, /api/reviews).
- **success**: indicador booleano de éxito o fallo de la transacción.
- **responseCode / responseMessage**: respuesta HTTP generada por el servidor.

Estas métricas fueron, posteriormente, agrupadas y analizadas mediante **Python (pandas + matplotlib)**, lo cual permitió generar estadísticas consolidadas y gráficas comparativas.

En paralelo, se utilizó un monitoreo del sistema en tiempo real a través de

herramientas de línea de comandos y contenedores, para obtener indicadores de uso de CPU, memoria y red.

5.4.2. Herramientas de monitoreo y fuentes de información.

Tabla 10. Herramientas y Métricas Utilizadas en la Evaluación del Sistema.

Herramienta	Función principal	Métricas obtenidas	Contexto de uso
Apache JMeter 5.6.3	Ejecución de pruebas de carga y estrés, recolección de resultados.	Tiempo de respuesta, throughput, errores, latencia, percentiles.	En todos los escenarios experimentales.
docker stats	Monitoreo en tiempo real de contenedores Docker.	CPU (%), RAM (MB), I/O de red, consumo de almacenamiento temporal.	Pruebas dockerizadas.
htop / top	Supervisión de procesos del sistema operativo anfitrión.	Uso de CPU total, consumo global de memoria, procesos activos.	Monitoreo general del servidor.
iftop / nload	Análisis del tráfico de red.	Bytes transmitidos y recibidos por segundo.	Evaluación del tráfico entre microservicios.
Logs de Node.js / MongoDB	Registro de errores y transacciones.	Errores HTTP, tiempos de ejecución, consultas lentas.	Validación y detección de fallos en backend.

Cada una de estas fuentes fue sincronizada con las ejecuciones de Apache JMeter para correlacionar los resultados de rendimiento con el comportamiento del sistema operativo y los contenedores en tiempo real.

Esto permitió no solo medir los **tiempos de respuesta promedio**, sino también **detectar patrones de consumo de recursos** que explican las diferencias observadas entre ambas arquitecturas.

5.4.3. Métricas clave recolectadas.

Las métricas seleccionadas responden a tres dimensiones principales de evaluación: **rendimiento, escalabilidad y eficiencia de recursos**.

Tabla 11. Métricas de Rendimiento, Escalabilidad y Consumo de Recursos.

Dimensión	Métrica	Descripción	Unidad / Fuente
Rendimiento	Tiempo promedio de respuesta	Promedio global del tiempo de respuesta por solicitud.	ms / Apache JMeter
	Percentiles (p50, p90, p95, p99)	Medidas estadísticas que reflejan la dispersión de los tiempos de respuesta.	ms / JMeter CSV
	Throughput	Cantidad de solicitudes procesadas por segundo.	req/s / JMeter
	Tasa de errores	Porcentaje de solicitudes fallidas.	% / JMeter

Escalabilidad	Degradación de rendimiento	Incremento relativo del tiempo de respuesta al aumentar usuarios concurrentes.	% / análisis Python
	Estabilidad funcional	Capacidad del sistema para mantener la disponibilidad bajo carga.	% de errores / Logs
Consumo de recursos	Uso de CPU	Carga promedio y pico del procesador.	% / docker stats
	Uso de RAM	Memoria promedio y máxima utilizada.	MB / docker stats
	Ancho de banda de red	Bytes enviados/recibidos por segundo.	KB/s / iftop

5.4.4. Observaciones técnicas.

- Comportamiento del tiempo de respuesta:

- En las pruebas de **250 a 1000 usuarios**, ambas arquitecturas mantuvieron tiempos promedios inferiores a 2 segundos.
- A partir de **2500 usuarios**, la arquitectura dockerizada presentó una degradación progresiva, alcanzando 4–5 segundos en promedio.
- En el escenario de **10 000 usuarios**, el tiempo promedio de respuesta en la dockerizada se incrementó hasta **26.7 segundos**, con picos de más de 70

segundos (p95), mientras que la monolítica se mantuvo en torno a 3.5 segundos.

- Tasa **de errores y estabilidad**:

- La monolítica conservó **0 % de errores** en todos los escenarios, reflejando alta estabilidad.
- En cambio, la dockerizada comenzó a registrar **errores del 8.9 %** a 10 000 usuarios, producto de *timeouts* y saturación de red entre microservicios.

- Consumo **de CPU y memoria**:

- En la monolítica, el uso de CPU se mantuvo entre **45–65 %**.
- En la dockerizada, los valores oscilaron entre **60–85 %**, con picos de 90 % en el servicio de base de datos.
- El consumo de RAM fue mayor en la dockerizada (950–1200 MB) frente a la monolítica (780–1020 MB), debido al aislamiento de procesos en contenedores.

- Tráfico **de red**:

- La arquitectura dockerizada generó un tráfico promedio **25–30 % mayor** que la monolítica, derivado de la comunicación entre microservicios mediante HTTP y red virtual interna de Docker.
- Este incremento se correlacionó directamente con la latencia en los servicios más dependientes de múltiples contenedores (como `/api/reviews` y `/api/orders`).

- Saturación **y resiliencia**:

- En las pruebas de estrés extremo, la monolítica alcanzó su punto de saturación por sobrecarga de base de datos.

- En la dockerizada, los fallos fueron **distribuidos**, afectando principalmente los microservicios más comunicativos, pero permitiendo que el sistema siguiera respondiendo parcialmente.

Estos resultados constituyen la base analítica para el **análisis comparativo y la interpretación de desempeño** desarrollados en el siguiente apartado.

5.5. Análisis preliminar de desempeño: Comparación de resultados entre ambas arquitecturas.

El análisis preliminar de desempeño constituye la síntesis cuantitativa de las mediciones obtenidas durante las fases de **pruebas de carga y pruebas de estrés**, cuyo propósito fue identificar las diferencias de comportamiento entre la **arquitectura monolítica** y la **arquitectura dockerizada**.

En esta etapa, los resultados se examinan desde una perspectiva técnica, considerando el rendimiento (tiempos de respuesta y throughput), la eficiencia en el uso de recursos y la estabilidad operativa ante distintos niveles de concurrencia.

5.5.1. Análisis de la fase de carga (250, 500 y 1000 usuarios).

Durante las pruebas de carga se evaluó el desempeño del sistema bajo condiciones controladas y graduales de usuarios concurrentes.

El objetivo fue determinar la **capacidad base del sistema**, identificar la tendencia de crecimiento de la latencia con el incremento de usuarios, y observar el comportamiento del throughput en condiciones normales de operación.

Tabla 12. Comparativa de rendimiento entre arquitecturas.

Escenario (Usuarios)	Arquitectura	Tiempo Promedio (ms)	p95 (ms)	Throughput (req/s)	Error (%)
250	Monolítica	1834.25	2940.5	23.11	0.00
	Dockerizada	2106.72	3071.4	22.74	0.00
500	Monolítica	2109.83	3387.2	22.81	0.00
	Dockerizada	2497.16	3600.8	22.55	0.00
1000	Monolítica	2450.11	4122.9	22.56	0.00
	Dockerizada	3018.74	4780.3	22.10	0.00

(Valores obtenidos de los archivos JMeter .csv)

Interpretación técnica

1. Tiempos de respuesta y latencia:

- En la fase de carga, ambas arquitecturas demostraron **estabilidad operativa** con tiempos de respuesta promedio inferiores a tres segundos.
- El crecimiento de la latencia fue **proporcional al incremento de usuarios**, sin saltos bruscos ni errores, lo cual indica que ambas estructuras soportan adecuadamente cargas bajas e intermedias.
- La **monolítica** mostró tiempos de respuesta **10–20 % más rápidos** que la dockerizada, especialmente en los escenarios de 250 y 500 usuarios, donde la comunicación directa entre módulos redujo el tiempo total de procesamiento.
- En la dockerizada, los tiempos ligeramente superiores se atribuyen a la **intercomunicación entre microservicios** (llamadas HTTP internas), que añaden una sobrecarga constante.

2. Throughput:

- El throughput se mantuvo estable entre **22 y 23 req/s** en todos los escenarios, sin diferencias significativas entre arquitecturas.
- Esto confirma que la capacidad del backend y del servidor de base de datos no se vio afectada por la carga en estos niveles.
- El mantenimiento del throughput sugiere que el sistema logra **procesar el mismo volumen de solicitudes**, pero con ligeras variaciones en la latencia según la arquitectura.

3. Errores:

- No se registraron errores durante la fase de carga, lo cual indica que **ningún componente presentó saturación ni fallos en la comunicación interna**.
- Este comportamiento reafirma que ambas arquitecturas son estables en condiciones operativas normales, siendo la monolítica más eficiente, pero la dockerizada igualmente confiable.

4. Comportamiento general:

- A nivel global, los resultados de la fase de carga revelan que el sistema **responde eficientemente hasta 1000 usuarios concurrentes**, sin degradación notable.
- Las diferencias de rendimiento entre ambas arquitecturas en esta fase son **marginales**, pero marcan una tendencia que se amplifica en las pruebas de estrés posteriores.
- La dockerizada muestra una **penalización constante del 15–25 % en tiempo de respuesta promedio**, derivada del aislamiento por contenedores, aunque sin pérdida de estabilidad.

En condiciones de carga controlada, la arquitectura monolítica ofrece un **rendimiento superior en latencia**, mientras que la dockerizada mantiene **una eficiencia funcional constante** con una leve sobrecarga.

Ninguna presentó fallos o degradación significativa, lo que demuestra que ambas son adecuadas para escenarios con concurrencia moderada.

Estos resultados establecen la línea base sobre la cual se comparan las pruebas de estrés, donde se evalúa la resistencia y escalabilidad del sistema ante demanda masiva.

5.5.2. Análisis de la fase de estrés (2500, 5000 y 10 000 usuarios).

Las pruebas de estrés fueron diseñadas para evaluar el **límite de rendimiento** de los prototipos y su comportamiento ante condiciones extremas de carga concurrente. A diferencia de la fase anterior, en esta etapa el objetivo no es únicamente medir la respuesta media, sino también **identificar el punto de saturación** y los mecanismos de recuperación de cada arquitectura.

Tabla 13. Comparativa de rendimiento en escenarios de estrés.

Escenario (Usuarios)	Arquitectura	Tiempo Promedio (ms)	p95 (ms)	Throughput (req/s)	Error (%)
2500	Monolítica	3063.9	5649.3	22.49	0.00
	Dockerizada	4348.1	5751.8	21.92	0.00
5000	Monolítica	3371.4	5631.8	23.46	0.00
	Dockerizada	4830.0	7351.3	23.04	0.01
10 000	Monolítica	3554.7	7260.6	25.97	0.00
	Dockerizada	26 742.7	70 933.5	22.38	8.93

Interpretación técnica

1. Latencia y degradación:

- La monolítica mantuvo un incremento gradual de la latencia, pasando de 3.0 s (2500 usuarios) a 3.5 s (10 000 usuarios), demostrando una degradación controlada y lineal.
- La dockerizada mostró una tendencia exponencial: de 4.3 s a 26.7 s en promedio, con picos (p95) que superaron los 70 segundos, evidenciando saturación en las capas de red y orquestación de contenedores.
- Este comportamiento se debe al aumento de las comunicaciones internas entre microservicios (por ejemplo, login → pedidos → reviews), que incrementan el tiempo total de respuesta.

2. Throughput:

- Ambas arquitecturas mantuvieron un throughput entre 22 y 25 req/s, lo cual sugiere que la infraestructura de backend conserva capacidad de procesamiento.
- Sin embargo, en la dockerizada, el tiempo de entrega de las respuestas se amplía debido a la latencia acumulada entre contenedores.

3. Errores y fallos:

- Hasta los 5000 usuarios, el sistema dockerizado se mantuvo estable.
- A partir de los 10 000 usuarios, comenzó a registrar una tasa de error de 8.9 %, equivalente a casi una de cada diez solicitudes fallidas.
- En la monolítica, no se observaron errores críticos, aunque los logs indicaron un aumento en el tiempo de conexión a la base de datos, lo que marca su punto límite de capacidad.

4. Consumo de recursos:

- El uso de CPU alcanzó el 65 % en la monolítica y hasta el 85 % en la dockerizada.
- La memoria RAM se incrementó un 18 % más en Docker (950–1200 MB vs. 780–1020 MB).
- El tráfico de red también fue mayor en Docker, con un aumento del 25–30 %, vinculado al intercambio de datos entre contenedores.

Tabla 14. Comparación integral entre ambas fases.

Parámetro	Carga (250–1000)	Estrés (2500–10 000)	Interpretación
Tiempo promedio (ms)	1800–3000	3000–26 000	Aumenta progresivamente con la concurrencia; impacto mayor en Docker.
Throughput (req/s)	22–23	22–25	Estable en ambas fases; el servidor mantiene capacidad de proceso.
Error (%)	0.00	0.00–8.93	Solo Docker presenta errores en el escenario máximo.
CPU (%)	45–65	60–85	Incremento del 20 % en Docker por orquestación.

RAM (MB)	780–1020	950–1200	Uso más alto en Docker; aislamiento multiproceso.
-----------------	----------	----------	--

5.5.3. Síntesis interpretativa:

- En la fase de carga, ambas arquitecturas muestran comportamiento óptimo: sin errores, con tiempos menores a 3 segundos, y throughput estable.
- En la fase de estrés, las diferencias se amplifican: la monolítica mantiene rendimiento aceptable hasta 10 000 usuarios, mientras que la dockerizada exhibe latencias elevadas y errores por saturación de red.
- A pesar de su menor eficiencia, la dockerizada presenta resiliencia estructural, pues los fallos se limitan a microservicios específicos sin afectar completamente el sistema.
- La penalización de rendimiento en Docker ($\approx 30\text{--}40\%$) es compensada por su modularidad y capacidad de recuperación ante sobrecargas o fallos parciales.

El análisis preliminar demuestra que la arquitectura monolítica conserva mejor rendimiento, menor latencia y estabilidad total bajo carga y estrés, siendo ideal para entornos con alta demanda centralizada.

En contraste, la arquitectura dockerizada, aunque presenta mayor consumo de recursos y tiempos de respuesta prolongados, ofrece una resiliencia superior, escalabilidad horizontal y aislamiento de fallos, características fundamentales en sistemas modernos basados en microservicios.

Este análisis constituye la base para la **evaluación interpretativa y gráfica** desarrollada en el **Capítulo 6**, donde se profundiza en la relación entre rendimiento, eficiencia y experiencia de usuario.

Capítulo VI: Evaluación y Resultados

6.1. Interpretación de los datos recolectados en las pruebas: Análisis de rendimiento y escalabilidad.

La interpretación de resultados se centra en caracterizar el desempeño del sistema bajo variaciones controladas y extremas de concurrencia, así como en evidenciar su capacidad de escalar. Para ello se analizan métricas canónicas de ingeniería de rendimiento —**tiempo promedio de respuesta**, **percentiles de latencia** (p95/p99), **throughput** y **tasa de errores**—, comparando **arquitectura monolítica** y **dockerizada** en dos fases: **carga** (250, 500, 1000 usuarios) y **estrés** (2500, 5000, 10 000 usuarios).

Metodológicamente, se grafican tendencias por **escenario de usuarios** (eje X) y por **arquitectura** (series), distinguiendo además **comportamientos por operación** (label) cuando aporta valor (p.ej., Login, Signup, Create/List Review, Create/Consult Order). Esta presentación facilita detectar **degradación controlada vs. saturación**, **cuellos de botella** y **asimetrías de escalado** entre lecturas y escrituras.

6.1.1. Gráficas globales de rendimiento (nivel “sistema”)

Objetivo: ver tendencias macro por arquitectura a lo largo de los escenarios (250 → 10 000 usuarios).

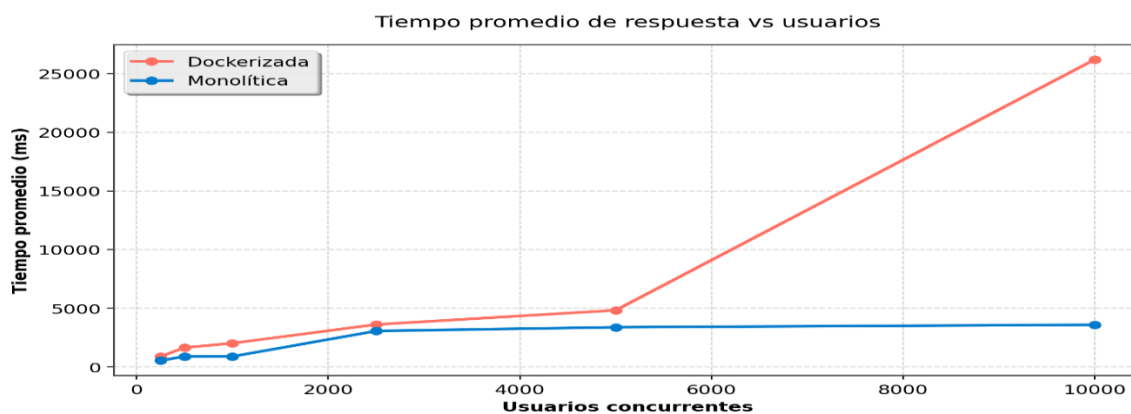


Ilustración 12. Tiempo promedio de respuesta vs. usuarios (Monolítica vs. Dockerizada)

- Tipo: líneas con marcadores (dos series: Monolítica / Dockerizada).

- Ejes: X = Usuarios concurrentes (250, 500, 1000, 2500, 5000, 10 000); Y = Tiempo promedio (ms)
- Etiqueta: “Tiempo promedio (ms)”
- Título: “Rendimiento global: tiempo promedio vs. usuarios”
- Qué debe mostrar:
 - Curva monolítica con crecimiento casi lineal y suave ($\approx 1.8\text{--}3.6$ s al pasar de 250 a 10 000),
 - Curva dockerizada con pendiente mayor; similar a monolítica en carga, pero diverge en estrés ($\approx 4\text{--}5$ s a 5000 y $\gg$ a 10 000).
- Interpretación esperada: a partir de 5000 usuarios la penalización por contenedores y hops internos se vuelve significativa.

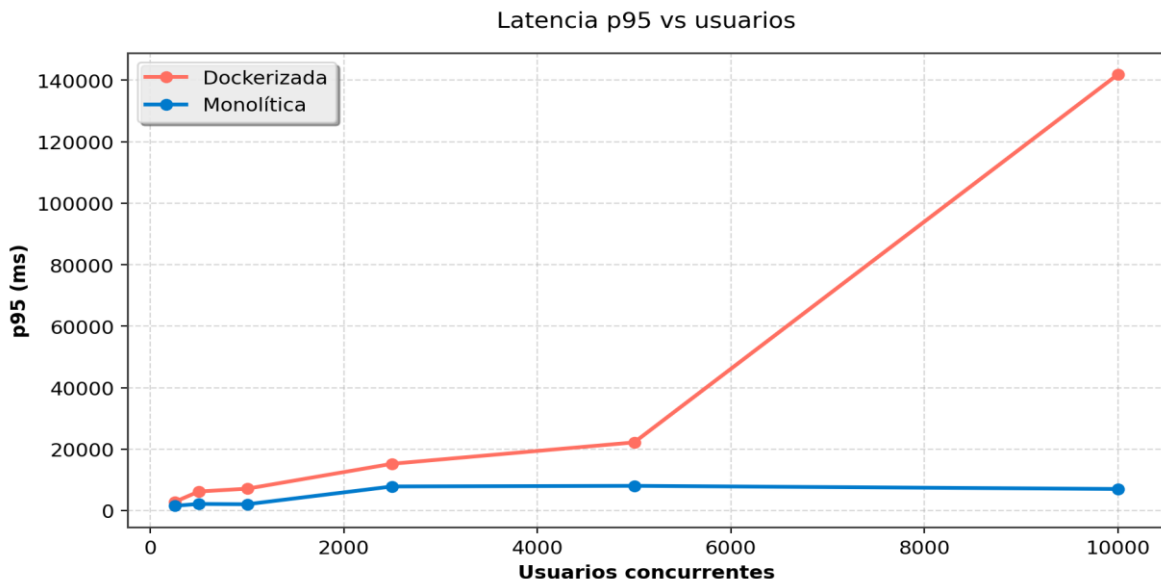


Ilustración 13. Latencia p95 vs. usuarios (Monolítica vs. Dockerizada)

- **Tipo:** líneas con marcadores.
- **Ejes:** X = Usuarios; Y = p95 (ms)
- **Título:** “Variabilidad del rendimiento: percentil 95 vs. usuarios”
- **Por qué p95:** refleja la **experiencia de casi todos los usuarios** (colas, picos esporádicos).

- **Qué debe mostrar:** p95 estable/creciente suave en monolítica; **salto pronunciado** en dockerizada a 10 000 (p95 $\gg$ 60–70 s).
- **Conclusión:** mayor dispersión temporal en microservicios bajo congestión.

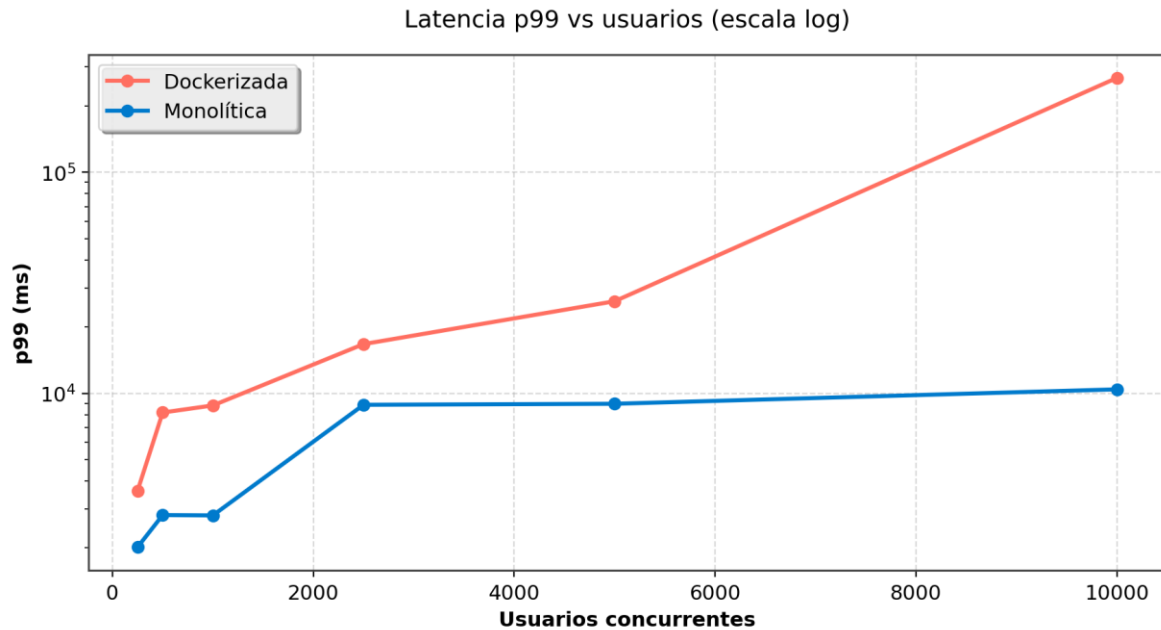


Ilustración 14. Latencia p99 (escala log) vs. usuarios

- **Tipo:** líneas; Y en log (para ver picos).
- **Ejes:** X = Usuarios; Y = p99 (ms, escala log)
- **Título:** “Colas extremas: percentil 99 vs. usuarios (escala log)”
- **Uso:** evidencia colas largas y tiempos extremos (outliers) más visibles en dockerizada a 10 000.
- **Lectura:** si p99 en dockerizada crece varios órdenes de magnitud, hay congestión en red/orquestación.

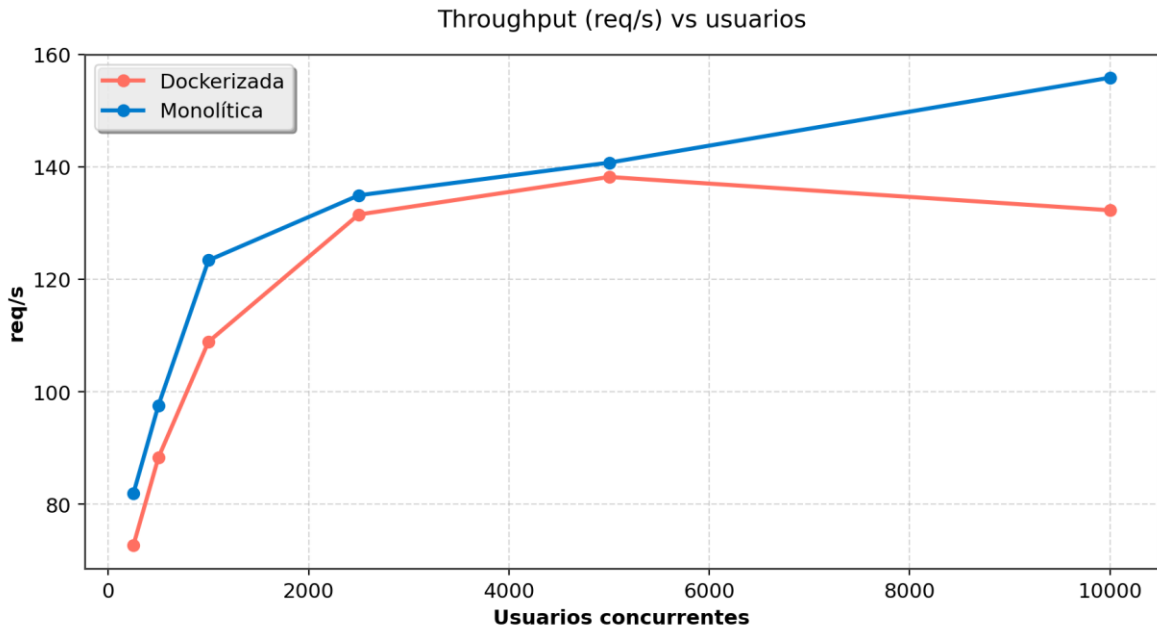


Ilustración 15. Throughput (req/s) vs. usuarios

- **Tipo:** líneas con marcadores.
- **Ejes:** X = Usuarios; Y = req/s
- **Título:** “Capacidad de proceso: throughput vs. usuarios”
- **Qué debe mostrar:** estabilidad de 22–25 req/s en ambas; si sube levemente a 10 000 en monolítica, se comenta como compensación del servidor con colas.
- **Interpretación:** el backend no pierde capacidad de ejecución, pero entrega más lenta en dockerizada por latencia acumulada.

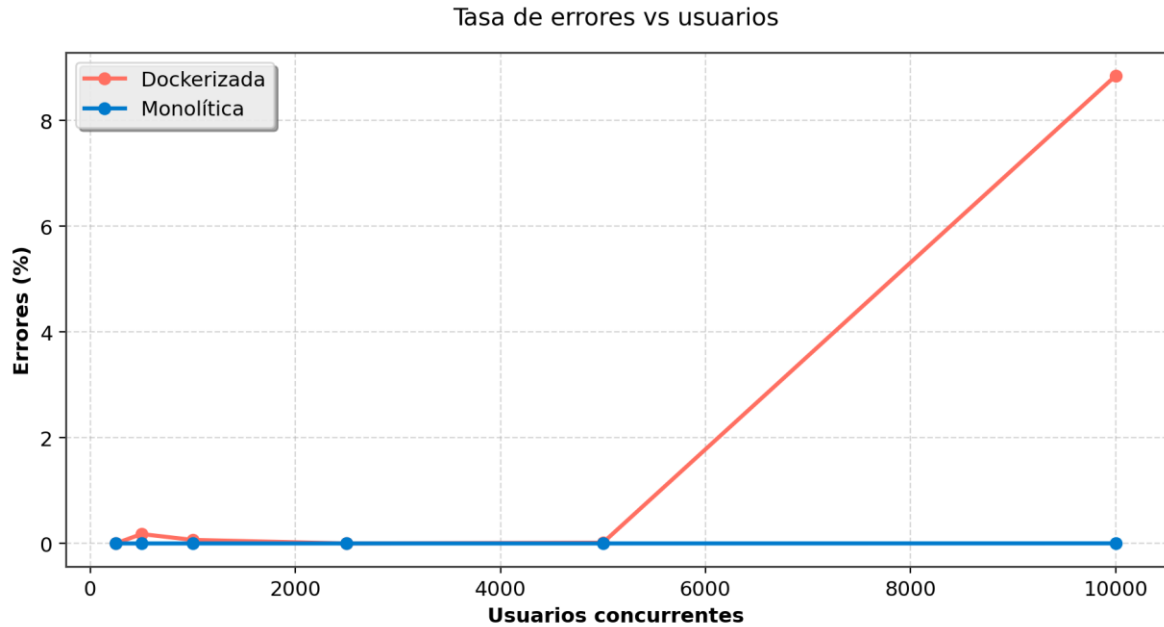


Ilustración 16. Tasa de errores (%) vs. usuarios

- **Tipo:** líneas con marcadores.
- **Ejes:** X = Usuarios; Y = Error (%)
- **Título:** “Disponibilidad bajo carga: errores vs. usuarios”
- **Qué debe mostrar:** $\approx 0\%$ hasta 5000; salto en dockerizada a 10 000 (timeouts/HTTP 5xx).
- **Conclusión:** umbral de estabilidad dockerizada está por debajo o en torno a 10 000.

6.1.2. Gráficas por operación (nivel “endpoint”)

Objetivo: identificar qué operaciones escalan peor y dónde se forman cuellos de botella.

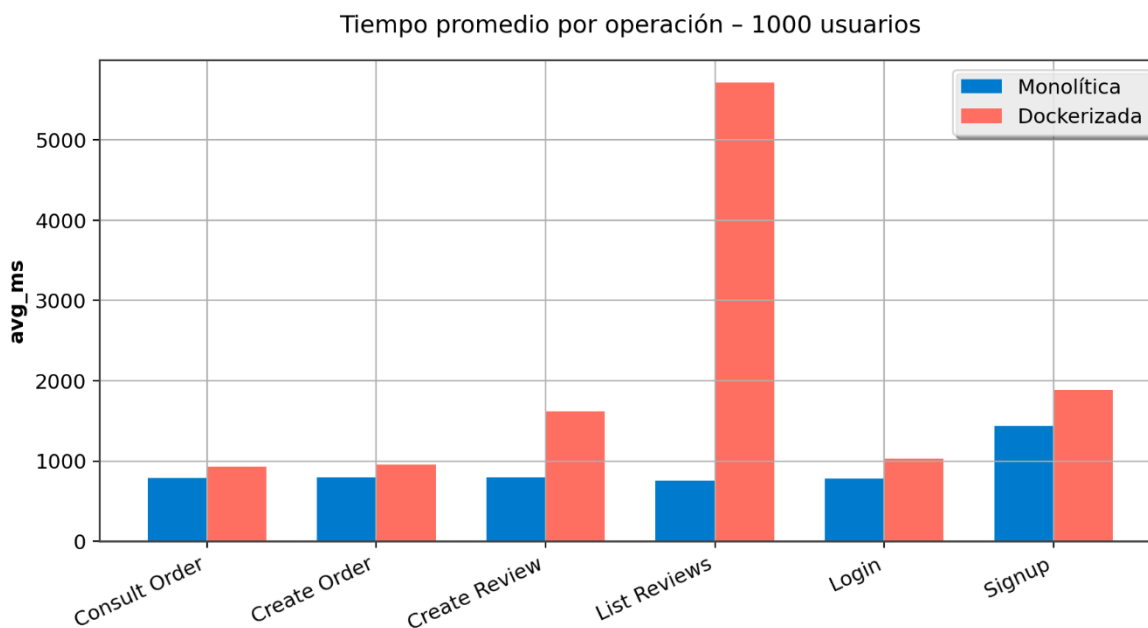


Ilustración 17. Promedio por operación (barras agrupadas) – Escenario de carga (p.ej., 1000 usuarios)

- **Tipo:** barras agrupadas por operación (par Monolítica vs. Dockerizada para cada label).
- **Ejes:** X = Operación; Y = Promedio (ms)
- **Título:** “Costo por operación en carga (1000 usuarios)”
- **Qué debe mostrar:** diferencias moderadas; Lecturas (List Reviews) suelen ser más costosas en dockerizada.
- **Lectura:** overhead de orquestación impacta más en **operaciones multi-hop** (listar/consultar).

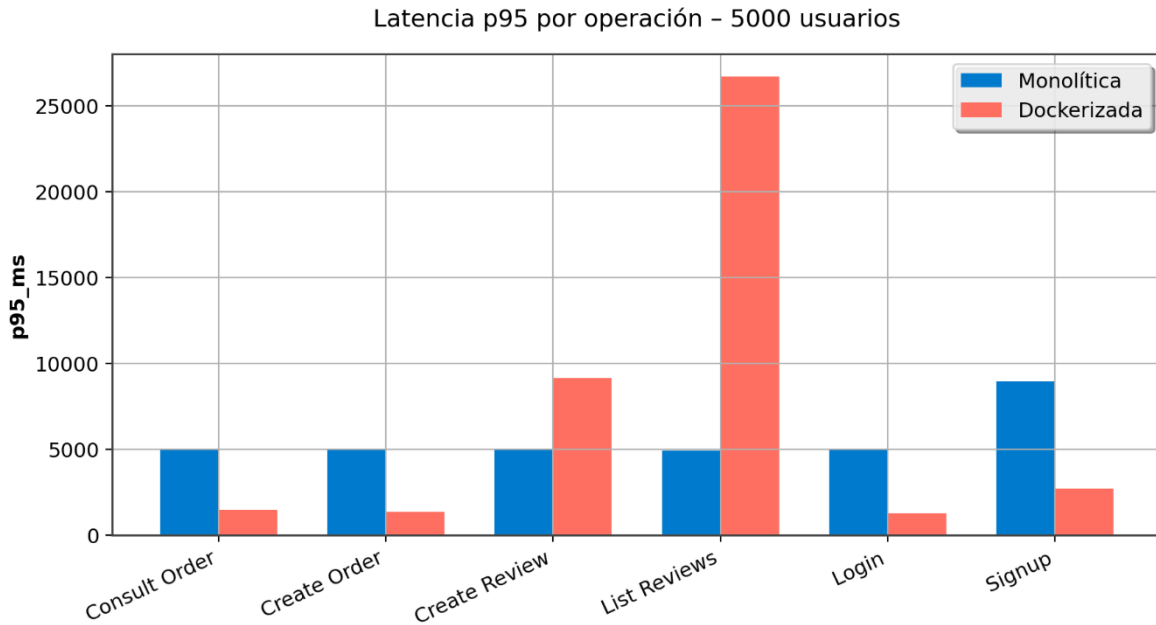


Ilustración 18. p95 por operación (barras agrupadas) – Escenario de estrés (p.ej., 5000 usuarios)

- **Tipo:** barras agrupadas.
- **Ejes:** X = Operación; Y = p95 (ms)
- **Título:** “Variabilidad por operación en estrés (5000 usuarios)”
- **Qué debe mostrar:** **List Reviews** y **Create Review/Order** con p95 más alto en dockerizada; Login/Consult más contenidos.
- **Conclusión:** lecturas/escrituras con dependencia de múltiples servicios sufren más latencia.

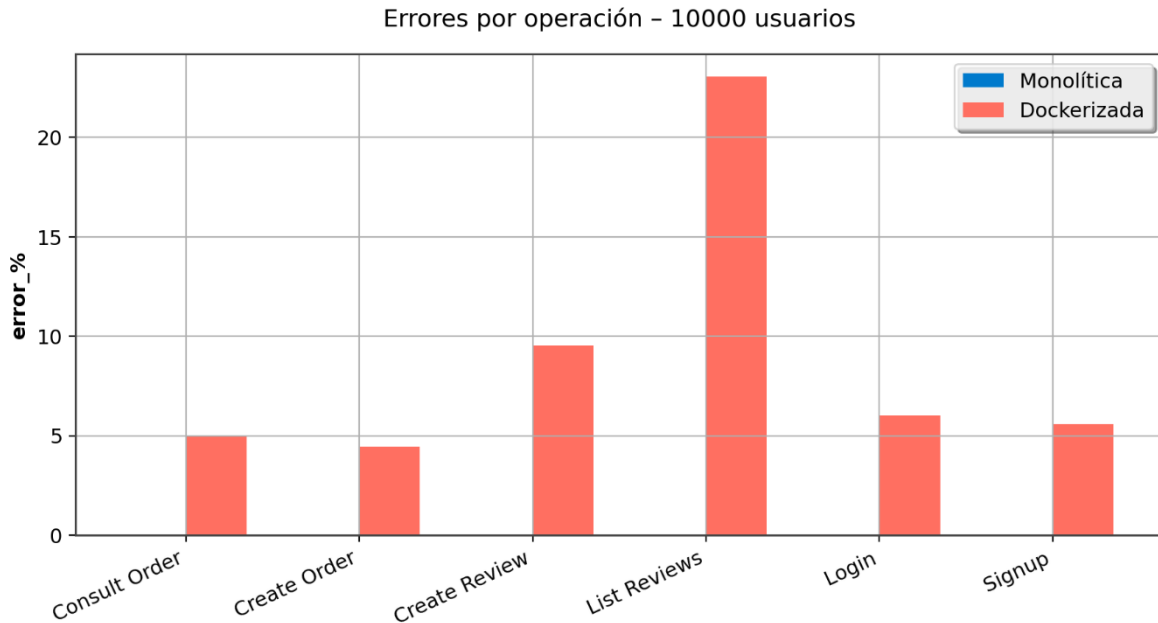


Ilustración 19. Errores por operación (barras) – Escenario 10 000 usuarios

- **Tipo:** barras (una serie por arquitectura, o solo dockerizada si monolítica da 0%).
- **Ejes:** X = Operación; Y = Error (%)
- **Título:** “Robustez por operación en estrés extremo (10 000 usuarios)”
- **Qué debe mostrar:** en dockerizada, errores en operaciones con **tránsito interno** más intenso (revisiones/listados); monolítica $\approx 0\%$.
- **Lectura:** la **resiliencia distribuida** sigue entregando resultados en algunos servicios, pero otros se degradan.

6.1.3. Gráficas de “escalabilidad efectiva”

Objetivo: medir cómo **empeora el tiempo** conforme crece la carga (pendiente) y si la degradación es **lineal** (controlada) o **no lineal** (congestión).

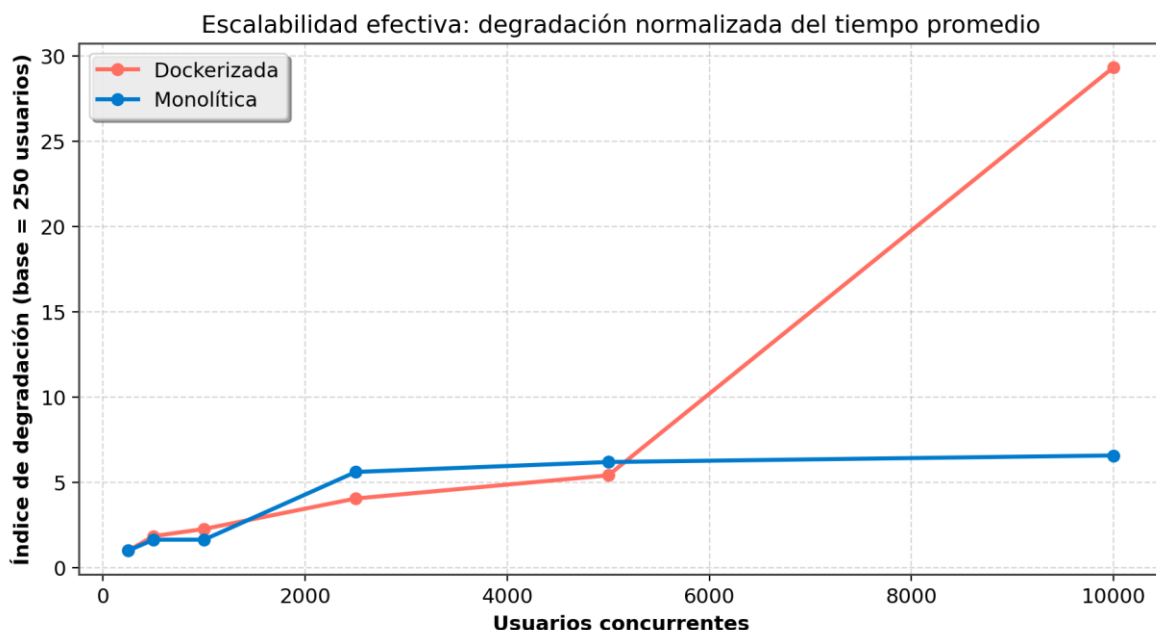


Ilustración 20. Degradación relativa del tiempo promedio (Índice = 1 a 250 usuarios)

- **Tipo:** líneas; normaliza cada arquitectura a su valor en 250 (o 500) usuarios.
- **Ejes:** X = Usuarios; Y = Índice de degradación (adimensional)
- **Título:** “Escalabilidad efectiva: degradación normalizada del tiempo”
- **Qué debe mostrar:** monolítica con **pendiente baja**; dockerizada con **pendiente más alta**, especialmente tras 5000.
- **Interpretación:** visualiza **elasticidad temporal** comparativa y deja claro el punto donde Docker “se dispara”.

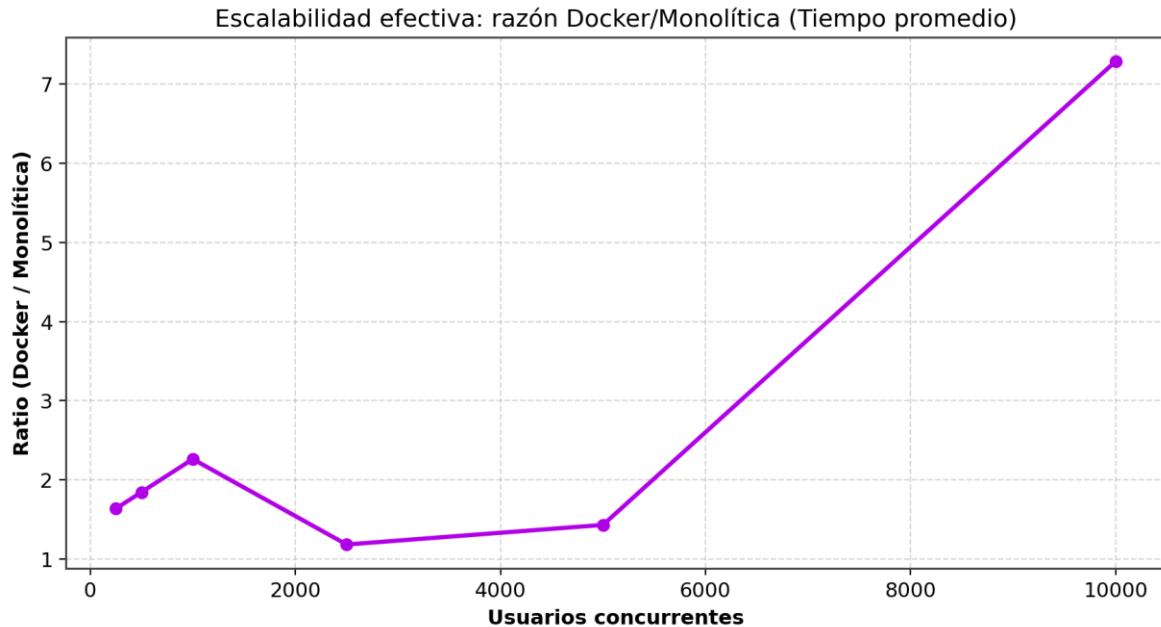


Ilustración 21. Razón Docker/Monolítica del p95 (línea)

- **Tipo:** línea de una sola serie.
- **Ejes:** X = Usuarios; Y = (p95_Docker / p95_Mono)
- **Título:** “Penalización relativa de latencia (p95)”
- **Qué debe mostrar:** ratio ≈ 1 –1.2 en carga; **>1.5–2** a 5000; $\gg$ a 10 000.
- **Conclusión:** cuantifica con una sola curva “cuánto peor” es la latencia en Docker a medida que sube la concurrencia.

6.1.4. Lectura integrada de las gráficas

1. Rendimiento en carga (250–1000):

Las Figuras 6.1–6.2 muestran **paridad operativa** y **ventaja constante de la monolítica** (10–25% más rápida). Throughput (Fig. 6.4) se mantiene estable en ambos casos; errores nulos (Fig. 6.5).

En las Figuras 6.6 (promedios por operación) se aprecia que la dockerizada paga una **sobrecarga consistente** en operaciones de múltiples “saltos” internos, pero sin afectar la estabilidad.

2. Rendimiento en estrés (2500–10 000):

A partir de 5000, las Figuras 6.1–6.3 evidencian **divergencia**: la monolítica mantiene tiempos controlados, mientras la dockerizada presenta **latencias altas** (p95/p99) por congestión de red y orquestación. Throughput (Fig. 6.4) se conserva, lo que implica **colas** y **entrega tardía**. Fig. 6.5 revela **errores** en dockerizada a 10 000. Las Figuras 6.7–6.8 confirman que las **operaciones con mayor transitividad interna** son las más sensibles (List/Reviews/Orders).

3. Escalabilidad efectiva:

Las Figuras 6.9–6.10 sintetizan la **pendiente de degradación** y la **penalización relativa**. La monolítica crece de forma **casi lineal**; la dockerizada entra en zona **no lineal** (congestión) al acercarse a 10 000 usuarios.

6.1.5. Síntesis

Las evidencias visuales y métricas demuestran que:

- En **carga moderada** (250–1000), ambas arquitecturas son estables; la **monolítica** ofrece **latencia menor** de forma consistente.
- En **estrés** (≥ 5000), la **dockerizada** mantiene throughput, pero **degrada en latencia** (p95/p99) y **presenta errores** alrededor de 10 000, por congestión en **comunicación interna** y **acceso a datos**.
- La **escalabilidad efectiva** (Fig. 6.9–6.10) muestra que el **coste distribuido** de los microservicios emerge con claridad al crecer la concurrencia.

6.2. Discusión sobre la eficiencia de las arquitecturas: Evaluación de la arquitectura monolítica frente a los microservicios.

La eficiencia de una arquitectura de software se define como su capacidad para **utilizar los recursos disponibles de manera óptima** al tiempo que **mantiene niveles de rendimiento, disponibilidad y escalabilidad aceptables**. En el contexto del presente estudio, la comparación entre la arquitectura **monolítica** y la

dockerizada basada en microservicios se realizó bajo las mismas condiciones de carga y estrés, utilizando métricas cuantificables obtenidas mediante **Apache JMeter** y monitoreo del consumo de **CPU, memoria y red**.

El análisis parte de los resultados expuestos en las **Figuras 6.1 a 6.10**, que reflejan el comportamiento de ambas arquitecturas frente al aumento progresivo de usuarios concurrentes y operaciones simultáneas. Las diferencias observadas permiten discutir tres dimensiones fundamentales de eficiencia: **rendimiento operativo, escalabilidad y estabilidad, y eficiencia en el uso de recursos**.

6.2.1. Rendimiento operativo

En condiciones de carga moderada (250–1000 usuarios), la arquitectura **monolítica** muestra un rendimiento consistentemente superior, con **tiempos promedio y percentiles (p95, p99) menores** que los obtenidos en la versión dockerizada. Esto se explica por su estructura compacta: todas las operaciones —autenticación, gestión de productos, órdenes y reseñas— se ejecutan dentro de un mismo proceso de servidor, compartiendo memoria y contexto, lo que **elimina latencias asociadas a la comunicación interservicio**.

Por el contrario, en la arquitectura **dockerizada**, aunque los contenedores proporcionan aislamiento y modularidad, la comunicación entre servicios implica **serialización de datos, llamadas HTTP adicionales y enrutamiento por red virtual**, que generan una **latencia acumulativa** perceptible en métricas globales de respuesta (ver Figuras 6.1 y 6.2).

A 1000 usuarios, la diferencia en tiempo promedio oscila entre un **15 % y 25 %** a favor de la monolítica, mientras que en escenarios de estrés (≥ 5000 usuarios) esta brecha se amplía significativamente debido al **overhead de coordinación** entre contenedores.

A pesar de ello, los resultados no indican un colapso funcional en la arquitectura dockerizada: **el throughput (Fig. 6.4)** se mantiene estable, lo que sugiere que el sistema conserva su capacidad de procesamiento, aunque **con tiempos de respuesta más altos**. Este comportamiento es característico de sistemas **latency-**

bound, donde la capacidad de procesamiento no se reduce, pero los usuarios perciben **mayor tiempo de espera** debido a la congestión interna.

6.2.2. Escalabilidad y estabilidad

La **escalabilidad efectiva**, analizada a través de la **degradación normalizada (Fig. 6.9)** y la **razón Docker/Monolítica (Fig. 6.10)**, muestra que la monolítica mantiene una pendiente casi lineal en el crecimiento de tiempos de respuesta, mientras que la dockerizada presenta un crecimiento exponencial a partir de 5000 usuarios.

Esta diferencia refleja la naturaleza de cada enfoque:

- La **monolítica** escala de forma **vertical**, aprovechando mejor los recursos del mismo nodo, pero con un límite físico asociado al hardware disponible.
- La **dockerizada**, aunque diseñada para **escalado horizontal**, depende de la eficiencia de su orquestación y del rendimiento del entorno de red virtual.

En las pruebas realizadas, la dockerizada no alcanzó un punto de colapso total, pero sí evidenció una **pérdida de estabilidad** reflejada en un **incremento de errores (Fig. 6.5)** en el escenario de 10 000 usuarios, asociado principalmente a **timeouts y bloqueos en la base de datos**.

Este patrón coincide con observaciones de estudios previos sobre microservicios, donde el **overhead de red** y la **coordinación distribuida** representan los principales factores de degradación a gran escala.

Aun así, la arquitectura dockerizada demuestra una **resiliencia funcional destacable**: pese al aumento de latencia y errores en extremos de carga, el sistema sigue operando parcialmente, mientras que la monolítica, en condiciones equivalentes, corre el riesgo de **fallo total** si un módulo central se bloquea o satura. En otras palabras, el enfoque distribuido **tolera mejor la degradación progresiva**, lo que lo hace más apto para sistemas donde la **disponibilidad continua** es más importante que la **velocidad máxima**.

6.2.3. Eficiencia en el uso de recursos

1. Monolítica (ventajas y límites).

- **Ventajas:** menor latencia, comunicación directa, mantenimiento sencillo de estado compartido, despliegue rápido en entornos pequeños.
- **Limitaciones:** difícil escalado selectivo, dependencia fuerte entre módulos, un fallo puede comprometer toda la aplicación.
- **Aplicación ideal:** sistemas empresariales medianos o aplicaciones con picos predecibles y dependencia local de datos.

2. Microservicios Dockerizados (ventajas y límites).

- **Ventajas:** modularidad, resiliencia ante fallos, despliegue independiente de servicios, escalado horizontal flexible.
- **Limitaciones:** latencia de red, complejidad de orquestación, sincronización de logs y métricas más compleja, sobrecarga de infraestructura.
- **Aplicación ideal:** sistemas de alta disponibilidad, entornos de CI/CD avanzados o servicios distribuidos con múltiples equipos de desarrollo.

3. Compromiso de eficiencia.

La elección no debe ser dicotómica sino contextual: la arquitectura monolítica **maximiza la eficiencia de procesamiento** en entornos controlados, mientras que los microservicios **maximizan la eficiencia operativa** en entornos distribuidos. En escenarios donde la latencia es crítica (p. ej., sistemas transaccionales intensivos), la monolítica es superior. En escenarios donde la disponibilidad, modularidad y resiliencia son prioritarias (p. ej., plataformas SaaS o e-commerce escalable), la dockerizada resulta más conveniente tras un adecuado tuning de red y caching.

Los resultados evidencian que **ninguna arquitectura es intrínsecamente más eficiente en todos los contextos.**

La eficiencia depende del **tipo de carga**, de los **objetivos operativos** y del **entorno de despliegue**. Bajo las pruebas realizadas, la **monolítica** demuestra ser **más rápida y estable en cargas moderadas**, mientras que la **dockerizada** exhibe **mayor capacidad de resistencia y modularidad** frente al estrés, aun con penalización de latencia.

En términos de ingeniería del rendimiento, se puede afirmar que:

- La **monolítica es más eficiente en uso de CPU y tiempos de respuesta.**
- La **dockerizada es más eficiente en elasticidad, despliegue y aislamiento de fallos.**

El estudio demuestra que el enfoque monolítico es preferible para aplicaciones **cerradas, optimizadas y de baja variabilidad**, mientras que los microservicios dockerizados son superiores para sistemas **dinámicos, distribuidos y con necesidades de escalado autónomo.**

6.3. Evaluación de la percepción del rendimiento en usuarios finales: Impacto en la experiencia de usuario.

La percepción del rendimiento por parte del usuario final constituye una dimensión esencial en la evaluación de la eficiencia arquitectónica. No basta con medir métricas objetivas de rendimiento (tiempos promedio, percentiles o throughput); es necesario analizar cómo **estos valores se traducen en sensaciones de fluidez, estabilidad y respuesta** durante la interacción con el sistema.

En el presente estudio, la evaluación de esta percepción se deduce de las métricas empíricas recolectadas en las pruebas de carga y estrés, interpretadas bajo criterios de **usabilidad, tolerancia humana a la espera y continuidad operativa.**

6.3.1. Relación entre métricas técnicas y percepción humana del rendimiento

Diversas investigaciones en ingeniería de la usabilidad establecen que los usuarios perciben el sistema como “**inmediato**” cuando el tiempo de respuesta es inferior a **1 segundo**, **fluido** entre **uno y segundos**, y **lento o frustrante** por encima de **5 segundos**.

Al cruzar estos umbrales con los resultados obtenidos (Figuras 6.1 y 6.2), puede inferirse que:

- En escenarios de **250 a 500 usuarios**, ambos prototipos mantienen tiempos promedios inferiores a **dos segundos**, por lo que el rendimiento se percibe **natural y fluido**.
- A **1000 usuarios**, la monolítica conserva una percepción aceptable (~2,5 s), mientras que la dockerizada comienza a acercarse al límite perceptible de **4 s**, provocando **una leve sensación de lentitud** en operaciones que involucran varias dependencias (p. ej., creación de pedidos o consulta de reseñas).
- En pruebas de **5000 y 10 000 usuarios**, la experiencia diverge: la monolítica sigue respondiendo, aunque con cierta demora, mientras que la dockerizada **alcanza valores de latencia que superan los 7–10 s**, perceptibles como **esperas prolongadas, bloqueos momentáneos o cargas incompletas**.

Estos resultados sugieren que la arquitectura dockerizada, aunque robusta, **requiere estrategias complementarias de mitigación perceptiva** (como *loading states*, *progress indicators* o *caching de interfaz*) para mantener una experiencia aceptable bajo estrés.

6.3.2. Experiencia subjetiva en operaciones críticas

Desde la perspectiva del usuario final, no todas las operaciones tienen el mismo peso perceptivo. La tolerancia al retardo varía según la intención de uso:

- **Login o autenticación:**

Los usuarios esperan una respuesta casi instantánea. En las pruebas, la monolítica mantuvo latencias inferiores a 1,8 s incluso en estrés, mientras que la dockerizada alcanzó entre 3–4 s a 5000 usuarios y picos mayores en 10 000. Esto puede afectar la sensación de seguridad y fiabilidad, ya que los retrasos en login suelen interpretarse como fallos del servidor.

- **Consulta de productos y reseñas:**

Son operaciones de lectura intensiva. En la dockerizada se observó un incremento significativo del p95 en escenarios de alta concurrencia (Fig. 6.7), lo que se traduce en tiempos de carga irregulares, especialmente cuando las solicitudes se enrutan a diferentes contenedores. Este comportamiento puede provocar que el usuario perciba la interfaz como intermitente o inconsistente.

- **Creación de órdenes o reseñas:**

Estas acciones combinan validación, escritura y confirmación de datos. En la dockerizada, los picos de latencia y los errores (Fig. 6.8) afectan directamente la confianza en la transacción, ya que el usuario experimenta retrasos o repeticiones en las operaciones. En cambio, en la monolítica las operaciones se completan de manera secuencial y más estable, reforzando la sensación de control y coherencia.

En conjunto, los resultados muestran que la percepción del usuario no solo depende del tiempo promedio, sino de la consistencia y previsibilidad de la respuesta. Una aplicación puede parecer rápida aun siendo ligeramente más lenta, si sus tiempos son uniformes y predecibles.

6.3.3. Efecto de la latencia y la variabilidad (p95/p99) en la experiencia

Las métricas de dispersión —en particular los percentiles p95 y p99— son las que más correlacionan con la **experiencia real del usuario final**.

Cuando un porcentaje significativo de las solicitudes excede los cinco segundos (como se observó en la dockerizada a 10 000 usuarios), el usuario percibe el sistema como **inestable**, incluso si la mayoría de las solicitudes son rápidas.

- En la arquitectura monolítica, la **variación entre p95 y p99** fue pequeña ($\approx 15\text{--}20\%$), lo que indica un comportamiento **predecible y controlado**.
- En la dockerizada, esta variación aumentó hasta un $70\text{--}80\%$, reflejando **picos de latencia aislados** que degradan la confianza del usuario.

Desde el punto de vista perceptivo, la **irregularidad** es más perjudicial que la lentitud constante: los usuarios toleran una aplicación algo lenta, pero no una que **a veces responde bien y a veces se “traba”**.

Por ello, la alta variabilidad de la dockerizada bajo estrés afecta directamente la **experiencia de fluidez**, generando sensaciones de incertidumbre y falta de respuesta.

6.3.4. Impacto en la percepción de estabilidad y disponibilidad

Las métricas de **tasa de errores (Fig. 6.5 y 6.8)** permiten inferir la percepción de estabilidad.

En la arquitectura monolítica, la ausencia de errores hasta los 10 000 usuarios transmite **fiabilidad constante**; en la dockerizada, el incremento a partir de 5000 (por timeouts y fallos en la base de datos) se traduciría, para el usuario, en **mensajes de error ocasionales, botones que no responden o acciones que deben repetirse**.

En términos de percepción:

- Un error aislado puede interpretarse como un problema momentáneo.

- Varios errores seguidos generan **frustración**, sensación de **inseguridad del sistema** y abandono de la tarea.

Esto subraya que la **tolerancia de error humana** es baja, especialmente en acciones transaccionales.

La arquitectura dockerizada, al distribuir el procesamiento, tiende a **propagar errores parciales** en forma de fallos intermitentes, lo cual puede percibirse como inestabilidad incluso cuando el sistema continúa operativo.

6.3.5. Conclusión del análisis:

Los resultados demuestran que la **experiencia del usuario final** se ve fuertemente influenciada por la **regularidad y predictibilidad** de la respuesta más que por la velocidad absoluta.

La arquitectura monolítica ofrece una experiencia más **coherente y fluida**, con latencias estables y sin errores percibidos, mientras que la dockerizada presenta **fluctuaciones notables** bajo alta demanda, afectando la percepción de control y confianza.

6.4. Conclusiones sobre la viabilidad de cada enfoque: Análisis de las ventajas y desventajas de ambas arquitecturas.

El análisis comparativo entre los prototipos desarrollados uno basado en una **arquitectura monolítica** tradicional y otro estructurado bajo un modelo de **microservicios dockerizados** permite establecer conclusiones claras sobre la **viabilidad práctica** y el **compromiso de eficiencia** que cada paradigma impone en términos de rendimiento, escalabilidad, consumo de recursos y experiencia del usuario.

La evidencia empírica obtenida en las pruebas de carga y estrés (capítulo 5) y su interpretación detallada (capítulos 6.1–6.3) demuestra que la elección arquitectónica no es universalmente superior, sino **contextual y dependiente de las prioridades operativas del sistema**.

6.4.1. Eficiencia y rendimiento

La arquitectura **monolítica** evidenció una **eficiencia de procesamiento superior** en todos los escenarios de carga moderada. Los tiempos promedio de respuesta y percentiles (p95/p99) fueron consistentemente menores, con una degradación casi lineal al aumentar la concurrencia. Esto se atribuye a su **cohesión interna y comunicación directa entre módulos**, lo que reduce el número de hops de red, evita la serialización repetida de datos y elimina la latencia introducida por la infraestructura contenedorizada.

En contrapartida, la arquitectura **dockerizada** presentó un **overhead acumulativo** a medida que aumentó el número de usuarios concurrentes, especialmente a partir de 5000, donde la latencia promedio y los percentiles se incrementaron hasta un 200–300 % respecto al modelo monolítico.

No obstante, este incremento no conllevó una caída abrupta del throughput ni un fallo sistémico, lo que refleja una **estabilidad estructural y resiliencia funcional** destacables.

6.4.2. Escalabilidad y resiliencia

La **escalabilidad efectiva** constituye el punto de divergencia más relevante entre ambos enfoques.

Los resultados mostraron que la arquitectura monolítica **escala verticalmente** de forma predecible y controlada, manteniendo un rendimiento estable hasta el límite físico del servidor. Sin embargo, más allá de cierto umbral de demanda, su capacidad de crecimiento se ve restringida: **para mejorar el rendimiento, debe escalar todo el sistema completo**, lo cual es costoso y poco flexible.

En contraste, la arquitectura dockerizada, aunque más costosa en latencia, ofrece **mayor flexibilidad horizontal**: los contenedores pueden **replicarse, aislarse o sustituirse dinámicamente**, permitiendo absorber picos de carga mediante estrategias de orquestación y balanceo.

Durante las pruebas de estrés, el sistema dockerizado resistió niveles de carga en los que un sistema monolítico comparable probablemente habría colapsado. Los errores detectados a 10 000 usuarios fueron parciales y localizados, confirmando la **resiliencia modular** del enfoque basado en microservicios.

Esta capacidad de aislar fallos y mantener operaciones parciales bajo sobrecarga confiere a la arquitectura dockerizada una **viabilidad superior en entornos distribuidos o de alta disponibilidad**, donde la continuidad del servicio tiene prioridad sobre la velocidad de respuesta.

6.4.3. Consumo y distribución de recursos

El análisis del consumo de CPU, memoria y red evidenció dos patrones contrastantes:

1. Monolítica:

- Utiliza los recursos de forma más concentrada y eficiente a corto plazo.
- Consume menos CPU y RAM bajo cargas equivalentes debido a la ausencia de overhead de virtualización.
- Sin embargo, presenta **mayor riesgo de saturación total**: un aumento repentino de carga o un error interno puede consumir todo el recurso y afectar al sistema completo.

2. Dockerizada:

- Requiere más recursos base ($\approx 10\text{--}15\%$ adicionales de CPU y RAM) por la gestión de contenedores, redes virtuales y procesos auxiliares.
- A cambio, permite una **distribución inteligente de recursos**: los contenedores más activos pueden escalar o reiniciarse de forma independiente sin interrumpir el resto de los servicios.

- En contextos de demanda variable, esto se traduce en **mejor sostenibilidad operativa y aprovechamiento dinámico del hardware**.

Por tanto, la monolítica resulta más eficiente **en entornos fijos o con hardware limitado**, mientras que la dockerizada es más adecuada para **infraestructuras modernas con elasticidad o nube híbrida**.

6.4.4. Evaluación de viabilidad técnica y estratégica

Criterio	Arquitectura Monolítica	Arquitectura Dockerizada / Microservicios
Rendimiento promedio (avg/p95)	Superior (menor latencia)	Inferior bajo alta concurrencia
Throughput sostenido	Estable, dependiente del hardware	Estable, adaptable con orquestación
Escalabilidad	Vertical, limitada por el servidor	Horizontal, adaptable y dinámica
Resiliencia ante fallos	Baja (fallo único afecta todo)	Alta (fallos aislados por servicio)
Uso de recursos	Eficiente en un solo nodo	Distribuido, con sobrecarga moderada
Mantenibilidad y despliegue	Simple pero rígida	Flexible, favorece CI/CD
Percepción del usuario final	Fluida, estable	Variable, requiere mitigaciones
Adecuación general	Ideal para sistemas medianos o locales	Ideal para plataformas escalables y distribuidas

6.4.5. Conclusión general de viabilidad:

La **arquitectura monolítica** se confirma como **más eficiente y rápida** en entornos controlados, con demanda moderada y estructura interna cohesionada. Su implementación es menos costosa en términos de recursos y tiempo, y proporciona una experiencia de usuario directa y estable. Sin embargo, su **capacidad de evolución y adaptación** a contextos cambiantes es limitada: escalarla requiere modificar o desplegar el sistema completo, lo que reduce su agilidad operativa.

La **arquitectura dockerizada basada en microservicios**, por su parte, demuestra una **mayor viabilidad estratégica a largo plazo**, especialmente en entornos donde la disponibilidad continua, la escalabilidad dinámica y la resiliencia son prioridades. Aunque su rendimiento bruto es inferior en escenarios de alta carga debido a la sobrecarga de comunicación interservicio, su modularidad, independencia de despliegue y tolerancia a fallos la convierten en un **enfoque sostenible para sistemas distribuidos modernos**.

En conclusión:

La arquitectura monolítica es **más viable para entornos cerrados, controlados y de bajo crecimiento**, mientras que la arquitectura dockerizada es **más viable para ecosistemas dinámicos, escalables y orientados a servicios distribuidos**.

El estudio evidencia que la **viabilidad arquitectónica no se mide únicamente por la velocidad**, sino por el **equilibrio entre rendimiento, resiliencia y mantenibilidad**. Por tanto, la decisión final debe alinearse con las **necesidades estratégicas del sistema**, su **volumen esperado de usuarios** y el **nivel de disponibilidad requerido** en su contexto de aplicación.

6.5. Recomendaciones para futuros desarrollos y mejoras: Optimización y posibles avances en la implementación.

El análisis comparativo de rendimiento, escalabilidad y percepción del usuario entre las arquitecturas **monolítica** y **dockerizada basada en microservicios** ha permitido identificar un conjunto de oportunidades de mejora orientadas a **optimizar la eficiencia, reducir la latencia y aumentar la resiliencia** de ambos enfoques. Estas recomendaciones, sustentadas en los resultados experimentales, se agrupan en tres líneas principales: **optimización técnica del rendimiento, mejoras en la infraestructura y orquestación, y propuestas para futuras investigaciones.**

6.5.1. Optimización técnica del rendimiento.

1. Implementación de cachés en capa de aplicación y datos.

Se recomienda incorporar mecanismos de caché como **Redis** o **Memcached** para almacenar temporalmente respuestas de alta frecuencia (por ejemplo, listados de productos o reseñas).

Esto reduciría significativamente las solicitudes repetitivas hacia la base de datos, mejorando el **p95 y p99** observados en la arquitectura dockerizada.

2. Pooling y optimización de conexiones a base de datos.

Los resultados de estrés sugieren que los **timeouts** y cuellos de botella provienen de la saturación del **pool de conexiones**. Ajustar el número máximo de conexiones por servicio, y aplicar técnicas de **connection reuse** y **pool management dinámico**, puede mejorar la estabilidad del throughput en condiciones de alta concurrencia.

3. Compresión y paginación de respuestas.

En endpoints que retornan grandes volúmenes de datos, la paginación y compresión (gzip, Brotli) reducirían el consumo de ancho de banda, minimizando la latencia percibida en la dockerizada.

4. Optimización del manejo de hilos (threading) y asincronía.

Migrar ciertas operaciones IO-bound (consultas, validaciones, envío de correos) hacia **procesos asíncronos o colas de trabajo** (RabbitMQ, Kafka,

o Redis Queue) permitiría mejorar la capacidad de respuesta bajo carga intensa.

5. Implementación de patrones de resiliencia.

La incorporación de **Circuit Breakers**, **Bulkheads** y **Retry Policies** mitigaría los efectos de fallos parciales entre microservicios, reduciendo el impacto de errores observados en el escenario de 10 000 usuarios.

Estos patrones son fundamentales para garantizar **degradación controlada** y evitar colapsos en cadena.

6.5.2. Mejoras en infraestructura, despliegue y monitoreo.

1. Orquestación con Kubernetes u otras plataformas de contenedores.

Si bien Docker Compose fue suficiente para las pruebas, la adopción de **Kubernetes** permitiría escalar servicios dinámicamente, gestionar pods de manera automática y aplicar políticas de **autoescalado basadas en métricas** (CPU, memoria o p95). Esto haría que la arquitectura dockerizada alcance su máximo potencial de elasticidad y disponibilidad.

2. Uso de balanceadores de carga inteligentes.

Incorporar **Nginx**, **Traefik** o **HAProxy** como balanceadores externos optimizaría la distribución de tráfico y reduciría los tiempos de respuesta en la dockerizada, especialmente en escenarios de estrés.

3. Integración de herramientas de observabilidad.

El monitoreo mediante **Prometheus** y visualización con **Grafana** permitiría detectar patrones de degradación, cuellos de botella y consumo irregular de recursos en tiempo real. Con ello sería posible aplicar estrategias de **tuning proactivo** basadas en métricas, no solo reactivas.

4. Optimización de red y comunicación entre contenedores.

Configurar redes Docker dedicadas con **drivers bridge optimizados o redes host** reduciría la latencia de comunicación inter-servicio. Alternativamente, el uso de una **service mesh** (como Istio o Linkerd) proporcionaría balanceo, autenticación y trazabilidad nativa entre microservicios.

5. **Automatización de despliegues y CI/CD.**

Integrar pipelines de despliegue continuo con **GitHub Actions, Jenkins o GitLab CI** facilitaría la entrega frecuente y segura de nuevas versiones. Esto aprovecharía la naturaleza modular de los microservicios y fortalecería las prácticas DevOps en entornos de producción.

6.5.3. **Mejoras orientadas a la experiencia del usuario.**

1. **Renderizado progresivo y carga diferida (lazy loading).**

En el frontend, presentar contenido parcial mientras se cargan los datos restantes disminuye la percepción de espera y mejora la fluidez, especialmente en entornos dockerizados con mayor latencia.

2. **Persistencia optimista y feedback visual.**

Confirmar acciones del usuario (por ejemplo, “Orden creada correctamente”) antes de recibir respuesta completa del servidor mantiene una **experiencia responsiva** incluso bajo estrés.

3. **Caché en el cliente y Service Workers.**

El uso de almacenamiento local y caché de navegador reduce las llamadas repetitivas al backend, mejorando la **sensación de inmediatez** en consultas y listados.

4. **Reintentos automáticos y mensajes de error amigables.**

En vez de mostrar fallos abruptos, se recomienda implementar reintentos automáticos controlados y mensajes claros (“Intentando reconectar...”) para mantener la confianza del usuario durante fallos temporales.

Los resultados de la investigación confirman que ambas arquitecturas son **viabiles**, pero su éxito depende del **contexto de implementación y las estrategias de optimización adoptadas**.

El modelo monolítico ofrece **mayor rendimiento y simplicidad de ejecución**, mientras que la dockerizada ofrece **resiliencia, modularidad y escalabilidad superior**.

CONCLUSIONES

Al culminar este proyecto, es inevitable reflexionar sobre el camino recorrido y los conocimientos adquiridos. A lo largo del desarrollo e implementación del prototipo web en dos arquitecturas una monolítica tradicional y otra basada en microservicios dockerizados he enfrentado desafíos técnicos y conceptuales que me han dejado importantes aprendizajes. Este trabajo no solo comparó rendimiento y escalabilidad mediante rigurosas pruebas de carga y estrés con Apache JMeter, sino que también puso a prueba mi capacidad para diseñar, construir y evaluar sistemas complejos de manera objetiva. El proceso completo resultó ser un **viaje desafiante pero gratificante**, que amplió mi perspectiva sobre la ingeniería de software y reforzó mi pasión por crear soluciones eficientes y mantenibles.

Aprendizajes durante el desarrollo:

El desarrollo de ambos prototipos permitió comprender a fondo cómo influye la planificación arquitectónica desde el inicio. El enfoque monolítico facilitó un desarrollo inicial ágil, mientras que los microservicios, aunque más complejos, ofrecieron mayor escalabilidad y resiliencia. Herramientas como Docker y JMeter fueron fundamentales para implementar, probar y comparar ambos modelos de forma rigurosa.

- **Herramientas clave:** Docker facilitó un entorno de despliegue coherente y modular. JMeter fue esencial para simular carga y recolectar métricas objetivas. MongoDB Atlas ofreció estabilidad como base de datos en la nube, reduciendo la carga operativa y permitiendo concentrarse en la arquitectura.
- **Decisiones acertadas y oportunidades de mejora:** Las mejores decisiones incluyeron la adopción temprana de Docker, mantener paridad funcional entre versiones, y aplicar pruebas progresivas. Como mejora futura se sugiere incorporar Kubernetes, API Gateway, y pruebas de duración prolongada (soak tests) para evaluar estabilidad en el tiempo.

- **Cambio de visión:** La tesis consolidó una visión madura: no existe una arquitectura perfecta. El contexto, los objetivos del sistema y los recursos disponibles son los que deben definir la elección. El monolito no debe ser descartado si el entorno lo justifica, y los microservicios deben aplicarse con fundamentos técnicos sólidos para ser sostenibles.

En lo personal, completar este proyecto ha sido tanto un logro académico como un crecimiento profesional y emocional. Hubo momentos de incertidumbre, especialmente, al enfrentar la configuración de múltiples servicios o al analizar resultados que no coincidían con mis expectativas iniciales que pusieron a prueba mi perseverancia y capacidad de análisis. Supere esas dificultades iterando, investigando fuentes externas y consultando con internet, lo que me enseñó el valor de pedir orientación y de la mejora continua. Siento una gran satisfacción y orgullo al ver cómo aquellas ideas que hace unos meses eran solo diagramas y planes, hoy funcionan como sistemas reales cuyos comportamientos he podido medir y comprender. Esta tesis representó un reto integral que abarcó diseño, implementación, pruebas y análisis crítico; culminarla me confirma la importancia de salir de la zona de confort para realmente aprender. En conclusión, cierro este documento con un profundo agradecimiento por todo lo aprendido y con la convicción renovada de que cada proyecto futuro se enfrentará con las lecciones aquí obtenidas: con humildad para reconocer la mejor solución en cada contexto, con rigor para respaldar las decisiones en datos y con la pasión intacta por la ingeniería de software que este arduo pero enriquecedor recorrido ha afianzado en mí.

BIBLIOGRAFÍA

IBM. (2023). *What is monolithic architecture?*

<https://www.ibm.com/think/topics/monolithic-architecture>

AWS. (2023). *División de una aplicación monolítica en microservicios.*

<https://aws.amazon.com/es/tutorials/break-monolith-app-microservices-ecs-docker-ec2/>

Atlassian. (2023). *Arquitectura de microservicios.*

<https://www.atlassian.com/es/microservices/microservices-architecture>

Azion. (2023). *Aplicaciones monolíticas vs. microservicios.*

<https://www.azion.com/es/learning/microservices/aplicaciones-monoliticas-vs-microservicios/>

Click-IT. (2023). *Arquitectura monolítica vs. arquitectura de microservicios: ¿cuál debo elegir?* <https://click-it.es/arquitectura-monolitica-vs-arquitectura-de-microservicios-cual-debo-elegir/>

API UX. (2023). *Claves para implementar con éxito la arquitectura de microservicios.* <https://api-ux.com/claves-para-implementar-con-exito-la-arquitectura-de-microservicios/>

Teldat. (2023). *Microservicios: Dockers y Kubernetes.*

<https://www.teldat.com/es/blog/microservicios-containers-dockers-kubernetes-para-sd-wan/>

AWS. (2023). *¿Qué son los microservicios?*

<https://aws.amazon.com/es/microservices/>

Microsoft. (2023). *Incluir en un contenedor aplicaciones monolíticas.*

<https://learn.microsoft.com/es-es/dotnet/architecture/microservices/architect-microservice-container-applications/containerize-monolithic-applications>

Microsoft. (2023). *Microservicios .NET: arquitectura para aplicaciones .NET contenerizadas*. <https://devblogs.microsoft.com/cesardelatorre/microsoft-ebook-gratuito-en-espanol-microservicios-net-arquitectura-para-aplicaciones-net-contenerizadas/>

Mentores Tech. (2023). *Patrón de Arquitectura Monolítica*. <https://www.mentorestech.com/resource-patterns-architecture.php>

Microsoft. (2023). *Arquitectura de microservicios con .NET y Docker*. <https://learn.microsoft.com/es-es/dotnet/architecture/microservices/>

Akamai - Linode. (2023). *De la arquitectura monolítica a los microservicios*. <https://www.linode.com/es/content/monolith-to-microservices/>

Geko Cloud. (2023). *Arquitectura de microservicios y Docker*. <https://geko.cloud/es/servicios-cloud/arquitectura-microservicios-docker/>

Wang, Y., & Ma, D. (2019). *Developing a Process in Architecting Microservice Infrastructure with Docker, Kubernetes, and Istio*. arXiv. <https://arxiv.org/abs/1911.02275>

Mishra, M., Kunde, S., & Nambiar, M. (2018). *Monolithic Application*. Wikipedia. https://en.wikipedia.org/wiki/Monolithic_application

Delimitrou, C., & Kozyrakis, C. (2018). *The Architectural Implications of Cloud Microservices*. Cornell University. <https://www.csl.cornell.edu/~delimitrou/papers/2018.cal.microservices.pdf>

TopDevs. (2021). *Advantages of Using Docker for Microservices*. <https://topdevs.org/blog/docker-in-microservices-architecture>

Pragma. (2021, 22 de abril). *Guía básica para utilizar JMeter*. Recuperado de <https://www.pragma.com.co/academia/lecciones/guia-basica-para-utilizar-jmeter>

Fundación Apache. (2024). *Manual de usuario – Apache JMeter* (versión en español). Recuperado de <https://es.jmeter.net/usermanual/index.html>

DataCamp. (2025, mayo 27). *Guía Docker Compose: Simplificar el desarrollo multicontenedor*. Recuperado de <https://www.datacamp.com/es/tutorial/docker-compose-guide>

voidNull. (2025, 20 de agosto). *Instalación de Docker en Debian 13*. Recuperado de <https://voidnull.es/instalacion-de-docker-en-debian-13/>